

RUST · SYSTEMS PROGRAMMING

Coding Interviews for Cracked Rustaceans

A Zero-Cost Abstraction Obsession Anatomy

Arpan Pathak

Contents

<i>Preface</i>	4
<i>The code</i>	8

Part I: Sequences and Maps

Part page.....	10
1. Two Sum.....	11
2. Valid Parentheses	17
3. Min Stack.....	23
4. Longest Substring Without Repeating Characters	29
5. Merge Intervals.....	33
6. Top K Frequent Elements	37

Part II: Trees and Graphs

Part page.....	41
7. Binary Tree	42
8. Binary Search Tree.....	48
9. Binary Search on a Rotated Array	55
10. Trie	59
11. Topological Sort	64

Part III: Recursion and Dynamic Programming

Part page.....	69
12. Coin Change	70
13. Subsets.....	75

Part IV: Linked Structures

Part page.....	80
14. Reverse a Linked List.....	81
15. Singly Linked List.....	86
16. Doubly Linked List	91

Part V: Caches

Part page.....	100
17. LRU Cache.....	101
18. LRU Cache, Array-Backed.....	107

Part VI: Concurrency and the Operating System

Part page.	113
19. Workload State Machine	114
20. Token Bucket Rate Limiter	121
21. Worker Pool	126
22. Mutex Poisoning	132
23. Counting Islands	139
24. Syscall Overhead	146
25. Shadowing and Mutation.	152

Part VII: Files, Directories, and Arguments

Part page.	158
26. Reading and Writing Files.	159
27. Paths and Directory Traversal	164
28. File Operations and Errors	169
29. Command-Line Arguments	173

Part VIII: Coordination and Parallelism

Part page.	179
30. A Bounded Buffer	180
31. Parallel Sum.	186

Part IX: Sequences, Revisited

Part page.	192
32. Running Median	193
33. Reversing a String In Place.	199

Part X: Reliability

Part page.	204
34. Idempotent Operations	205

Closing

Part page.	211
References	16
Index	215

Preface

What you don't use, you don't pay for. What you do use, you couldn't hand code any better.

Bjarne Stroustrup, *Foundations of C++*, ECOOP 2012

What this book is

This book reads one repository of Rust, `rust-interview-lab`, one file at a time. Each chapter covers a single file: the problem it solves, the design it uses, the code as the repository holds it, a trace of that code on a concrete input, the cost of each operation, and the inputs it does not handle.

The files are working programs and modules rather than fragments written for the page. Each is short enough to read in full, and each keeps its own tests.

The repository is at <https://github.com/arpanpathak/cracking-the-systems-programming-interview>, and every chapter links the exact source file it reads.

Who this book is for

The book assumes that you can read Rust and that you know what `Vec`, `HashMap`, `Option` and `Result` are. It assumes nothing else. Every structure the text uses is built in the text, and every cost claim is either derived on the page or measured by a program printed beside it.

Each chapter follows the same order: the problem statement, the design, the implementation, a worked example, the cost, and the limits. Read straight through, or read one chapter on its own; each stands alone.

How the chapters are built

Most chapters have the same sections.

Problem Statement. The input, the output, and the boundary cases that decide whether an answer is complete.

Designing a Solution. The mechanism the solution rests on, and the reason it is correct.

Implementation. The file the chapter reads, followed by a reading of its decisions.

Intuition. The state of the program after each step on a concrete input, as a table or a diagram.

Time and Space Complexity. The cost of each operation, with the condition stated in the same row when a bound holds only under an assumption.

Limitations. The cases the code does not handle, the assumptions it makes about its input, and the resources it does not bound.

A chapter that measures rather than traces prints the program that produced the figure, so that the number can be reproduced.

How the book is organised

The chapters are grouped by subject. Chapters 1 to 13 cover the problems of the standard interview repertoire: array and string work, trees, graphs, recursion and search. Chapters 14 to 18 cover linked structures and caches, in which the deciding question is ownership. Chapters 19 to 25 cover concurrency and the operating system, including a program that measures the cost of a system call. Chapters 26 to 29 cover files, directories and command-line arguments. Chapters 30 and 31 cover coordination and parallelism. Chapters 32 and 33 return to sequences with a running median and an in-place reversal. Chapter 34 covers idempotent operations.

Read straight through for the argument, or follow one part for a topic. The index and the cross-references in each chapter are enough to read out of order.

The repository

The code is in `rust-interview-lab`, in this repository:

<https://github.com/arpanpathak/cracking-the-systems-programming-interview>

To run it:

```
cd rust-interview-lab
cargo test                # library and program tests
cargo run --bin count_islands
cargo run --release --bin syscall_overhead
cargo run --bin mutex_poisoning
cargo run --bin shadowing
cargo run --bin read_write_file
```

```
cargo run --bin list_directory
cargo run --bin bounded_buffer
cargo run --release --bin parallel_sum
```

The listings reproduce the files as the repository holds them. Chapter 18 prints a test with an assertion added, and says so where it appears.

Conventions

A trace is a table with one row per step. Where a structure needs a picture rather than a table, boxes inside a frame are values on the stack, and boxes to the right of the frame joined by an arrow are values on the heap. A solid arrow is a pointer that owns what it points at; a dashed arrow is a pointer that does not.

```

stack                                heap
+-----+                          +-----+
| variable: value | ----->      | owned value |
+-----+                          +-----+
```

Cost tables use the usual notation. A bound that holds only under an assumption states the assumption in the same row:

Resource	Cost	Condition
Time	$O(n)$ expected	the hash function distributes keys evenly
Space	$O(n)$	up to n entries

A measured figure names the machine that produced it, because a measurement without its environment cannot be reproduced.

What this book does not cover

It is not a Rust tutorial. It does not teach the language from the beginning, and it does not survey the ecosystem. It is not a catalogue of interview questions: the subject is the reasoning around each program, not the answer to it. And it is not a Kubernetes or GPU manual; the operator and the cluster work live in the repository and in the study guides beside it, not in these chapters.

Acknowledgments

The claims about the Rust language and its standard library rest on the documentation the Rust Project Developers maintain: the *Rust Book*, the *Rust Reference*, the *Rustonomicon*, the standard library reference, and the API Guidelines. The algorithms are attributed in each chapter and listed with their sources in the References. The epigraph is Bjarne Stroustrup's statement of the zero-overhead principle, and the quotations that follow it in the chapters are attributed where they appear.

Arpan Pathak

The Code

Layout

The repository holds one Cargo package. The tree below lists the files this book reads; the package also contains concurrency, asynchronous, networking and benchmarking code that lies outside the scope of these chapters.

```
rust-interview-lab/
├── Cargo.toml
├── README.md
├── BENCHMARKS.md
├── src
│   ├── lib.rs                                the crate root and its re-exports
│   └── problems
│       ├── mod.rs                            the module list
│       ├── backtracking.rs                  Chapter 13
│       ├── binary_search.rs                Chapter 9
│       ├── binary_tree.rs                  Chapter 7
│       ├── dp.rs                           Chapter 12
│       ├── graph_topology.rs               Chapter 11
│       ├── linked_list.rs                  Chapter 14
│       ├── lru_cache.rs                    Chapter 17
│       ├── merge_intervals.rs              Chapter 5
│       ├── min_stack.rs                    Chapter 3
│       ├── rate_limiter.rs                 Chapter 20
│       ├── sliding_window.rs               Chapter 4
│       ├── state_machine.rs                Chapter 19
│       ├── top_k_frequent.rs                Chapter 6
│       ├── trie.rs                         Chapter 10
│       ├── two_sum.rs                      Chapter 1
│       ├── valid_parentheses.rs            Chapter 2
│       └── worker_pool.rs                  Chapter 21
├── bin
│   ├── append_to_file_open_options.rs      Chapter 26
│   ├── bounded_buffer.rs                   Chapter 30
│   ├── bst_clean.rs                        Chapter 8
│   ├── command_line_args.rs                Chapter 29
│   ├── copy_rename_delete_file.rs          Chapter 28
│   ├── count_islands.rs                    Chapter 23
│   ├── file_error_handling.rs              Chapter 28
│   ├── idempotent_operation.rs              Chapter 34
│   ├── idempotent_operation_with_error_propagation.rs  Chapter 34
│   ├── list_directory.rs                   Chapter 27
│   ├── ll.rs                               Chapter 16
│   ├── median_finder.rs                    Chapter 32
│   ├── mutex_poisoning.rs                  Chapter 22
│   ├── parallel_sum.rs                     Chapter 31
│   ├── path_buff.rs                        Chapter 27
│   ├── read_write_file.rs                  Chapter 26
│   ├── readfile_line_by_line.rs            Chapter 26
│   ├── recursive_directory_walk.rs          Chapter 27
│   ├── reverse_string.rs                   Chapter 33
│   ├── shadowing.rs                        Chapter 25
│   ├── singly_linked_list.rs               Chapter 15
│   └── syscall_overhead.rs                 Chapter 24
└── benchmarking_examples/                  the measured structures and the benchmark
```

Two files in `src/problems/` are empty, `hash_map.rs` and `lru_cache_easy.rs`, and nothing in the crate refers to either. Chapter 18 describes the array-backed cache those two names were intended for; that implementation now lives in `benchmarking_examples/cache/arena.rs`.

Modules and programs

Seventeen of the files above are library modules. Each holds one problem, one type, or one pair of a type and its policy, and each ends with a `#[cfg(test)] mod tests`, so the list printed by `cargo test` reads as an index of this book.

Twenty-two are programs. Files under `src/bin/` are separate crates that link the library, and each has a `main` function that runs without arguments. Most of them also carry the tests for the code they contain, so `cargo test` exercises the programs as well as the library.

What the files have in common

Four conventions run through the code. They are stated once here so that the chapters need not repeat them.

Enumerations instead of flags. Where a value has a fixed set of states, the code uses an `enum` and a `match` with no fall-through arm, so that adding a state is a compile error at every place that must handle it. `WorkloadState` in Chapter 19 and `ApiError` beside it are examples.

Option and Result instead of sentinels. Absence is `Option` and failure is `Result`, so the type states which. Chapter 12 keeps a sentinel inside the dynamic program's table and translates it at the boundary, so a caller never sees it.

Ownership written out. The linked structures in Chapters 14 to 16 give three different answers to the question of who owns the next node, namely `Box`, `Rc` with `Weak`, and a vector of indices, and each answer is visible in the type rather than in a comment.

Tests beside the code. A unit test that reaches a private item is a licence to change that item. Each module keeps its own tests in a `#[cfg(test)] mod tests`, so the list printed by `cargo test` reads as an index of the book.

Part I: Sequences and Maps

Two Sum

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/two_sum.rs`. Test it with `cargo test two_sum`.

Problem Statement

Given a slice of integers and a target value, return the indices of two positions whose values sum to the target, or report that no such pair exists. The usual statement of the problem guarantees that at most one answer exists, and permits the two positions to hold the same value as long as they are different positions.

The straightforward solution compares every element with every element after it. For a slice of length `n`, that is $n(n-1)/2$ comparisons, which is $O(n^2)$. The cost is easy to underestimate because the code is short. A slice of a million elements, an ordinary size for a batch job, would need about five hundred billion comparisons; at one comparison per nanosecond that is more than six days. The rest of this chapter reduces that to a single pass.

Designing a Solution

The useful observation is that for a given element `x` the value of its complement is not unknown. If the pair sums to a target `t`, then the complement of `x` is exactly `t - x`. That value is determined by `x` and `t` before any search takes place.

This changes the shape of the work. The naive solution asks a *search* question, "is there an element equal to `t - x` somewhere in the rest of the slice?", and every answer costs a scan of what remains. The reformulated solution asks a

membership question, "have I already seen `t - x`?", and membership is the question a hash table answers in constant expected time.

```
x      the element under the cursor
t - x  the only value that would complete a pair with x
```

Walking the slice once and recording each element as it is passed therefore suffices. When the cursor reaches the second element of a pair, the first element is already in the table and the pair can be reported. Nothing earlier in the walk could have reported that pair, because at that point only one of its two halves had been seen.

Two decisions in the representation follow from the problem statement. The table maps a value to the *position* where that value was found, because the answer is a pair of indices and not a pair of values. And the table is keyed by the element's own value rather than by its complement, because the role an element plays depends on what comes after it: the same element may be the second half of one pair and the first half of another, so it has to be findable by its own value when a later element arrives.

Implementation

```
/// Two Sum: find the pair of indices whose values add up to a target.
///
/// Use a `HashMap` for one pass:  $O(n)$  time,  $O(n)$  space.

use std::collections::HashMap;

pub fn two_sum(nums: &[i32], target: i32) -> Option<(usize, usize)> {
    let mut seen = HashMap::with_capacity(nums.len());
    for (idx, &num) in nums.iter().enumerate() {
        match seen.get(&(target - num)) {
            Some(&prev) => return Some((prev, idx)),
            None => {
                seen.insert(num, idx);
            }
        }
    }
    None
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_pair() {
        let nums = [2, 7, 11, 15];
        assert_eq!(two_sum(&nums, 9), Some((0, 1)));
    }

    #[test]
    fn returns_none_when_missing() {
```

```

    }
    assert_eq!(two_sum(&[1, 2, 3], 100), None);
}

```

The signature returns `Option<(usize, usize)>`, so the absence of a pair is part of the answer. A caller cannot mistake an index of zero for a failure, because the indices exist only inside `Some`. Both are `usize`, which is what `enumerate` produces and what a slice accepts.

The map is created with `HashMap::with_capacity(nums.len())`. Without the hint the table rehashes as it fills, moving every entry each time it grows, and the number of such movements is logarithmic in the final size. The hint is an upper bound on the entries the loop can insert, so with it the table allocates once.

The loop header, `for (idx, &num) in nums.iter().enumerate()`, reads from the inside out. `nums.iter()` yields `&i32`. `enumerate` pairs each element with its position, yielding `(usize, &i32)`. The pattern in the loop header deconstructs that pair, and the `&` in `&num` copies the integer out of the reference, so the body works with `num: i32` rather than with a borrow. Copying an `i32` is a load from the slice that the loop needed anyway.

The lookup `seen.get(&(target - num))` computes the complement and searches for it in one expression. `get` takes a reference to the key, which is why the complement is written inside `&(...)`. An intermediate binding would be equally clear, and would name a value used once. The borrow of `seen` that `get` returns ends with the `match`, and that is what allows the `None` arm to take a mutable borrow of the same map.

The pattern `Some(&prev)` copies the stored index out of the reference that `get` returned. Writing `Some(prev)` would attempt to move a `usize` out of a map the caller does not own, and the borrow checker refuses it.

Intuition

The first trace uses the slice from the test suite, where the pair is completed on the second element.

		nums = [2, 7, 11, 15]		target = 9		
step	idx	num	complement	9-num	present in the map?	map after the step
1	0	2	7		no	{2: 0}
2	1	7	2		yes, at index 0	return Some((0, 1))

The second trace covers the case the problem statement is written to permit: two positions holding the same value.

```

nums = [3, 3]          target = 6

step  idx  num  complement  6-num  present in the map?  map after the step
  1    0    3      3          3         no                {3: 0}
  2    1    3      3          3        yes, at index 0    return Some((0, 1))

The two positions are distinct, which the answer requires; the two values are
equal, which the problem statement allows.

```

A third case is worth following for what it does not do. With `nums = [1, 2, 3]` and `target = 100`, every complement is negative, no lookup succeeds, and the loop runs to the end with three entries in the map. The function then returns `None`, and the map is released when the function returns. The memory used by the search therefore depends on the input, and not on whether an answer exists.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$ expected	the hash function distributes keys evenly
Space	$O(n)$	one entry per element, when every element is distinct

The time bound is expected rather than guaranteed. A hash table computes a hash of the key, uses part of it to select a bucket, and probes from there. When keys distribute evenly the average number of probes per operation is a small constant, close to one in the standard library's implementation. When keys collide systematically, every operation inside a colliding group degenerates into a scan, and n insertions cost $O(n^2)$ in total.

The standard library's default hasher is SipHash-1-3 with a key chosen at random for each map. That is what keeps the worst case out of reach of an attacker who supplies the input: which keys collide depends on a seed the attacker cannot observe. A program that installs its own hasher, as programs that hash integers do for speed, gives up that protection, and the expected bound then becomes a claim the program's own input can break.

Space is $O(n)$ because the table holds an entry per distinct value visited, and in the worst case every element is distinct and no pair exists. For a slice of integers the table is larger than the input. A million `i32` values oc-

copy four megabytes, while a map of a million entries carries a key, a value, and the table's own bookkeeping for each one. The memory cost of the faster algorithm is the price of the membership test, and it is paid in full when the answer is absent.

Limitations

The subtraction `target - num` is arithmetic on `i32`, and it can overflow. The case is reachable with a single element: `two_sum(&[i32::MIN], 0)` evaluates `0 - i32::MIN`, and `i32::MIN` has no counterpart of the opposite sign that fits in an `i32`. In a debug build that expression panics with an arithmetic overflow. In a release build the same expression wraps, the wrapped value is a number no element can hold, and the function returns `None`. The two build profiles therefore disagree about the same input, which means a program tested in debug and shipped in release can change behaviour on a value that arrived from a file or a network. The remedy is one call:

```
let complement = match target.checked_sub(num) {
    Some(value) => value,
    None => {
        seen.insert(num, idx);
        continue;
    }
};
```

`checked_sub` reports the impossible case instead of overflowing, and the element that has no complement is still recorded, because a later element may need to find it.

The map keeps the most recent index for a repeated value, since the `None` arm writes with `insert` and an existing key is overwritten. When a pair is completed by the second occurrence of a value, the index returned is the one the first occurrence recorded, so the answer is correct. The detail matters only if the contract changes: a caller that asks for the pair whose first index is smallest, or for every pair, needs evidence that this function discards.

Only one pair is reported. The function returns as soon as the second element of any pair is reached, so on an input containing several valid pairs the answer depends on the order of the elements rather than on a property of the pairs. Two related problems look like this one and need different code. Reporting every pair requires the loop to continue and the map to

hold every index for each value. Reporting the pair closest to the front of the slice requires the earliest index to be kept, which turns the `None` arm into `seen.entry(num).or_insert(idx)`.

The element type is fixed at `i32` and the index type at `usize`. Nothing in the algorithm depends on either choice, but the signature fixes both, so a slice of `i64`, or of a type that is not `Copy`, cannot be passed without changing the declaration.

Summary

- The complement of an element is determined by the target, so the search for a complement becomes a membership test among the elements already seen. That reformulation is what makes the one-pass loop possible.
- The cost is expected $O(n)$ rather than guaranteed $O(n)$, and the assumption behind the bound is that the hash function distributes keys evenly. The worst case is $O(n^2)$ with a hasher that collides.
- `target - num` overflows on adversarial input. The expression passes every test written with small positive numbers and fails on the first input near `i32::MIN`. Chapter 9 makes the same point about the midpoint of a range.

References

- Standard library, `HashMap`, on the default hasher and its random seed.
- Standard library, `HashMap::with_capacity`.
- Standard library, `i32::checked_sub`, and `HashMap::entry` for the variant that keeps the earliest index.
- The Rust Reference, [Arithmetic overflow](#), on the difference between debug and release builds.

Valid Parentheses

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/valid_parentheses.rs`. Test it with `cargo test valid_parentheses`.

Problem Statement

Given a string built from the six delimiter characters `()[]{}` , decide whether every opening delimiter is eventually closed by its own kind, with the pairs properly nested. Any other character makes the string invalid.

The problem is the smallest useful model of a task that appears throughout programming: deciding whether a sequence is well formed. A text editor does it to highlight matching braces, a configuration loader does it before parsing a document, and a compiler does it before it can build an expression tree. The rules are what make a stack the natural tool, and the exercise is the shortest route to seeing why a counter is not enough.

Designing a Solution

Two conditions have to hold when the scan reaches the end of the string. No closing delimiter may appear when the delimiter it has to match is not the most recent unmatched one, and nothing may remain unmatched.

The second condition is easy to satisfy with a counter, and the first is not. Consider `([)]`. Every kind of delimiter appears in matching quantities, so a count of openings and closings reports balance, and the string is nevertheless invalid: the `)` closes the `(` while the `[` is still open. Correctness depends on the *order*

of the unmatched openings rather than only on their number, and a value that remembers an ordered sequence of pending openings is exactly a stack.

The stack holds the opening delimiters whose counterpart has not yet been seen, with the most recently opened on top. That is the delimiter a closing character has to match, so the scan maintains the invariant

the stack contains the unmatched opening delimiters, in the order they appeared,

after every character. An opening delimiter is pushed. A closing delimiter pops the top and checks its kind. If the stack is empty when a closing delimiter arrives, there was nothing for it to close, and the string is invalid at that point. When the scan finishes, the stack has to be empty, because anything left in it is an opening delimiter with no matching opener.

The machine this scan describes has a name. It is a pushdown automaton, the weakest kind of machine that recognises a language in which nesting has to be matched. Its counterpart is a finite automaton, which has no stack, and that is why a regular expression cannot decide this question: no finite pattern can count arbitrarily deep nesting.

Implementation

```

/// Valid Parentheses: classic stack problem with idiomatic pattern matching.
///
pub fn is_valid(s: &str) -> bool {
    use std::collections::HashMap;

    let mut stack = Vec::with_capacity(s.len());

    // Because manually adding if else is not an extensible solution.
    let expected = HashMap::from([
        (')', '('),
        ('}', '{'),
        (']', '['),
        // You can write whatever grammar you'd like, such as '<', '|', '$'....
    ]);

    for ch in s.chars() {
        // Look at the tasteful thickness of "পূর্ণাঙ্গ বিন্যাস মিলকরণ", We love Unicode!
        match ch {
            '(' | '[' | '{' => stack.push(ch),
            // Rust compiler will throw error for not handling all the possible character ranges.
            _ => { // So ask it to chill with an if guard....I've got you Sir!! Thank
                if stack.pop().as_ref() != expected.get(&ch) { return false; } }
        }
    }

    stack.is_empty()
}

```

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn valid_cases() {
        assert!(is_valid("()"));
        assert!(is_valid("([]{})"));
        assert!(is_valid("{[]}"));
    }

    #[test]
    fn invalid_cases() {
        assert!(!is_valid("]"));
        assert!(!is_valid("([]]"));
        assert!(!is_valid("("));
        assert!(!is_valid(")"));
        assert!(!is_valid("(("));
    }
}

```

The capacity hint is `s.len()`, the byte length of the string. A character count is never larger than a byte count, so the hint is an upper bound on the entries the loop can push, and the vector never grows while the scan runs. For text outside ASCII the hint over-allocates by the ratio between the two lengths: a few bytes of waste measured against the cost of a reallocation in the worst case.

`expected` is a map from each closing delimiter to the opening delimiter it must match. Building it once per call keeps the pairing in one structure rather than in one `match` arm per delimiter; adding a pair to the grammar is a longer array rather than a longer `match`.

The loop has two arms. The first pushes every opening delimiter. The second is the catch-all arm the compiler requires, and it performs the comparison for every other character:

```

_ => {
    if stack.pop().as_ref() != expected.get(&ch) {
        return false;
    }
}

```

`stack.pop()` returns `Option<char>`, and `.as_ref()` turns it into `Option<&char>` so that it can be compared with `expected.get(&ch)`, which has the same type. The guard returns `false` whenever the two options differ, which covers a mismatched delimiter and a closing delimiter on an empty stack. The next section shows the case it does not cover.

The function ends with `stack.is_empty()`. Without that expression, a string of opening delimiters alone would be accepted and `"("` would be reported as valid.

Intuition

input: `{[()]}`

char	stack before	pop result	stack after	value returned
{	[]	-	[{]	
[	[{]	-	[{[]]	
(	[{[]]	-	[{[()]	
)	[{[()]	Some(' (')	[{[]]	
]	[{[]]	Some(' [')	[{]	
}	[{]	Some('{')	[]	
end				true

input: `([)]`

char	stack before	pop result	stack after	value returned
(	[]	-	[(]	
[	[(]	-	[([]]	
)	[([]]	Some(' [')	[(]	-
]	[(]	Some(' (')	[]	false

The delimiters appear in matching quantities and are nested in the wrong order, so the scan reports the failure at the final character.

input: `)`

char	stack before	pop result	stack after	value returned
)	[]	None	[]	false

A closing delimiter with an empty stack ends the scan immediately.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	one push or one pop per character, each constant
Space	$O(n)$	for a string of opening delimiters only
Space on typical input	$O(d)$	d is the greatest nesting depth reached

The worst case is a string such as `"(((("`, where the stack grows with the input. The typical case is far smaller, because the stack holds only the openings that have not yet been closed, which is the nesting depth at the cursor rather than the length of the string.

The scan is iterative, so nesting depth is bounded by available memory rather than by the thread's stack. A recursive version of the same algorithm would exhaust the stack on a deeply nested string, which means a program that validates a document supplied by a user would have a denial-of-service condition. This is the one place in the book where an iterative formulation is a robustness requirement rather than a preference, and it costs nothing: the loop is as short as the recursion would be.

Limitations

A character outside the alphabet is accepted when the stack is empty. The catch-all arm compares two `Option<&char>` values. For a character that is not one of the six delimiters, `expected.get(&ch)` is `None`. If the stack is also empty, `stack.pop().as_ref()` is `None` as well, so the two are equal and the scan continues instead of rejecting the character. `is_valid("abc")` and `is_valid("()x")` therefore return `true`. The defect does not appear when the stack is non-empty, because a popped `Some(...)` can never equal `None`, so the same character is rejected after an opening delimiter. No test in the file covers an input of this kind, which is why the behaviour is easy to miss.

The function returns `bool`, so it cannot say where the failure occurred. A parser that reports "unmatched (at offset 12" needs the position. A version returning `Result<(), usize>` carries it with one change: iterate with `enumerate` and return the index in place of `false`.

The capacity hint is measured in bytes while the stack stores characters. For a string of three-byte characters the allocation is three times larger than the characters can justify, and the vector's length still never exceeds the character count. The bound is safe and the allocation is loose.

The stack element type is `char`, one Unicode scalar value per entry. The algorithm does not need that. A version that reports positions would push the index of the opening character instead, and the comparisons would be against indices rather than against characters.

Summary

- The invariant is that the stack holds the unmatched openings in order, with the most recent on top. Every branch of the loop preserves it, and the final `is_empty` is the statement that nothing was left unmatched.
- Counting delimiters is not sufficient. `([)]` has matching counts and no valid parse, which is what the stack is for.
- The scan rejects a closing delimiter whose opener is not on top, a closing delimiter with an empty stack, and a non-empty stack at the end.
- A character outside the alphabet is rejected only while the stack is non-empty. With an empty stack it is accepted, which is the defect described in Limitations.
- The scan is a pushdown automaton, which is why a regular expression cannot decide the question.

References

- Standard library, `Vec::pop`, on the `Option` it returns.
- Standard library, `str::chars`, on iterating scalar values rather than bytes.
- Michael Sipser, *Introduction to the Theory of Computation*, 3rd edition, Cengage Learning, 2012, Chapter 2, on pushdown automata and the limits of finite automata.

Min Stack

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/min_stack.rs`. Test it with `cargo test min_stack`.

Problem Statement

Support four operations on a stack, `push`, `pop`, `top`, and `get_min`, with each of them in constant time. The first three are what a stack always provides. The fourth is the whole problem: the smallest value currently in the stack, also in constant time.

A single vector of values cannot do it. The minimum of the stack is easy to track while values are only added: keep a running smallest value and update it on each push. Popping destroys it. When the smallest value leaves the stack, the next smallest is whatever the second smallest was at that moment, and that value was overwritten by the running minimum when the smaller one arrived. The information needed to answer the question after a pop is information the running minimum discarded.

Designing a Solution

The fix is to stop tracking one minimum and start recording the minimum at every depth. Keep a second vector, the same length as the first, in which entry `i` holds the smallest value among the first `i + 1` entries of the stack.

values	3	1	4	1	5
mins	3	1	1	1	1
	^	^			
		the smallest of 3, 1			
	the smallest of 3				

The invariant is the sentence

`mins[i]` is the minimum of `values[0..i]`.

Two facts follow from it, and between them they explain the whole implementation. The answer to `get_min` is the last entry of `mins`, because the last index covers the whole stack. And `pop` can remove the last entry of each vector and remain consistent, because after removing index `i` from both, the invariant still holds for every remaining index. Nothing has to be recomputed, because what was recorded at each depth happens to be exactly what is needed at that depth.

Push does not compute a minimum; it records the decision that was made at the new depth. The value appended to `mins` is either the incoming value, if it is smaller than everything already there, or the previous minimum, if it is not. Both are available in constant time, which is where the constant time bound comes from.

Implementation

```
///! Min Stack: O(1) push, pop, top, and get_min.
///!
///! We keep a parallel `mins` stack. Each push stores the current minimum at that
///! point in history, so pop is symmetric.

#[derive(Debug, Default)]
pub struct MinStack {
    values: Vec<i32>,
    mins: Vec<i32>,
}

impl MinStack {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn push(&mut self, val: i32) {
        match self.mins.last() {
            Some(&min) if min <= val => self.mins.push(min),
            _ => self.mins.push(val),
        }
        self.values.push(val);
    }

    pub fn pop(&mut self) -> Option<i32> {
        self.mins.pop();
        self.values.pop()
    }

    pub fn top(&self) -> Option<i32> {
        self.values.last().copied()
    }

    pub fn get_min(&self) -> Option<i32> {
```

```

        self.mins.last().copied()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stack_tracks_minimum() {
        let mut stack = MinStack::new();
        stack.push(-2);
        stack.push(0);
        stack.push(-3);
        assert_eq!(stack.get_min(), Some(-3));
        assert_eq!(stack.pop(), Some(-3));
        assert_eq!(stack.top(), Some(0));
        assert_eq!(stack.get_min(), Some(-2));
    }

    #[test]
    fn handles_duplicates() {
        let mut stack = MinStack::new();
        stack.push(1);
        stack.push(1);
        stack.pop();
        assert_eq!(stack.get_min(), Some(1));
    }
}

```

`MinStack::new` returns `Self::default()`, so the constructor and the `Default` implementation cannot drift apart. The derive provides `Default`, and `new` is the name callers expect.

`push` matches on the last entry of `mins` with a guard. `Some(&min)` if `min <= val` handles the case where the stack is non-empty and the incoming value is not smaller, in which case the previous minimum is repeated. The underscore arm covers both remaining cases at once: the stack is empty, so there is no previous minimum, or the incoming value is strictly smaller, so it becomes the new minimum. In both, the value pushed is `val`.

The comparison is `<=` rather than `<`. With `<` the two arms would still be correct for the minimum, because equal values give the same answer. The choice matters for the variant that stores counts, where equality against the current minimum is what increments the count instead of appending an entry.

`pop` discards the last entry of each vector and returns the value from the first. Both vectors lose exactly one element per call, so the equal-length invariant is maintained by construction, and the discarded entry of `mins` is never read.

`top` and `get_min` use `copied`, which turns `Option<&i32>` into `Option<i32>` by copying the value out of the reference. Without it the returned `&i32` would bor-

row the stack, and a caller could not push while holding the result, a restriction that a four-line accessor should not impose.

Intuition

call	values	mins	returns
push(-2)	[-2]	[-2]	-
push(0)	[-2, 0]	[-2, -2]	-
push(-3)	[-2, 0, -3]	[-2, -2, -3]	-
get_min()	[-2, 0, -3]	[-2, -2, -3]	Some(-3)
pop()	[-2, 0]	[-2, -2]	Some(-3)
top()	[-2, 0]	[-2, -2]	Some(0)
get_min()	[-2, 0]	[-2, -2]	Some(-2)
pop()	[-2]	[-2]	Some(0)
pop()	[]	[]	Some(-2)
get_min()	[]	[]	None

The sequence in the test suite exercises the case the running-minimum design would fail: after `-3` is popped, `get_min` reports `-2`, the value that was second smallest. A single tracked minimum would still report `-3`, a value no longer in the stack.

The second test covers equality. Two equal values are pushed and one is popped; the minimum after the pop is still that value, because the surviving entry also records it. Both vectors held two entries and one remains, so the derivation of the invariant is unaffected by the repetition.

Time and Space Complexity

Operation	Time	Space
push	$O(1)$ amortised	one <code>i32</code> in each vector
pop	$O(1)$	releases one <code>i32</code> from each vector
top	$O(1)$	none
get_min	$O(1)$	none

Push is amortised rather than constant because a `Vec` occasionally reallocates and moves its elements. The event is rare and the capacity doubles, so n pushes perform $O(n)$ total movement, which is the amortised $O(1)$ the caller receives.

The structure stores two integers per element to hold one. For a stack of a million entries that is eight megabytes instead of four. The overhead is the

price of the constant time minimum, and it is the measurement to give when someone asks what the approach costs. The alternative is described under failure modes: it stores a pair per *run* of equal minimum values rather than a value per element, which is smaller when the minimum repeats and equal to or larger than this design when it does not.

Limitations

The memory overhead is proportional to the number of elements rather than to the number of distinct minimum values. A stack whose minimum changes on every push, which is what an increasing sequence produces, records the incoming value every time, and both vectors hold distinct entries. A stack whose minimum never changes records the same value repeatedly, and the second vector is redundant. A design that stores `(value, repeat_count)` pairs, appending an entry only when the minimum strictly decreases and incrementing the count when it is repeated, holds fewer entries on inputs with repeated minima and never more than one entry per element. That variant is not implemented here, and the gain appears only on inputs where the minimum repeats.

Pop on an empty stack returns `None` and changes nothing: `Option::pop` on an empty `Vec` returns `None` without panicking, and both vectors are left empty. `top` and `get_min` also return `None` on an empty stack, so an empty stack reports no minimum rather than a minimum of zero, which is the distinction that a sentinel value would lose.

The structure has no `len` and no `is_empty`. A caller cannot ask how many entries the stack holds, and the two fields are private, so the depth is not observable from outside the module. A caller can push, pop, read the top and read the minimum, and nothing else; an application that needs the depth has to track it alongside the structure, or wait for the accessor to be added.

The value type is `i32`. Making the structure generic over a type `T` with the `Ord` bound requires a type parameter on the struct, the implementation, and the two vectors, and the comparison in `push` is the only operation that needs the bound. Nothing in the algorithm depends on the values being integers, and nothing in it depends on `Copy` either: the second vector stores cloned values, which would need a `Clone` bound on the same type parameter.

The order of the two pushes in `push`, `mins` first, then `values`, is an aspect of the invariant that the type system does not enforce. The code appends to `mins` before it appends to `values`, so if the second append could fail, the vectors would differ in length and the invariant would be broken. In Rust an allocation failure aborts the process rather than returning an error, so the case is not reachable through this function. The invariant holds because of the order of two adjacent statements rather than because of anything the compiler checks, and a reader verifying the structure should know that.

Summary

- The invariant is that `mins[i]` is the minimum of `values[0..=i]`. Every question about the design follows from it.
- A single tracked minimum is not sufficient: the value that the running minimum overwrites is the value needed after the pop, which is the problem the parallel vector solves.
- The cost is constant amortised time and two integers per element. The time bound alone does not decide whether the structure is usable for a large stack.
- The `(value, count)` pair is an alternative that occupies less memory when consecutive values repeat, and the shape of the input decides between the two designs. Only the bound changes if the variant is requested.

References

- Standard library, `Vec::last`.
- Standard library, `Option::copied`, on the conversion from a borrowed to an owned value.
- Standard library, `Vec::push`, on the amortised cost of growth.

Longest Substring Without Repeating Characters

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/sliding_window.rs`. Test it with `cargo test sliding_window`.

Problem Statement

Given a text, return the length of the longest contiguous substring that contains no repeated character. The answer is a length, not the substring, which lets the implementation discard the content of the window and keep only its start.

Designing a Solution

Maintain a window `[start, end)` over the characters and a map from each character to the last index where it was seen. Extend the window by one character at a time. If the new character was last seen at an index at or after `start`, the window must begin after that index.

"pwwkew"

end	char	window	last_seen	longest
0	p	[p]	{p:0}	1
1	w	[pw]	{p:0, w:1}	2
2	w	[w]	start = 1 + 1 = 2	2
3	k	[wk]	{w:2, k:3}	2
4	e	[wke]	{e:4}	3
5	w	[ew]	start = 2 + 1 = 3	3

The line that carries the algorithm is `start = start.max(previous + 1)`. The `max` is not defensive coding. A character may have been seen, left the window, and been seen again; in that case its recorded index is *before* `start`, and assigning `previous + 1` unconditionally would move the window backwards and re-scan characters that have already been counted. `start` is monotonic, and that monotonicity is what makes the scan linear.

Implementation

```

///! Longest substring without repeating characters.
///!
///! Sliding window with `HashMap<char, usize>`; the window start is monotonic.

use std::collections::HashMap;

pub fn length_of_longest_substring(s: &str) -> usize {
    let mut last_seen = HashMap::with_capacity(s.len());

    // it's magic compiler is able to deduce the datatype as usize based on its first
usage, which is kinda cool, isn't it ?
    let (mut start, mut longest) = (0, 0);

    for (end, ch) in s.chars().enumerate() {
        if let Some(&previous) = last_seen.get(&ch) {
            start = start.max(previous + 1);
        }
        longest = longest.max(end - start + 1);
        last_seen.insert(ch, end);
    }

    longest
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn examples() {
        assert_eq!(length_of_longest_substring("abcabcbb"), 3);
        assert_eq!(length_of_longest_substring("bbbbbb"), 1);
        assert_eq!(length_of_longest_substring("pwwkew"), 3);
        assert_eq!(length_of_longest_substring(""), 0);
    }
}

```

`for (end, ch) in s.chars().enumerate()` iterates over Unicode scalar values, not bytes. `end` is therefore a character index, and `end - start + 1` is a length in characters. A byte-oriented version would report a different number for any text outside ASCII.

`if let Some(&previous) = last_seen.get(&ch)` copies the index out of the map, so the borrow ends at the end of the `if let` and the `insert` below is allowed.

Intuition

A case where the repeat has left the window, which is the case the `max` exists for:

"abba"

end	char	previous	start before	start after	window	longest
0	a	none	0	0	[a]	1
1	b	none	0	0	[ab]	2
2	b	Some(1)	0	2	[b]	2
3	a	Some(0)	2	2	[ba]	2

At end = 3 the recorded index of 'a' is 0, which is before the window start of 2. ``start.max(0 + 1)`` keeps start at 2. Assigning `0 + 1` instead would set start to 1 and count "abb" as a window.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	one step per character; start never decreases
Space	$O(\min(n, k))$	k distinct characters occur in the text

Limitations

The map is sized by a byte count. `HashMap::with_capacity(s.len())` uses the byte length as the capacity hint, which over-allocates for multi-byte text. The hint is safe because a byte count bounds a character count.

The answer is a length, and the substring is discarded. A caller that needs the text of the longest window must record `start` alongside `longest` and slice the original at the end. That change is two lines and is not in the file.

Non-ASCII text is measured in characters, not in grapheme clusters. A combining sequence such as `e` followed by a combining acute accent is two scalar values, so the function counts it as two characters. A user-facing length would need grapheme segmentation, which the standard library does not provide.

The map keeps an entry per distinct character ever seen. Entries for characters that have left the window are not removed, so the space bound is over the whole text rather than over the current window. An implementa-

tion that removes entries on eviction would bound memory by the window instead.

Summary

- The `max` on the first match is required for correctness. Without it the function fails on `"abba"`.
- `start` is monotonic, which is what reduces the scan to a single pass with no backtracking and an $O(n)$ bound.
- The space bound is $O(\min(n, \text{alphabet}))$ rather than $O(n)$. For `char` keys the alphabet cannot exceed 1,112,064, the number of Unicode scalar values.
- Lengths are counted in scalar values, so a combining sequence counts as two. A user-visible length would need grapheme segmentation, which the standard library does not provide.

References

- Standard library, `str::chars`.
- Standard library, `Iterator::enumerate`.
- Standard library, `Ord::max`.

Merge Intervals

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/merge_intervals.rs`. Test it with `cargo test merge_intervals`.

Problem Statement

Given a list of closed intervals `[start, end]`, return the list of maximal intervals covering exactly the same points. Overlapping intervals are merged, and so are intervals that only touch: `(1, 4)` and `(4, 5)` become `(1, 5)`, because `4` belongs to both. Whether touching intervals merge is the first property to fix, since half-open intervals would not merge.

Designing a Solution

Sorting by start removes the need to compare intervals with each other. After the sort, an interval can only overlap the most recent one in the output, so each interval costs one comparison.

input	(1, 3) (2, 6) (8, 10) (15, 18)	already sorted here
interval	merged so far	decision
(1, 3)	[(1, 3)]	the first interval is always pushed
(2, 6)	[(1, 6)]	$2 \leq 3$, so extend the end to $\max(3, 6)$
(8, 10)	[(1, 6), (8, 10)]	$8 > 6$, a gap, so push
(15, 18)	[..., (15, 18)]	$15 > 10$, a gap, so push

The comparison uses `max` for the end. With `(1, 10)` followed by `(2, 3)`, the incoming end is smaller than the end already recorded, and assigning it would shrink the interval.

Implementation

The file contains two functions. The first consumes the vector it is given; the second borrows it.

```

/// Merge Intervals: sort by start, then merge overlapping intervals in one pass.
///
/// The `match out.last_mut()` pattern is intentionally clean: no deep nesting,
/// no confusing boolean flags.

pub fn merge_intervals(mut intervals: Vec<(i32, i32)>) -> Vec<(i32, i32)> {
    if intervals.is_empty() {
        return intervals;
    }

    intervals.sort_unstable_by_key(|(start, _)| *start);

    let mut merged: Vec<(i32, i32)> = Vec::with_capacity(intervals.len());
    for (start, end) in intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.1 => {
                previous.1 = previous.1.max(end);
            }
            _ => merged.push((start, end)),
        }
    }
    merged
}

pub fn merge_intervals2(&mut intervals: Vec<(i32, i32)>) -> Vec<(i32, i32)> {
    if intervals.is_empty() {
        return intervals;
    }

    intervals.sort_unstable_by_key(|(start, _)| *start);

    let mut merged: Vec<(i32, i32)> = Vec::with_capacity(intervals.len());

    for &(start, end) in &intervals {
        match merged.last_mut() {
            Some(previous) if start <= previous.1 => {
                previous.1 = previous.1.max(end);
            }
            _ => merged.push((start, end)),
        }
    }
    merged
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn merges_overlapping_and_adjacent() {
        let intervals = vec![(1, 3), (2, 6), (8, 10), (15, 18)];
        assert_eq!(merge_intervals(intervals), vec![(1, 6), (8, 10), (15, 18)]);
    }

    #[test]
    fn handles_empty_and_disjoint() {
        assert_eq!(merge_intervals(vec![]), vec![]);
    }
}

```

```

    }
    assert_eq!(merge_intervals(vec![(1, 2), (3, 4)]), vec![(1, 2), (3, 4)]);
}

```

`sort_unstable_by_key` sorts by the start value and does not preserve the relative order of intervals with equal starts. That is correct here: intervals with equal starts are merged into one, so their order cannot matter.

`for (start, end) in intervals` consumes the vector, which moves each pair out rather than borrowing it. `for &(start, end) in &intervals` in the second function copies each pair out of the borrowed vector. Since `(i32, i32)` is `Copy`, both loops work, and the difference is only whether the caller keeps the vector.

`match merged.last_mut()` with a guard keeps the three cases flat: an overlap, an empty output, and a gap. The last two share an arm because they share an action.

Intuition

input: [(15, 18), (2, 3), (1, 10), (1, 10)]

sorted by start: [(1, 10), (1, 10), (2, 3), (15, 18)]

interval	merged before	overlap?	merged after
(1, 10)	[]	output is empty	[(1, 10)]
(1, 10)	[(1, 10)]	1 ≤ 10	[(1, 10)]
(2, 3)	[(1, 10)]	2 ≤ 10	[(1, 10)]
(15, 18)	[(1, 10)]	15 > 10	[(1, 10), (15, 18)]

end stays max(10, 10)
end stays max(10, 3)

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n \log n)$	the sort dominates; the sweep is $O(n)$
Space	$O(n)$	the output holds at most one interval per input

The input vector is taken by value, so its allocation is available to the function; the code still allocates a second vector for the output rather than sorting and merging in place.

Limitations

The second function is unreferenced. `merge_intervals2` is not exported from `lib.rs`, no test in the file calls it, and no integration test calls it. Its only difference from `merge_intervals` is the loop, which borrows the vector instead of consuming it. An unused `pub fn` in a library is not a compiler warning, so the two functions coexist without a signal that one of them is unused.

There is no test for the borrow-based variant. Even if the function is kept, nothing checks that it agrees with the other one. A one-line assertion, `assert_eq!(merge_intervals2(v.clone()), merge_intervals(v))`, would cover it.

`i32` bounds silently overflow in the merged output. The end of a merged interval is `previous.1.max(end)`, so no arithmetic is performed and no overflow is possible. The bound of `i32` matters only in the sense that a caller with larger times cannot use the function.

Summary

- Whether intervals that touch are merged is a specification decision rather than a detail of the implementation. It changes one comparison from `<` to `<=`, and the two choices give different answers for `[1, 2]` followed by `[2, 3]`.
- The $O(n \log n)$ bound comes from the sort. The sweep that follows is linear, so the sort determines the cost of the function.
- After sorting by start, one comparison per interval is sufficient. Merging extends the last interval to the largest end seen, so that interval is the only one a new interval can overlap.

References

- Standard library, `slice::sort_unstable_by_key`.
- Standard library, `Vec::last_mut`.
- Standard library, `Ord::max`.

Top K Frequent Elements

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/top_k_frequent.rs`. Test it with `cargo test top_k_frequent`.

Problem Statement

Given a slice of integers and a number `k`, return the `k` values that occur most often. The result order is only defined if the function states how ties are broken, and the file's ordering answers that question by construction: the heap key is the pair `(count, value)`, compared lexicographically, so a larger value wins a tie.

Designing a Solution

Two passes. The first counts occurrences in a hash map. The second moves every `(count, value)` pair into a `BinaryHeap` and pops the largest keys.

`BinaryHeap` in Rust is a max-heap, so pushing `(count, num)` and popping puts the highest count first. That is the opposite of the min-heap-with-size-`k` design that keeps $O(m \log k)$ time, and the file chooses the simpler of the two. The cost table below states what that choice costs.

Implementation

```
/// Top K Frequent Elements.
///
/// Count occurrences with a HashMap, then use a max-heap (`BinaryHeap`) to pull
/// out the top k elements.  $O(n \log n)$  worst case; acceptable and readable.
use std::collections::{BinaryHeap, HashMap};
```

```

pub fn top_k_frequent(nums: &[i32], k: usize) -> Vec<i32> {
    let mut counts: HashMap<i32, usize> = HashMap::with_capacity(nums.len());
    for &num in nums.iter() {
        *counts.entry(num).or_insert(0) += 1;
    }

    // BinaryHeap is a max-heap in Rust, so (count, num) keeps highest counts first.
    let mut heap: BinaryHeap<(usize, i32)> = counts
        .into_iter()
        .map(|(num, count)| (count, num))
        .collect();

    let mut result = Vec::with_capacity(k);
    while result.len() < k {
        match heap.pop() {
            Some((_, num)) => result.push(num),
            None => break,
        }
    }
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_top_two() {
        let nums = [1, 1, 1, 2, 2, 3];
        let mut result = top_k_frequent(&nums, 2);
        result.sort_unstable();
        assert_eq!(result, vec![1, 2]);
    }

    #[test]
    fn k_larger_than_unique_values_returns_all() {
        let result = top_k_frequent(&[1, 2, 3], 10);
        assert_eq!(result.len(), 3);
    }
}

```

`*counts.entry(num).or_insert(0) += 1` is the counting idiom. `entry` looks up or inserts, `or_insert(0)` supplies the initial count, and the dereference lets the increment write through the returned `&mut` `usize`. The map is searched once per element.

`counts.into_iter()` consumes the map and yields owned pairs, which avoids borrowing it while the heap is built. `map(|(num, count)| (count, num))` reverses each pair so that the heap's ordering applies to the count first; this is the whole tie-break rule.

`while result.len() < k` with `None => break` handles `k` larger than the number of distinct values. The heap empties and the loop stops, leaving a result shorter than `k`.

Intuition

```

nums = [1, 1, 1, 2, 2, 3]    k = 2

pass 1: counts = {1: 3, 2: 2, 3: 1}

pass 2: heap after collecting the pairs (count, value):
        pop order is (3, 1), (2, 2), (1, 3)

pop 1 -> push value 1    result = [1]
pop 2 -> push value 2    result = [1, 2]    length is k, stop

The heap still holds (1, 3), which is not popped.

```

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(n + m \log m)$ expected	m distinct values; the first pass is $O(n)$, the heap operations are $O(m \log m)$
Space	$O(m)$	the map and the heap each hold one entry per distinct value

The heap holds every distinct value, not k of them. For a slice of a million elements with half a million distinct values and $k = 10$, the program builds a heap of half a million entries to return ten. The bounded variant keeps a min-heap of size k and discards each element smaller than the current root, which is $O(m \log k)$ and needs $O(k)$ space. The file does not implement it, and the comment in the file states the trade in one line: $O(n \log n)$ worst case; acceptable and readable.

Limitations

The heap is unbounded in the number of distinct values. Every distinct value counted becomes an entry in the heap, so the structure holds m entries at its largest, where m is the number of distinct values in the input.

The result order depends on the heap's internal comparison and nothing else. The pair ordering makes the order deterministic, and the first test sorts the result before asserting, so the file never pins the order. A

caller that prints the result directly sees the order the heap produced, which is by descending count and then by descending value.

A `k` of zero returns an empty vector. `Vec::with_capacity(0)` allocates nothing and the loop does not run, so the answer is empty rather than a panic. The file has no test for it.

The tie-break rule is not documented. A reader has to derive it from `(count, num)` and from the fact that `BinaryHeap` is a max-heap. Two sentences in the module documentation would make it part of the contract rather than a consequence of the implementation.

Summary

- The tie-break rule is part of the specification. "The `k` most frequent" is incomplete where counts are equal, and this implementation resolves a tie in favour of the larger value, because the heap orders `(count, value)` pairs and is a max-heap.
- Two heap designs are available. A heap holding every distinct value costs $O(m \log m)$; a heap bounded at `k` entries costs $O(m \log k)$. This file uses the bounded heap, and the result is therefore in heap order rather than sorted.
- `counts.entry(num).or_insert(0) += 1` performs one map lookup rather than a lookup followed by an insertion.

References

- Standard library, `BinaryHeap`, which documents that the heap is a max-heap.
- Standard library, `cmp::Reverse`, the wrapper that turns a max-heap into a min-heap.
- Standard library, `HashMap::entry`.

Part II: Trees and Graphs

Binary Tree

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/binary_tree.rs`. Test it with `cargo test binary_tree`.

Problem Statement

Represent a binary tree of integers, then compute its depth, mirror it in place, and produce its three depth-first traversals. No allocation during the traversals, and no recursion beyond what the shape of the type requires.

Designing a Solution

The type is an algebraic data type with two variants:

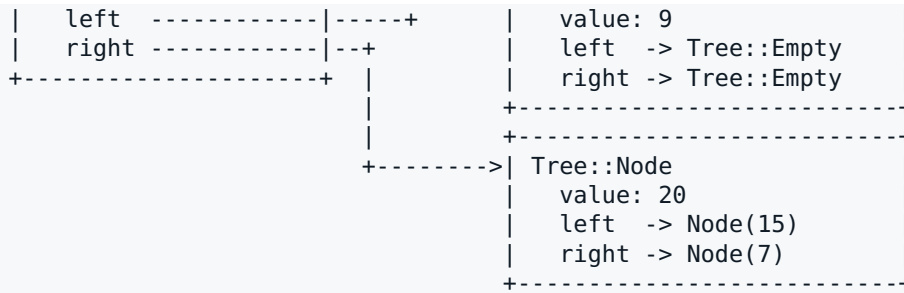
```
Tree = Empty
      | Node { value: i32, left: Box<Tree>, right: Box<Tree> }
```

Two consequences follow from writing it this way rather than with nullable pointers.

A node always has both children. There is no state in which the left child is absent and the right one is present, because the `Node` variant has no optional fields. Every function that matches on `Tree::Node` can bind `left` and `right` without testing them.

The recursion is in the type. A `Node` owns its children, and each child is boxed, so the size of a `Tree` value does not depend on how many nodes exist:

stack	heap
<pre>+-----+ Tree::Node value: 3 +-----+</pre>	<pre>+-----+ Tree::Node +-----+</pre>



Each arrow is one eight-byte ownership pointer. The `Empty` variant is a discriminant with no payload, so it occupies no heap space.

Implementation

```

///! Binary tree as an algebraic data type.
///! ```text
///! Tree = Empty
///! | Node { value, left: Box<Tree>, right: Box<Tree> }
///! ```

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Tree {
    Empty,
    Node {
        value: i32,
        left: Box<Tree>,
        right: Box<Tree>,
    },
}

impl Tree {
    pub fn max_depth(&self) -> usize {
        match self {
            Tree::Empty => 0,
            Tree::Node { left, right, .. } => 1 + left.max_depth().max(right.max_depth()),
        }
    }

    pub fn invert(&mut self) {
        match self {
            Tree::Empty => {},
            Tree::Node { left, right, .. } => {
                left.invert();
                right.invert();
                std::mem::swap(left, right);
            }
        }
    }

    pub fn preorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_preorder(&mut out);
        out
    }

    pub fn inorder(&self) -> Vec<i32> {
        let mut out = Vec::new();
        self.collect_inorder(&mut out);
    }
}

```

```

    out
}

pub fn postorder(&self) -> Vec<i32> {
    let mut out = Vec::new();
    self.collect_postorder(&mut out);
    out
}

fn collect_preorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            out.push(*value);
            left.collect_preorder(out);
            right.collect_preorder(out);
        }
    }
}

fn collect_inorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_inorder(out);
            out.push(*value);
            right.collect_inorder(out);
        }
    }
}

fn collect_postorder(&self, out: &mut Vec<i32>) {
    match self {
        Tree::Empty => {}
        Tree::Node { value, left, right } => {
            left.collect_postorder(out);
            right.collect_postorder(out);
            out.push(*value);
        }
    }
}
}

#[cfg(test)]
mod tests {
    use super::*;

    fn leaf(value: i32) -> Tree {
        Tree::Node {
            value,
            left: Box::new(Tree::Empty),
            right: Box::new(Tree::Empty),
        }
    }

    fn node(value: i32, left: Tree, right: Tree) -> Tree {
        Tree::Node {
            value,
            left: Box::new(left),
            right: Box::new(right),
        }
    }

    fn sample() -> Tree {
        node(3, leaf(9), node(20, leaf(15), leaf(7)))
    }
}

```

```

#[test]
fn max_depth_works() {
    assert_eq!(Tree::Empty.max_depth(), 0);
    assert_eq!(leaf(1).max_depth(), 1);
    assert_eq!(sample().max_depth(), 3);
}

#[test]
fn invert_swaps_subtrees() {
    let mut tree = sample();
    tree.invert();

    match &tree {
        Tree::Node { left, right, .. } => {
            match left.as_ref() {
                Tree::Node { value, .. } => assert_eq!(*value, 20),
                _ => panic!("expected left node"),
            }
            match right.as_ref() {
                Tree::Node { value, .. } => assert_eq!(*value, 9),
                _ => panic!("expected right node"),
            }
        }
        _ => panic!("expected node"),
    }
}

#[test]
fn traversals_work() {
    let tree = sample();
    assert_eq!(tree.preorder(), vec![3, 9, 20, 15, 7]);
    assert_eq!(tree.inorder(), vec![9, 3, 15, 20, 7]);
    assert_eq!(tree.postorder(), vec![9, 15, 7, 20, 3]);
}

```

`invert` swaps the two children after recursing into both. `std::mem::swap(left, right)` exchanges the two `Box` values, which swaps two pointers per node and moves no subtree. Swapping before recursing would also be correct; the two subtrees are independent, so the order is a matter of taste.

`max_depth` uses `..` in the pattern to ignore `value`, and computes `1 + left.max_depth().max(right.max_depth())` without a binding for the child depths. `Ord::max` on two `usize` values is a single instruction.

The three collection helpers take `&mut Vec<i32>`. Returning a new vector from each recursive call would copy the whole output at every level of the tree, which turns a linear traversal into a quadratic one.

The test helpers `leaf` and `node` exist because building a tree means writing `Box::new` at every level, and the constructors keep the shape of the sample tree readable. The constructors are in the test module, so they are not part of the library's API.

Intuition

```
sample tree
      3
     /\
    9  20
   /\  \
  15 7

preorder  value, left, right  3 9 20 15 7
inorder   left, value, right  9 3 15 20 7
postorder left, right, value  9 15 7 20 3

max_depth 1 + max(depth of 9, depth of the 20 subtree)
           depth of 9 = 1
           depth of 20 = 1 + max(1, 1) = 2
           result      = 1 + max(1, 2) = 3

invert     left and right exchanged at every node
      3
     /\
    20 9
   /\  \
  7  15
```

Time and Space Complexity

Operation	Time	Space
max_depth	$O(n)$	$O(h)$ stack frames
invert	$O(n)$	$O(h)$ stack frames, no allocation
each traversal	$O(n)$	$O(n)$ for the output, $O(h)$ stack frames

h is the height of the tree: $O(\log n)$ when the tree is balanced and $O(n)$ when it is a chain.

Limitations

A deep tree exhausts the stack. All five methods recurse, so each uses one stack frame per level. A left-leaning tree of a few hundred thousand nodes overflowing the default thread stack aborts the process, because a stack overflow is not a panic that unwinds. The traversals cannot be reformulated iteratively without an explicit work list, and `#[derive(Clone)]` has the same problem: the derived `clone` recurses as well.

The derived destructor recurses too. Dropping a deep tree follows the boxes, one frame per level, for the same reason. Freeing the memory is therefore as unsafe for a deep tree as cloning it.

The value type is fixed at `i32`. Every method works for any type, and making the declaration generic is one change to the `enum` and one to each signature, but the file as written is not generic.

Tree has no `PartialOrd`, no iteration, and no `len`. A caller that needs the node count writes a recursive helper or a loop with a work list.

The test for `invert` inspects two values and not the shape. It checks that the roots of the two subtrees were exchanged, which is enough to catch an `invert` that does nothing, and not enough to catch one that swaps the children of the root only. Asserting `tree.preorder()` against the mirrored sequence would test the whole tree in one line.

Summary

- The type itself removes the empty case: both children of a `Node` are present, so no function tests for a missing one.
- A traversal order is named by where the value is visited. Pre-order visits it before the children, in-order between them, and post-order after them.
- The stack cost is $O(h)$, and `h` takes two values. A balanced tree gives $O(\log n)$ and a chain gives $O(n)$.
- The `Box` is what makes the type finite. A `Node` holding another `Node` directly would have infinite size, and the compiler rejects the declaration.

References

- The Rust Book, [Defining an enum](#).
- The Rust Book, [Enabling recursive types with boxes](#).
- Standard library, `std::mem::swap`.

Binary Search Tree

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/bst_clean.rs`. Run it with `cargo run --bin bst_clean`.

Problem Statement

Build an ordered tree, a binary search tree, supporting insertion, membership, removal, and a range query. Removal is the interesting one, because a node with two children cannot simply be unlinked: something has to take its place without breaking the ordering.

Designing a Solution

The ordering invariant is the whole structure:

```
for every node:  every value in the left subtree < the node's value
                  every value in the right subtree > the node's value
```

A lookup follows one path from the root, discarding a subtree at each comparison. An in-order traversal reads the values in ascending order, which is the standard way to test that the invariant holds.

Removal has three shapes:

```
delete a leaf           the link becomes Empty

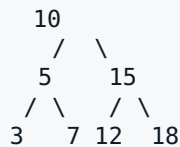
delete a node with one child the child takes its place

delete a node with two children    the smallest value of the right subtree takes its
place; it is the next value in order, so the
ordering holds on both sides

example: remove 10 from
         10
```



the right subtree is 15, whose smallest value is 15, so 15 takes the place of 10



remove 5 -> its only child 7 moves up
 remove 10 -> 12, the successor, moves up

Implementation

```

use std::cmp::Ordering;

type TreeLink<T> = Box<BST<T>>;

#[derive(Debug, Clone, Default)]
pub enum BST<T> {
    #[default]
    Empty,
    Node {
        value: T,
        left: TreeLink<T>,
        right: TreeLink<T>,
    },
}

impl<T: Ord> BST<T> {
    pub fn new() -> Self {
        Self::Empty
    }

    pub fn insert(&mut self, val: T) {
        match self {
            Self::Empty => {
                *self = Self::Node {
                    value: val,
                    left: Box::new(Self::Empty),
                    right: Box::new(Self::Empty),
                };
            }
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.insert(val),
                Ordering::Greater => right.insert(val),
                Ordering::Equal => {},
            },
        }
    }

    pub fn contains(&self, val: &T) -> bool {
        match self {
            Self::Empty => false,
            Self::Node { value, left, right } => match val.cmp(value) {
                Ordering::Less => left.contains(val),
                Ordering::Greater => right.contains(val),
                Ordering::Equal => true,
            },
        }
    }
}

```

```

}

pub fn remove(&mut self, val: &T) -> bool {
    match self {
        Self::Empty => false,
        Self::Node { value, left, right } => match val.cmp(value) {
            Ordering::Less => left.remove(val),
            Ordering::Greater => right.remove(val),
            Ordering::Equal => {
                if left.is_empty() && right.is_empty() {
                    *self = Self::Empty;
                    return true;
                }
                if left.is_empty() {
                    *self = *std::mem::take(right);
                    return true;
                }
                if right.is_empty() {
                    *self = *std::mem::take(left);
                    return true;
                }
                let min_val = right.remove_min().unwrap();
                *value = min_val;
                true
            }
        },
    }
}

fn remove_min(&mut self) -> Option<T> {
    match self {
        Self::Empty => None,
        Self::Node { left, .. } if left.is_empty() => {
            let node = std::mem::take(self);
            if let Self::Node { value, right, .. } = node {
                *self = *right;
                Some(value)
            } else {
                unreachable!()
            }
        }
        Self::Node { left, .. } => left.remove_min(),
    }
}

pub fn range(&self, low: &T, high: &T) -> Vec<&T> {
    let mut result = Vec::new();
    self.range_helper(low, high, &mut result);
    result
}

// Explicit lifetime 'a ties the references in the vector to the lifetime of
&self.
fn range_helper<'a>(&'a self, low: &T, high: &T, acc: &mut Vec<&'a T>) {
    match self {
        Self::Empty => {}
        Self::Node { value, left, right } => {
            if value < low {
                right.range_helper(low, high, acc);
            } else if value > high {
                left.range_helper(low, high, acc);
            } else {
                left.range_helper(low, high, acc);
                acc.push(value);
                right.range_helper(low, high, acc);
            }
        }
    }
}

```

```

    }
}

fn is_empty(&self) -> bool {
    matches!(self, Self::Empty)
}

fn main() {
    let mut bst = BST::new();
    bst.insert(10);
    bst.insert(5);
    bst.insert(15);
    bst.insert(3);
    bst.insert(7);
    bst.insert(12);
    bst.insert(18);

    println!("Contains 7: {}", bst.contains(&7));
    println!("Contains 12: {}", bst.contains(&12));

    let range = bst.range(&4, &16);
    println!("Values in [4, 16]: {:?}", range);

    println!("Remove 5: {}", bst.remove(&5));
    println!("Remove 10: {}", bst.remove(&10));
    println!("Remove 99: {}", bst.remove(&99));

    println!("After removals: {:#?}", bst);
}

```

The type is its own node. `BST<T>` is either `Empty` or a node with two boxed subtrees, so `Box<BST<T>>` is the link type and there is no separate wrapper struct. The `Default` derive with `#[default]` on `Empty` gives `BST::default()` as an empty tree.

`match val.cmp(value)` produces three arms for the three comparison results. Using `cmp` rather than a pair of comparisons means the ordering is decided once per node, and the `Ordering::Equal` arm is explicit at every call site.

`remove` handles the two-children case by moving the smallest value out of the right subtree into the node being deleted. `right.remove_min()` returns that value, and the `unwrap` is safe because the branch is only reached when the right subtree is non-empty. The node structure is untouched: values move, links stay.

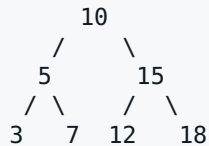
`std::mem::take(right)` replaces the right subtree with `Empty` and returns the old one. `*self = *std::mem::take(right)` then moves the subtree into the node, which detaches the whole subtree from its old position in one assignment.

`range_helper` prunes: a subtree entirely below `low` is only searched on its right, and one entirely above `high` is only searched on its left. The lifetime parameter is written `'a` and ties the references in `acc` to the tree that `self` borrows, which is what allows the returned vector to outlive the call.

Intuition

bst_clean output, with the tree built by main:

```
insert 10, 5, 15, 3, 7, 12, 18
```



```
in order      : 3 5 7 10 12 15 18
```

```
range (4, 16) : 5 7 10 12 15
```

```

remove(&5)    -> true    5 has one child, so 7 moves up
remove(&10)   -> true    10 has two children, so 12 moves up
remove(&99)   -> false   the descent ends at Empty
  
```

Time and Space Complexity

Operation	Time	Space
insert	$O(h)$	$O(h)$ stack frames
contains	$O(h)$	$O(h)$ stack frames
remove	$O(h)$	$O(h)$ stack frames, plus one descent to the successor
range	$O(h + k)$	$O(k)$ for the output, $O(h)$ stack frames

h is the height. The tree does not balance, so h is $O(\log n)$ only when the input arrives in an order that produces a balanced shape, and $O(n)$ when it arrives sorted.

Limitations

The file has no tests. `src/bin/bst_clean.rs` contains no `#[cfg(test)]` module, and `main` prints values without asserting anything, so a break in the ordering invariant shows up as output on the terminal rather than as a failing test. `cargo test` reports no test for this file. The behaviour to pin down is the ordering invariant and the three removal shapes; a test that inserts a known sequence, removes a leaf, a one-child node, and a two-child node, and asserts the in-order traversal after each step covers all of it.

The tree does not balance. Inserting 1 to 1000 in ascending order produces a chain of height 1000, and every operation becomes linear. A pro-

duction ordered collection uses a red-black tree or a B-tree; the standard library's `BTreeMap` is the one to reach for.

Removal by successor does not balance either. Repeated deletions under this rule can leave a tree taller than the input would suggest, and the effect is measurable over a long sequence of insertions and deletions.

`remove_min` contains an `unreachable!()` arm. The branch is genuinely unreachable: the guard `left.is_empty()` and the pattern `Self::Node` together guarantee that the taken node is a node. The arm exists because the compiler cannot see that, and `unreachable!()` records that guarantee, but it is a panic path in a function whose caller does not expect one.

`remove` panics if the tree has a node whose left child is empty and whose right child is not `Node`. The two-child branch calls `right.remove_min().unwrap()`, and `remove_min` returns `None` only for an empty subtree. The `unwrap` cannot fail while the invariant holds; it is a second panic path.

The value type needs `Ord`, and the range query needs `PartialOrd`. Those are the correct bounds. A caller with a type that is only `PartialOrd`, such as a floating-point number, cannot use the tree at all, because `insert` matches on `Ordering` and `f64` does not implement `Ord`.

Summary

- Removal has three shapes. Removing a leaf and removing a node with one child are bookkeeping; removing a node with two children is the case the ordering invariant makes interesting.
- The replacement value is the next value in order, which is also the minimum of the right subtree. The first description states why the invariant survives the substitution.
- The tree does not balance. A sorted input produces a chain, so $O(h)$ becomes $O(n)$, and `BTreeMap` is the standard library structure that keeps the $O(\log n)$ bound under any insertion order.
- The file has no tests. `main` prints values without asserting them, so a fault in the ordering invariant appears as output on the terminal rather than as a failing test.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 12, "Binary Search Trees".
- Standard library, `std::cmp::Ordering`.
- Standard library, `std::mem::take`.
- Standard library, `BTreeMap`.

Binary Search on a Rotated Array

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/binary_search.rs`. Test it with `cargo test binary_search`.

Problem Statement

A sorted slice of distinct values has been rotated by an unknown amount. Find the index of a target value in $O(\log n)$, or report that it is absent.

```
[4, 5, 6, 7, 0, 1, 2]      a rotation of [0, 1, 2, 4, 5, 6, 7]
      sorted          sorted
```

Designing a Solution

The loop keeps a half-open range `[low, high)` and the invariant

the target, if it is present, lies in `nums[low..high]`.

In a rotated array, at least one half of any range is sorted. Testing `nums[low] <= nums[mid]` says which half. If the left half is sorted, a range test decides whether the target can be in it; if it cannot, the target must be in the right half. The test costs one comparison, which is what keeps the loop logarithmic.

```
nums = [4, 5, 6, 7, 0, 1, 2]      target = 0

  low  high  mid  nums[mid]  left sorted?  decision
   0    7    3    7         yes, 4 <= 7  target not in [4, 7) -> low = 4
   4    7    5    1        no, 0 <= 1  target not in (1, 2] -> high = 5
   4    5    4    0         found                      -> Some(4)
```

Implementation

```

/// Binary search on a rotated sorted array.
///
/// Classic  $O(\log n)$  interview problem. The code uses explicit range bounds and
/// sorted-half checks; no recursion or nested `if` chains are required.

pub fn search_rotated(nums: &[i32], target: i32) -> Option<usize> {
    let mut low = 0;
    let mut high = nums.len();

    while low < high {
        let mid = low + (high - low) / 2;
        if nums[mid] == target {
            return Some(mid);
        }

        // 6,7, 1,2,3,4,5
        // start = 0 , end = 6, 0 + (6-0)/2 = 3

        // 4,5,6,1,2,3
        let left_is_sorted = nums[low] <= nums[mid];
        if left_is_sorted {
            if nums[low] <= target && target < nums[mid] {
                high = mid;
            } else {
                low = mid + 1;
            }
        } else if nums[mid] < target && target <= nums[high - 1] {
            low = mid + 1;
        } else {
            high = mid;
        }
    }

    None
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn finds_target_in_rotated_array() {
        let nums = [4, 5, 6, 7, 0, 1, 2];
        assert_eq!(search_rotated(&nums, 0), Some(4));
        assert_eq!(search_rotated(&nums, 3), None);
        assert_eq!(search_rotated(&nums, 5), Some(1));
    }

    #[test]
    fn handles_unrotated_and_single() {
        assert_eq!(search_rotated(&[1, 2, 3, 4], 3), Some(2));
        assert_eq!(search_rotated(&[1], 1), Some(0));
        assert_eq!(search_rotated(&[1], 2), None);
    }
}

```

`high` starts at `nums.len()`, not at `nums.len() - 1`, because the range is half-open. That choice removes the special case for an empty slice: `low == high == 0` and the loop does not run.

`let mid = low + (high - low) / 2` avoids the overflow that `(low + high) / 2` would have in a debug build for a slice near the maximum length. The midpoint satisfies `low <= mid < high`, so `nums[mid]` and `nums[high - 1]` are both in bounds.

The three tests of the left half are written as value comparisons, not as searches. `nums[low] <= target && target < nums[mid]` is a half-open interval test: the lower bound is inclusive and the upper bound is exclusive, matching the range convention used throughout.

Intuition

```

nums = [4, 5, 6, 7, 0, 1, 2]          target = 3, absent

  low  high  mid  nums[mid]  left sorted?  test                                next
  0     7     3     7        yes, 4 <= 7   4 <= 3? no                             low = 4
  4     7     5     1        no, 0 <= 1    1 < 3 and 3 <= nums[6]=2? no         high = 5
  4     5     4     0        found? no      left sorted? 0 <= 0 yes
                                           0 <= 3 and 3 < 0? no
  5     5     -     -        low == high, loop ends
                                           low = 5
                                           None

```

Time and Space Complexity

Resource	Cost
Time	$O(\log n)$, the range halves on every iteration
Space	$O(1)$, three indices and no recursion

Limitations

Repeated values break the sorted-half test. With duplicates, `nums[low] <= nums[mid]` can be true for both halves, so the code can discard the half holding the target. On `[1, 1, 1, 0, 1]` the search for `0` fails. The function is correct only for strictly increasing inputs, which the problem statement guarantees and the documentation does not mention.

The returned index is not the first or the last match. With duplicates, any index holding the target is a valid answer for the problem as stated, and callers that want a boundary need a different function.

The slice must be sorted and then rotated, not merely rotated. An arbitrary input that happens to be two ascending runs is not enough; the runs

must be the tail and the head of one sorted sequence, which is what makes the value ranges meaningful.

There is no test for a two-element rotation. `[2, 1]` is the smallest input where `nums[low] == nums[mid]` is false and the sorted-half test must choose correctly. That input is the one the `<` versus `<=` in `left_is_sorted` controls.

Summary

- The loop invariant is that the target lies in `nums[low..high]` if it lies anywhere.
- At least one half is sorted. The rotation point lies in one half or the other, and the half that does not contain it is a contiguous piece of the original sorted array.
- The half-open range convention is used consistently. Mixing `high = nums.len()` with `nums[high]` is the usual source of an out-of-bounds index here.
- Duplicates are not handled. On `[1, 1, 1, 0, 1]` the search for `0` fails, because `nums[low] <= nums[mid]` can hold for both halves, so the function is correct only for strictly increasing input.

References

- Standard library, `slice::binary_search`, the unrotated case.
- Standard library, `usize`, whose arithmetic the midpoint formula is written to avoid overflowing.

Trie

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/trie.rs`. Test it with `cargo test trie`.

Problem Statement

Store a set of strings so that two questions are cheap: is this exact string present, and does any stored string begin with this prefix. The second question is the reason the structure exists; a hash map answers the first in constant time and the second only by scanning every key.

Designing a Solution

Each node holds a map from a character to a child node, plus a flag saying whether the path that ends at this node spells a stored string. A string is present when walking its characters from the root succeeds and the final node has the flag set.

```
words: "gpu", "gpucloud"

      (root)
      |
      g
      |
      p
      |
      u   <- terminal: "gpu" is a word
      |
      c
      |
      l
      |
      o
      |
```

```

u
|
d  <- terminal: "gpucloud" is a word

```

The flag is what distinguishes a stored word from a prefix of a longer word. Without it, "gp" and "gpu" would be indistinguishable, and `search("gp")` would report `true`.

Implementation

```

///! Trie / prefix tree.
///!
///! Relevant to NVIDIA-style SDK/CLI/codegen work: autocomplete, filtering GPU
///! SKUs, command completion, and prefix matching.

use std::collections::HashMap;

#[derive(Default)]
pub struct Trie {
    root: Node,
}

#[derive(Default)]
struct Node {
    children: HashMap<char, Node>,
    terminal: bool,
}

impl Trie {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn insert(&mut self, word: &str) {
        let mut node = &mut self.root;
        for ch in word.chars() {
            node = node.children.entry(ch).or_default();
        }
        node.terminal = true;
    }

    pub fn search(&self, word: &str) -> bool {
        self.find_node(word).is_some_and(|node| node.terminal)
    }

    pub fn starts_with(&self, prefix: &str) -> bool {
        self.find_node(prefix).is_some()
    }

    fn find_node(&self, word: &str) -> Option<&Node> {
        let mut node = &self.root;
        for ch in word.chars() {
            node = node.children.get(&ch)?;
        }
        Some(node)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

```

```

#[test]
fn trie_insert_search_and_prefix() {
    let mut trie = Trie::new();
    trie.insert("gpu");
    trie.insert("gpucloud");

    assert!(trie.search("gpu"));
    assert!(trie.search("gpucloud"));
    assert!(!trie.search("gp"));
    assert!(!trie.search("gpucloudapi"));

    assert!(trie.starts_with("gp"));
    assert!(trie.starts_with("gpucloud"));
    assert!(!trie.starts_with("h100"));
}
}

```

`insert` walks with `&mut` and creates children on demand: `node.children.entry(ch).or_default()` inserts an empty node the first time the character is seen and returns a mutable reference to the existing one otherwise. The walk ends with a mutable reference to the final node, and one assignment sets the flag.

`find_node` is the read-only counterpart, and it uses `?` to leave the loop early: `node.children.get(&ch)?` returns `None` from the function when the character is absent. That one expression replaces a conditional and a `return`.

`is_some_and(|node| node.terminal)` combines two questions into one expression: does the path exist, and is its end a word. Without the second part, `search` would answer the same question as `starts_with`.

Intuition

```

insert("gpu")
    node = root
    'g' -> root.children has no 'g', so it is created; node = that node
    'p' -> created; node = that node
    'u' -> created; node = that node
    node.terminal = true

insert("gpucloud")
    'g', 'p', 'u' all exist, so the walk reuses the nodes built above
    'c', 'l', 'o', 'u', 'd' are created
    node.terminal = true

search("gp")           the walk succeeds, the final node's terminal flag is false
                        -> false
starts_with("gp")      the walk succeeds -> true
search("gpucloudapi")
                        'a' is absent after "gpucloud" -> the walk returns None
                        -> false

```

Time and Space Complexity

Operation	Time	Space
insert	$O(k)$ expected	one node per character not already present
search	$O(k)$ expected	none
starts_with	$O(k)$ expected	none

k counts characters, because `word.chars()` iterates over Unicode scalar values.

Limitations

One node per distinct character position. The structure stores a `HashMap` per node, and a `HashMap` is larger than the characters it holds. A set of strings with no shared prefixes occupies more memory than the same strings in a vector. A radix tree compresses chains of single-child nodes and recovers most of that space; the file does not implement one.

There is no deletion. Removing a word means clearing a terminal flag and then pruning the nodes that are no longer on any path to a word. Neither operation is present, so the only way to empty the structure is to drop it.

There is no query that returns the words under a prefix. `starts_with` answers a boolean, and a caller that wants the completions must write its own traversal. The structure supports the query; the API does not expose it.

The node type is not public, and neither is the root. A caller cannot inspect the tree's shape, count its nodes, or measure its memory from outside the module. For a teaching structure that is a defensible choice, and it means a memory question can only be answered by reading the source.

Summary

- The trie's advantage over a hash map is prefix queries in $O(k)$. Its cost is one node per character position, each holding a `HashMap` larger than the

characters it contains, so a set of strings with no shared prefixes occupies more memory than the same strings in a vector. A radix tree compresses chains of single-child nodes and recovers part of that space; the file does not implement one.

- The terminal flag is what distinguishes a stored key from a prefix of a longer key. A key can be both at once, so the flag cannot be inferred from the presence of children.
- `find_node` returns `Option<&Node>` and `search` post-processes it, which separates the traversal from the question asked of it. One traversal answers both `search` and `starts_with`.

References

- Edward Fredkin, "Trie memory", *Communications of the ACM* 3(9), 1960, pages 490–499.
- Donald E. Knuth, *The Art of Computer Programming*, Volume 3, 2nd edition, Addison-Wesley, 1998, Section 6.3, "Digital Searching".
- Standard library, `HashMap::entry`.
- Standard library, `Option::is_some_and`, stabilised in Rust 1.70.0.

Topological Sort

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/graph_topology.rs`. Test it with `cargo test graph_topology`.

Problem Statement

Given a directed graph, produce an ordering of its vertices in which every edge points forward. If the graph contains a cycle, no such ordering exists and the function reports that instead.

```
edges:  0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3

  0 -----> 1
  |           |
  v           v
  2 -----> 3

valid orders: [0, 1, 2, 3], [0, 2, 1, 3]
```

Designing a Solution

Kahn's algorithm. A vertex can be placed as soon as every vertex that must precede it has been placed, so counting the number of unplaced predecessors, the in-degree, is enough to decide when a vertex becomes available.

```
in-degree of each vertex: 0:0   1:1   2:1   3:2
ready queue: [0]

take 0 -> order [0]           in-degree 1:0, 2:0           queue [1, 2]
take 1 -> order [0, 1]       in-degree 3:1               queue [2]
take 2 -> order [0, 1, 2]   in-degree 3:0               queue [3]
take 3 -> order [0, 1, 2, 3]                                queue []
```

Four vertices were placed out of four, so the graph is acyclic. If the loop ends with fewer vertices placed than the graph holds, the vertices left in the queue are empty and the unplaced vertices are exactly those that lie on or after a cycle: each vertex of a cycle waits for another vertex of the same cycle, so none of them ever reaches in-degree zero.

The check is therefore one comparison at the end, and the algorithm detects cycles without a separate traversal.

Implementation

```
/// Topological sort (Kahn's algorithm).
///
/// Common in NVIDIA-style scheduling questions: prerequisite graphs, pipeline
/// stages, dependency ordering of GPU jobs.

use std::collections::VecDeque;

pub fn topological_sort(num_nodes: usize, edges: &[(usize, usize)]) -> Option<Vec<usize>> {
    let mut graph = vec![Vec::new(); num_nodes];
    let mut indegree = vec![0; num_nodes];

    for &(from, to) in edges {
        graph[from].push(to);
        indegree[to] += 1;
    }

    let mut queue: VecDeque<usize> = (0..num_nodes).filter(|&n| indegree[n] == 0).collect();
    let mut order = Vec::with_capacity(num_nodes);

    while let Some(node) = queue.pop_front() {
        order.push(node);

        for &next in &graph[node] {
            indegree[next] -= 1;
            if indegree[next] == 0 {
                queue.push_back(next);
            }
        }
    }

    (order.len() == num_nodes).then_some(order)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn returns_order_for_dag() {
        // 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
        let edges = [(0, 1), (0, 2), (1, 3), (2, 3)];
        let order = topological_sort(4, &edges).expect("DAG has an order");
        assert_eq!(order.len(), 4);
        assert!(
            order.iter().position(|&n| n == 0).unwrap()
                < order.iter().position(|&n| n == 1).unwrap()
        );
    }
}
```

```

    );
    assert!(
        order.iter().position(|&n| n == 1).unwrap()
            < order.iter().position(|&n| n == 3).unwrap()
    );
}

#[test]
fn detects_cycle() {
    // 0 -> 1 -> 0 is a cycle.
    let edges = [(0, 1), (1, 0)];
    assert_eq!(topological_sort(2, &edges), None);
}

#[test]
fn disconnected_nodes_are_included() {
    let edges = [(0, 1)];
    let order = topological_sort(3, &edges).expect("DAG has an order");
    assert_eq!(order.len(), 3);
}
}

```

`vec![Vec::new(); num_nodes]` builds the adjacency list with one vector per vertex. The inner vectors grow as edges are added, so the total allocation is proportional to the edge count.

`queue` is seeded with every vertex whose in-degree is already zero. A `VecDeque` is used because vertices arrive at the back and leave from the front: `pop_front` and `push_back` are both constant time, and a `Vec` removing from the front would be linear.

`indegree[next] -= 1` runs once per edge leaving `node`, so the total work of the inner loop across the whole algorithm is the edge count. That is what makes the cost $O(V + E)$ rather than something quadratic.

`(order.len() == num_nodes).then_some(order)` consumes `order` only when the comparison is true. If the graph has a cycle, the partially built order is dropped and the caller receives `None`, so no caller can accidentally schedule a partial order.

Intuition

```
num_nodes = 4, edges = [(0, 1), (0, 2), (1, 3), (2, 3)]
```

```
adjacency: 0 -> [1, 2]    1 -> [3]    2 -> [3]    3 -> []
indegree: [0, 1, 1, 2]
```

step	queue	pop	order	in-degree after	queue after
1	[0]	0	[0]	1:0, 2:0	[1, 2]
2	[1, 2]	1	[0, 1]	3:1	[2]
3	[2]	2	[0, 1, 2]	3:0	[3]
4	[3]	3	[0, 1, 2, 3]	-	[]

```
order.len() == 4 == num_nodes, so the result is Some(order).
```

A cycle: `num_nodes = 2, edges = [(0, 1), (1, 0)]`

`indegree: [1, 1]`

the queue is seeded empty, the loop body never runs, order has length 0, and `0 != 2`, so the result is `None`.

Time and Space Complexity

Re-source	Cost
Time	$O(V + E)$, each vertex enters the queue once and each edge is relaxed once
Space	$O(V + E)$, the adjacency list and the two auxiliary vectors

Limitations

An edge naming a vertex outside `0..num_nodes` panics. `graph[from].push(to)` and `indegree[to] += 1` index directly, so `topological_sort(2, &[(5, 0)])` panics with an index-out-of-bounds, and the panic message names a vector index rather than the offending edge. A caller that reads a graph from a file must validate the vertex numbers first; the function gives it no help.

None does not say which cycle was found. A caller that wants to report the cycle must run a depth-first search over the unplaced vertices to find one. The information is available inside the algorithm, the unplaced vertices are exactly the ones the loop never reached, but the return type discards it.

The choice among valid orders is unspecified. For the diamond graph both `[0, 1, 2, 3]` and `[0, 2, 1, 3]` are correct, and which one comes out depends on the order of the edges in the input and on the vertex numbering. The tests assert only the properties that must hold, which is the correct way to test this function, and a caller must not depend on a particular order.

Duplicate edges are tolerated but not detected. Two copies of `(0, 1)` add two to the in-degree of vertex 1 and the same edge is relaxed twice, so

the counts stay consistent and the result is unaffected. Nothing reports the duplication, which a caller might want to know about.

Summary

- The invariant is that the in-degree array counts the unplaced predecessors of each vertex. The remainder of the algorithm follows from it.
- A short result is the cycle test. The unplaced vertices are exactly those the main loop never reached, so a count below `num_nodes` means those vertices carry a cycle.
- The cost is $O(V + E)$. The E term comes from the relaxation loop, which examines each edge once, rather than from a repeated scan of the edge list.
- An edge whose endpoints fall outside `0..num_nodes` panics with an index-out-of-bounds whose message names the vector index rather than the edge. A caller reading a graph from a file must validate the vertex numbers before calling the function.

References

- Arthur B. Kahn, "Topological sorting of large networks", *Communications of the ACM* 5(11), 1962, pages 558–562.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 20.4, "Topological sort".
- Standard library, `VecDeque`.
- Standard library, `bool::then_some`.

Part III: Recursion and Dynamic Programming

Coin Change

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/dp.rs`. Test it with `cargo test dp`.

I call it my billion-dollar mistake. It was the invention of the null reference in 1965.

, C. A. R. Hoare, QCon London, 2009

Problem Statement

Given a list of coin denominations and a target amount, return the smallest number of coins whose values sum exactly to the amount, or report that no combination exists. Each denomination may be used any number of times.

Designing a Solution

A greedy rule does not work. Taking the largest coin that fits fails for `coins = [1, 3, 4]` and `amount = 6`: the greedy choice of 4 leaves 2, which needs two more coins, for a total of three, while `3 + 3` needs two.

The recurrence is over amounts. Let `dp[t]` be the smallest number of coins that sums to `t`. The last coin used is some denomination `c`, and everything before it must sum to `t - c` in the smallest possible number of coins:

```
dp[0] = 0
dp[t] = min over c of ( dp[t - c] + 1 )    for each c <= t
```

Each entry depends only on entries at smaller amounts, so filling the table in ascending order of `t` visits every dependency before it is needed.

```
coins = [1, 2, 5], amount = 11
```

```
t      : 0  1  2  3  4  5  6  7  8  9 10 11
dp     : 0  1  1  2  2  1  2  2  3  3  2  3
```

```
dp[11] was built from dp[11 - 5] + 1 = dp[6] + 1 = 2 + 1 = 3
and 11 = 5 + 5 + 1
```

Implementation

```
///! Coin Change: minimum number of coins to make an amount.
///!
///! Classic dynamic programming. `Option<i32>` expresses "impossible" instead of
///! a magic `-1`.

pub fn coin_change(coins: &[i32], amount: i32) -> Option<i32> {
    if amount < 0 {
        return None;
    }

    let amount = amount as usize;
    let mut dp = vec![usize::MAX; amount + 1];
    dp[0] = 0;

    for total in 1..=amount {
        for &coin in coins {
            let coin = coin as usize;
            if coin > total || dp[total - coin] == usize::MAX {
                continue;
            }
            dp[total] = dp[total].min(dp[total - coin] + 1);
        }
    }

    match dp[amount] {
        usize::MAX => None,
        count => Some(count as i32),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn computes_minimum_coins() {
        assert_eq!(coin_change(&[1, 2, 5], 11), Some(3)); // 5 + 5 + 1
        assert_eq!(coin_change(&[2], 3), None);
        assert_eq!(coin_change(&[1], 0), Some(0));
    }
}
```

`usize::MAX` is the sentinel for "no combination reaches this amount yet". It is safe as a marker because a real answer can never be that large: any combination uses at most `amount` coins, because every coin is worth at least one.

The guard `if coin > total || dp[total - coin] == usize::MAX` does two things at once. It skips a coin that is too large to be the last coin, and it skips a coin whose remainder is itself unreachable. Without the second test the addition below would add one to the sentinel and produce a wrong answer that looks reachable.

`dp[total] = dp[total].min(dp[total - coin] + 1)` keeps the best count found so far for this amount. Because `dp[total]` is read and written in the same statement, the loop order over coins does not matter.

The boundary conversion is where the sentinel stops. `match dp[amount]` translates `usize::MAX` into `None` and everything else into `Some(count as i32)`, so no caller ever sees the marker. The type of the answer is `Option<i32>`, which is the part of the design that matters: "impossible" is not a number.

Intuition

```
coins = [1, 2, 5], amount = 11

total = 1  coin 1: dp[1] = min(MAX, dp[0] + 1) = 1
total = 2  coin 1: dp[2] = min(MAX, dp[1] + 1) = 2
           coin 2: dp[2] = min(2, dp[0] + 1) = 1
total = 3  coin 1: dp[3] = min(MAX, dp[2] + 1) = 2
           coin 2: dp[3] = min(2, dp[1] + 1) = 2
total = 4  coin 1: dp[4] = min(MAX, dp[3] + 1) = 3
           coin 2: dp[4] = min(3, dp[2] + 1) = 2
total = 5  coin 1: dp[5] = min(MAX, dp[4] + 1) = 3
           coin 2: dp[5] = min(3, dp[3] + 1) = 3
           coin 5: dp[5] = min(3, dp[0] + 1) = 1
...
total = 11 coin 1: dp[11] = min(MAX, dp[10] + 1) = 3
           coin 2: dp[11] = min(3, dp[9] + 1) = 3
           coin 5: dp[11] = min(3, dp[6] + 1) = 3
```

```
coins = [2], amount = 3

total = 1  coin 2 is larger than total, skipped;    dp[1] = MAX
total = 2  coin 2: dp[0] = 0, so dp[2] = 1
total = 3  coin 2: dp[1] = MAX, skipped;            dp[3] = MAX

dp[3] is the sentinel, so the result is None.
```

Time and Space Complexity

Resource	Cost	Condition
Time	$O(\text{amount} \times \text{coins.len}())$	every amount considers every denomination

Resource	Cost	Condition
Space	$O(\text{amount})$	one machine word per amount

Limitations

The table is allocated from the amount alone. `coin_change(&[1], i32::MAX)` asks for a vector of `i32::MAX + 1` machine words, which is sixteen gigabytes on a 64-bit target. The allocation failure aborts the process, because the standard library's allocation error handling calls `handle_alloc_error`, which aborts. A version that accepts an amount from the network would validate it against a limit first, or use `Vec::try_reserve` to turn the failure into a `Result`.

A negative coin is treated as a very large positive one. `coin` as `usize` converts `-1` to `usize::MAX`, and the guard `coin > total` then skips it on every iteration. The function returns an answer that ignores the negative coin, which is the wrong answer if the caller meant the coin to be usable. Nothing reports the conversion.

A zero coin is skipped by the same guard, correctly. `coin = 0` passes the first test only when `total` is zero, and the loop starts at one, so a zero denomination contributes nothing. That is the right behaviour and it is a consequence of the guard rather than a stated rule.

The function returns the count, not the coins. A caller that wants to know which coins to use needs the predecessor that produced each entry, which means a second table recording decisions. The current one-minimum-per-entry design cannot reconstruct the combination.

`i32` bounds both the amount and the answer. An amount above `i32::MAX` cannot be passed, and the conversion `count` as `i32` is safe only because the answer is at most the amount.

Summary

- The recurrence decides on the last coin: `dp[total]` is one more than the best `dp[total - coin]`, which is what makes the table a dynamic program rather than a sequence of locally optimal choices.

- A greedy choice is not correct here. For `[1, 3, 4]` and the amount `6`, greedy produces `4 + 1 + 1` and the optimum is `3 + 3`.
- The table holds `usize::MAX` as a sentinel for an unreachable amount, and the boundary converts it to `None`, so a caller never sees the sentinel. The conversion `count as i32` is safe because the answer is at most the amount.
- The allocation is controlled by the input. `coin_change(&[1], i32::MAX)` requests a vector of `i32::MAX + 1` machine words, and a failed allocation aborts the process.
- The function returns the count and not the coins. Reconstructing the combination needs the predecessor that produced each entry, which is a second table.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Chapter 14, "Dynamic Programming".
- Richard Bellman, *Dynamic Programming*, Princeton University Press, 1957.
- Standard library, `usize::MAX`.
- Standard library, `Vec::try_reserve`.

Subsets

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/backtracking.rs`. Test it with `cargo test backtracking`.

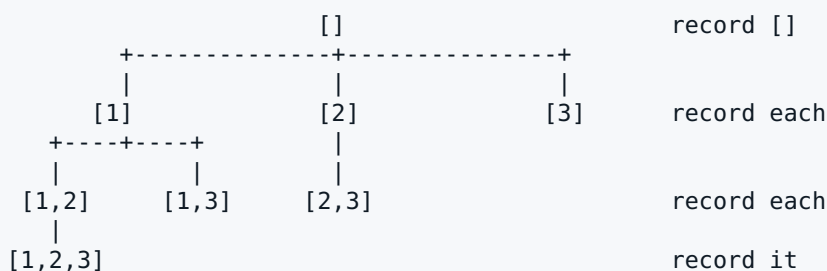
Problem Statement

Given a list of distinct values, return every subset. A list of n values has 2^n subsets, because each value is independently either present or absent, and the empty subset and the whole list are both included.

Designing a Solution

Backtracking walks a sequence of decisions. At each step the algorithm holds a partial selection and a position, records the selection, and then tries every value from that position onward as the next member. After each trial it undoes the choice, which is what allows the next trial to start from the same state.

values = [1, 2, 3]



8 records: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]

The invariant that makes the undo step correct is: on entry to the recursive call, `selection` holds a prefix of a decision sequence; on exit it holds exactly

what it held on entry. Every branch therefore starts from a state that the branches before it did not disturb.

The restriction that each recursive call may only choose a value at or after `start` is what prevents `[1, 2]` and `[2, 1]` from both appearing. The output is ordered by index, not by value.

Implementation

```

///! Subsets: enumerate every subset of a set of distinct integers.
///! Backtracking with clean `path.push(...)` + recursive exploration + `pop`.
///! `Vec<Vec<i32>>` is sorted in the tests to avoid order-dependent assertions.

pub fn subsets(nums: Vec<i32>) -> Vec<Vec<i32>> {
    let mut result = Vec::with_capacity(1 << nums.len());
    let mut path = Vec::new();

    fn backtrack(nums: &[i32], start: usize, path: &mut Vec<i32>, result: &mut Vec<Vec<i32>>) {
        result.push(path.clone());
        for idx in start..nums.len() {
            path.push(nums[idx]);
            backtrack(nums, idx + 1, path, result);
            path.pop();
        }
    }

    backtrack(&nums, 0, &mut path, &mut result);
    result
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn generates_all_subsets() {
        let mut subsets = subsets(vec![1, 2, 3]);
        subsets.sort_unstable();
        assert_eq!(subsets.len(), 8);
        assert!(subsets.contains(&vec![]));
        assert!(subsets.contains(&vec![1, 2, 3]));
    }

    #[test]
    fn empty_input_has_one_subset() {
        assert_eq!(subsets(vec![]), vec![vec![]]);
    }
}

```

`fn backtrack(...)` is a nested function inside `subsets`. A nested `fn` does not capture its environment, so the values it needs are passed as parameters: `nums`, `start`, `path`, and `result`. That is why the signature is longer than the body: the parameters replace the closure capture that a nested `fn` cannot have.

`result.push(path.clone())` records the current selection. The clone is required because `path` is reused for the rest of the search; all 2^n recorded subsets are independent vectors.

`backtrack(&nums, idx + 1, path, result)` passes `idx + 1`, not `start + 1`. Each chosen value advances the lower bound past itself, so a value cannot be chosen twice and the indices in a subset always increase.

`path.pop()` after the recursive call restores the invariant. Removing that line makes the selection grow for the remainder of the search, and the output would contain subsets that were never selected.

Intuition

```
subsets(vec![1, 2, 3])

call      path      recorded      loop
backtrack(nums, 0)  []          []          idx 0, 1, 2
  push nums[0] = 1  [1]          [1]          idx 1, 2
    backtrack(nums, 1) [1]          [1]          idx 2
      push nums[1] = 2 [1, 2]        [1, 2]        idx 2
        backtrack(nums, 2) [1, 2]      [1, 2]        loop does not run
          push nums[2] = 3 [1, 2, 3]    [1, 2, 3]
            backtrack(nums, 3) [1, 2, 3]  [1, 2, 3]
              pop          [1, 2]
            pop          [1]
          push nums[2] = 3 [1, 3]          [1, 3]        loop does not run
        backtrack(nums, 3) [1, 3]
          pop          [1]
        pop          []
      push nums[1] = 2    [2]
        backtrack(nums, 2) [2]          [2]          idx 2
          push nums[2] = 3 [2, 3]        [2, 3]
            backtrack(nums, 3) [2, 3]      [2, 3]
              pop          [2]
            pop          []
          push nums[2] = 3 [3]
            backtrack(nums, 3) [3]          [3]
              pop          []

recorded, in order: [], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]
```

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(2^n \times n)$	one clone of the selection per recorded subset, of length up to n
Space	$O(2^n \times n)$	the output dominates; the stack is $O(n)$

The clone is the reason for the extra factor of n . A caller that only needs to visit each subset can pass a closure instead of collecting them, which removes the factor.

Limitations

The capacity hint overflows for large inputs. `Vec::with_capacity(1 << nums.len())` evaluates `1 << nums.len()` as a `usize`. For a list of 64 or more values on a 64-bit target, the shift is not representable and panics in a debug build. The panic happens on an input the function is otherwise able to accept: the recursion itself has no such limit, and the allocation would fail long before the work finished. The hint is a promise the code cannot keep.

A list of 40 values cannot be enumerated at all. 2^{40} subsets is about a trillion vectors, and the output alone would exceed any machine's memory. The function has no bound on the input size and no way to report that the work is impossible, so the caller has to know.

Duplicate values produce duplicate-looking subsets. The function takes a list and treats every position as distinct, so `subsets(vec![1, 1, 1])` returns eight subsets (five of them empty-looking). The documentation says "distinct integers", which is a precondition the type does not enforce.

The output order is an implementation detail, and one test depends on it. `empty_input_has_one_subset` compares the whole output, which is safe for one element. The other test sorts before comparing, which is the right pattern; `generates_all_subsets` follows it.

There is no way to stop the search early or to stream the results. A caller that wants the first k subsets, or wants to search for a subset with a property, must collect all of them first.

Summary

- The recording happens at the start of each call, before the loop. That is what produces the empty subset and what makes every node of the recursion tree correspond to one output.

- The output is 2^n subsets, and cloning the working path into each one adds a factor of n . Replacing the collected vectors with a visitor closure removes the factor, which is a change of interface rather than a micro-optimisation.
- The `push` and `pop` pair is a single operation. The `pop` is what makes the next branch independent, not cleanup after the current one.
- The input size is unbounded. A list of 40 values produces about a trillion subsets, and the capacity hint `1 << nums.len()` overflows for 64 values on a 64-bit target.
- The documentation states "distinct integers" as a precondition the type does not enforce, so repeated values produce repeated-looking subsets.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 5.3, on recurrences, and Chapter 14 on the same recursive-search structure.
- Standard library, `Vec::clone`.
- Standard library, `Vec::with_capacity`.

Part IV: Linked Structures

Reverse a Linked List

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

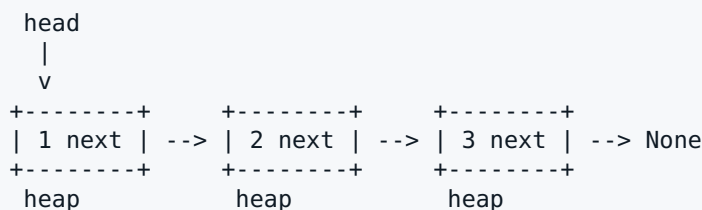
Source file: `src/problems/linked_list.rs`. Test it with `cargo test linked_list`.

Problem Statement

Reverse a singly linked list in place and return its new head. In place means no additional node allocations: the nodes are re-linked, not copied.

Designing a Solution

```
head: Option<Box<ListNode>>
```



Three properties follow from `Option<Box<ListNode>>`.

`None` is the end of the list, so an empty list is the value `None` rather than a node with a flag, and there is no sentinel to keep consistent.

The list owns its nodes. Handing over the head hands over the whole list, so there is no way to hold a list whose nodes have been freed and no separate deallocation call.

A link is one pointer wide. The compiler stores `None` in the null pointer, which is never a valid address for a `Box`, so no discriminant byte is added.

Reversal is one loop over three bindings. In a language with unrestricted pointers it is three assignments per node; in Rust, the same algorithm is written by taking ownership of one node at a time, which is what makes it checkable.

```
initial    previous = None                current = Some(1 -> 2 -> 3 -> None)

step 1     current = node.next.take()    current = Some(2 -> 3 -> None)
           node.next = previous          node      = Some(1 -> None)
           previous = Some(node)          previous = Some(1 -> None)

step 2     current = Some(3 -> None)      previous = Some(2 -> 1 -> None)
step 3     current = None                 previous = Some(3 -> 2 -> 1 -> None)

the loop ends because current is None, and the answer is previous
```

`Option::take` is the operation that makes this work. It moves the successor out of `node.next` and leaves `None` behind, so the node under the loop's cursor stands alone while the rest of the list is held in a separate binding. Without `take`, the assignment `node.next = previous` would have to write through a reference that `current` still holds, which the borrow checker rejects.

Implementation

```
/// Reverse a singly linked list.
///
/// Uses `Option<Box<ListNode>>` and a `while let` loop to move pointers in a
/// borrow-safe way.

#[derive(Debug, PartialEq, Eq)]
pub struct ListNode {
    pub val: i32,
    pub next: Option<Box<ListNode>>,
}

impl ListNode {
    pub fn from_slice(values: &[i32]) -> Option<Box<ListNode>> {
        let mut head = None;
        for &value in values.iter().rev() {
            head = Some(Box::new(ListNode {
                val: value,
                next: head,
            }));
        }
        head
    }

    pub fn to_vec(head: Option<Box<ListNode>>) -> Vec<i32> {
        let mut result = Vec::new();
        let mut current = head;
        while let Some(node) = current {
            result.push(node.val);
            current = node.next;
        }
        result
    }
}
```

```

pub fn reverse_list(head: Option<Box<ListNode>>) -> Option<Box<ListNode>> {
    let mut previous = None;
    let mut current = head;

    while let Some(mut node) = current {
        current = node.next.take();
        node.next = previous;
        previous = Some(node);
    }

    previous
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn reverses_list() {
        let head = ListNode::from_slice(&[1, 2, 3, 4]);
        assert_eq!(ListNode::to_vec(reverse_list(head)), vec![4, 3, 2, 1]);
    }

    #[test]
    fn handles_empty_and_single() {
        assert_eq!(reverse_list(None), None);
        assert_eq!(
            ListNode::to_vec(reverse_list(ListNode::from_slice(&[42]))),
            vec![42]
        );
    }
}

```

`from_slice` builds the list backwards, from the last value to the first, so each new node takes the list built so far as its successor. Building forwards would need a mutable reference to the tail, which is more code for the same result.

`to_vec` takes the list by value. That lets each node be dropped as the traversal proceeds: the value is copied out and the successor moves into `current`, so the node that was just read is freed at the end of the iteration. A version that took `&Option<Box<ListNode>>` would leave the list alive and would need the caller to drop it afterwards.

`while let Some(mut node) = current` binds the node by value, and the `mut` is required because the body assigns to `node.next`. The loop's three statements are the algorithm: detach the successor, point backwards, advance `previous`.

Intuition

input: 1 -> 2 -> 3 -> None

iteration	node	current after take	node.next after assignment	previous
start	-	Some(1 -> 2 -> 3)	-	None
1	1	Some(2 -> 3)	None	Some(1)
2	2	Some(3)	Some(1)	Some(2 -> 1)
3	3	None	Some(2 -> 1)	Some(3 -> 2 -> 1)

1)

current is None, so the loop ends and previous is returned.

Time and Space Complexity

Operation	Time	Space
reverse_list	$O(n)$	$O(1)$; no allocation, only pointer writes
from_slice	$O(n)$	$O(n)$, one allocation per node
to_vec	$O(n)$	$O(n)$ for the result, and it frees the nodes

Limitations

A long list is dropped recursively. The compiler's derived destructor for `Option<Box<ListNode>>` follows the `next` links, one stack frame per node. A list of a few hundred thousand nodes overflows the thread stack when it is dropped, and a stack overflow aborts the process rather than unwinding. This affects every caller, including one that only reverses a list that was built elsewhere: the destructor runs when the last owner goes out of scope.

to_vec consumes the list. A caller that wants both the reversed list and its values has to clone one of them, and `ListNode` does not implement `Clone`.

from_slice allocates one box per element. For a long slice that is a long chain of small allocations, each of which carries its own allocator overhead. An array-based representation holds the same data in one allocation, at the cost of not being a linked list.

The value type is fixed at i32. Nothing in the algorithm depends on the value type, and the declaration is not generic.

Summary

- The loop body is three lines, and `take` is the part that requires explanation. The successor has to leave the node before the node can be re-pointed, which is why `node.next` cannot be assigned directly.

- Reversal is $O(n)$ in time and $O(1)$ in space. The nodes already exist, so the operation adds no allocation.
- The derived destructor for `Option<Box<ListNode>>` follows the `next` links one stack frame per node, so dropping a list of a few hundred thousand nodes aborts the process. A manual `Drop` implementation that unlinks the chain in a loop removes that hazard.

References

- The Rust Book, [Enabling recursive types with boxes](#).
- Standard library, `Option::take`.
- Standard library, `Box`.

Singly Linked List

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/singly_linked_list.rs`. Run it with `cargo run --bin singly_linked_list`.

Problem Statement

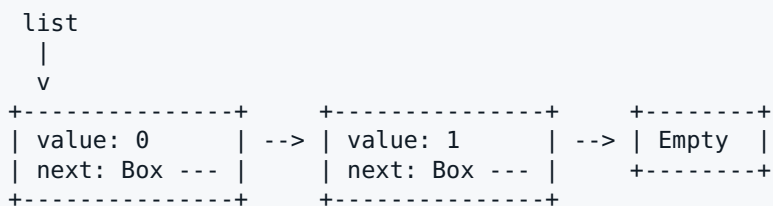
Build a list whose node set is described by an algebraic data type, with constant time insertion and removal at the front.

Designing a Solution

```
List<T> = Empty
      | Node { value: T, next: Box<List<T>> }
```

This is the same machine layout as `Option<Box<ListNode>>` from Chapter 14, with the empty case named in the type rather than expressed by `None`. The end of the list is `List::Empty`, and an empty list is `List::Empty` as well, so there is no separate sentinel and no nullable field.

```
list = Node { value: 0, next: Box<Node { value: 1, next: Box<Empty> }> }
```



The difference from the previous chapter is one of emphasis. `Box<Option<...>>` would have been the type of the earlier link; here the `Option` is gone, and every

link is a `Box` whose pointee is either a node or the empty list. Both designs compile to the same layout, and this one makes the "end of list" case part of the type instead of a field value.

`push_front` and `pop_front` both work by taking the whole list out of `self` with `std::mem::replace`, deciding what to do with it, and writing the result back. That pattern is what lets the code move a list value without cloning it and without holding a borrow of `self` across the move.

```
push_front(0) on [1, 2]

before: self = Node { 1, next: [2] }

replace(self, Empty) returns the old list [1, 2] and leaves self = Empty
then:    self = Node { 0, next: Box::new([1, 2]) }

pop_front() on [0, 1, 2]

replace(self, Empty) returns [0, 1, 2] and leaves self = Empty
the node is destructured into value = 0 and next = Box([1, 2])
then:    self = *next = [1, 2]
returns: Some(0)
```

Implementation

```
// Singly linked list using an algebraic data type.
//
//     List<T> = Empty
//             | Node { value: T, next: Link<T> }
//
// `Empty` is the end of the list.

type Link<T> = Box<List<T>>;

pub enum List<T> {
    Empty,
    Node { value: T, next: Link<T> },
}

impl<T> List<T> {
    pub fn new() -> Self {
        List::Empty
    }

    pub fn is_empty(&self) -> bool {
        matches!(self, List::Empty)
    }

    pub fn len(&self) -> usize {
        match self {
            List::Empty => 0,
            List::Node { next, .. } => 1 + next.len(),
        }
    }

    /// O(1)
    pub fn push_front(&mut self, value: T) {
        let old = std::mem::replace(self, List::Empty);
```

```

        *self = List::Node {
            value,
            next: Link::new(old),
        };
    }

    /// O(1)
    pub fn pop_front(&mut self) -> Option<T> {
        match std::mem::replace(self, List::Empty) {
            List::Empty => None,
            List::Node { value, next } => {
                *self = *next;
                Some(value)
            }
        }
    }
}

fn main() {
    let mut list = List::new();
    list.push_front(2);
    list.push_front(1);
    list.push_front(0);

    assert_eq!(list.len(), 3);
    assert_eq!(list.pop_front(), Some(0));
    assert_eq!(list.pop_front(), Some(1));
    assert_eq!(list.pop_front(), Some(2));
    println!("ok");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn push_pop_works() {
        let mut list = List::new();
        list.push_front(1);
        list.push_front(2);
        assert_eq!(list.pop_front(), Some(2));
        assert_eq!(list.pop_front(), Some(1));
        assert_eq!(list.pop_front(), None);
    }

    #[test]
    fn len_works() {
        let mut list = List::new();
        assert_eq!(list.len(), 0);
        list.push_front(1);
        list.push_front(2);
        assert_eq!(list.len(), 2);
    }
}

```

`push_front` moves the old list into a new `Box`. The old value is obtained by `std::mem::replace(self, List::Empty)`, which leaves `self` in a valid state that the following assignment overwrites. There is no moment at which `self` is uninitialised, which is what makes the code safe without `unsafe` and without `Option`.

`pop_front` deconstructs the taken value. `*self = *next` moves the inner list out of its box and into `self`; the box itself is freed, so the popped node is released rather than remembered.

`Link::new(old)` is `Box::new(old)` written through the type alias, which reads as "a new link holding the old list".

The two functions carry `/// 0(1)` doc comments and nothing else. They are the only documentation in the file beyond the module header and the type sketch.

Intuition

```

push_front(2)  self = Node { 2, next: Empty }
push_front(1)  self = Node { 1, next: Node { 2, next: Empty } }
push_front(0)  self = Node { 0, next: Node { 1, next: Node { 2, next: Empty } } }

len()          three recursive calls plus the empty base case, returning 3

pop_front()    replace returns the node holding 0
               value = 0, next = Box(Node { 1, ... })
               self = Node { 1, next: Node { 2, next: Empty } }
               returns Some(0)

pop_front()    returns Some(1), self = Node { 2, next: Empty }
pop_front()    returns Some(2), self = Empty
pop_front()    replace returns Empty, so the match yields None

```

Time and Space Complexity

Operation	Time	Space
<code>push_front</code>	$O(1)$	one allocation
<code>pop_front</code>	$O(1)$	freed one allocation
<code>is_empty</code>	$O(1)$	none
<code>len</code>	$O(n)$	$O(n)$ stack frames

Limitations

`List::new` has no Default implementation. Clippy's `new_without_default` lint fires on this pattern, and the reason it exists is that a caller writing a generic function over many types expects `Default` to work. Adding `impl<T> Default for List<T>` with a two-line body fixes it.

len is recursive. It walks the list with one stack frame per node, so a list that is long enough to matter cannot be measured. An iterative version keeps a `&List<T>` cursor and counts in a loop, which is the same code with the recursion replaced by a `while let`.

Dropping the list is recursive. The derived destructor for `List<T>` follows the `next` links, one frame per node. A long list aborts the process when it goes out of scope. This is the same hazard as Chapter 14, and the same fix applies: a manual `Drop` implementation that unlinks nodes into a loop.

pop_front on a list of a million nodes is constant time, and the drop that follows is not. The two facts belong together: the operation the API advertises is cheap, and the work the program does when the list is released is not.

There is no Clone, no Debug, and no iterator. A caller cannot print the list, cannot copy it, and cannot write `for value in &list`.

Summary

- `List::new` has no `Default` implementation, which is the pattern Clippy's `new_without_default` lint reports. A caller writing a generic function over many types expects `Default` to be available.
- `len` recurses, one stack frame per node, so a list long enough to matter cannot be measured. An iterative version keeps a `&List<T>` cursor and counts in a `while let` loop.
- Dropping the list recurses, one frame per node, so a long list aborts the process when it goes out of scope. This is the hazard described in Chapter 14, and the same manual `Drop` implementation removes it.
- `pop_front` is $O(1)$ on a list of any length, but the drop that follows is $O(n)$, so the two operations have to be considered together.
- The type implements neither `Clone` nor `Debug` and provides no iterator.

References

- The Rust Book, [Defining an enum](#).
- Standard library, `std::mem::replace`.
- Clippy, `new_without_default`.

Doubly Linked List

In this chapter

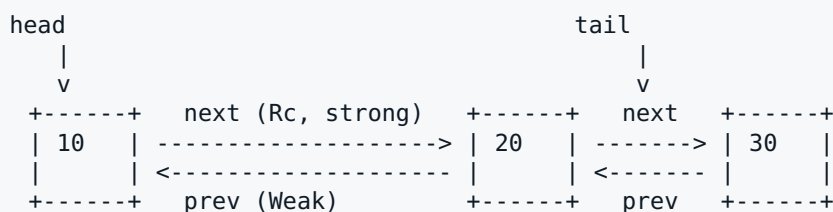
- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/ll.rs`. Run it with `cargo run --bin ll`.

Problem Statement

Build a list with constant time insertion and removal at both ends. A singly linked list cannot remove from the back in constant time, because reaching the penultimate node requires a walk. Something has to point backwards, and that something cannot own what it points at.

Designing a Solution



If both directions owned their target, the reference count of a two-node list would never reach zero and the nodes would leak: each node would be kept alive by the other. The resolution is to let the forward links own and the backward links observe:

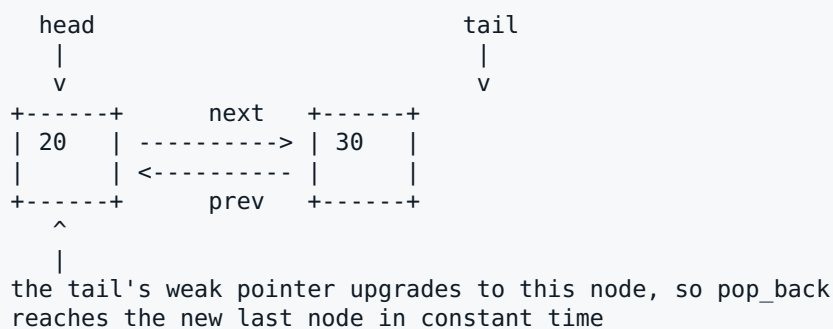
Link	Type	Keeps the node alive	Fails how
<code>next</code>	<code>Rc<RefCell<Node<T>>></code>	yes	the list outlives the value
<code>prev</code>	<code>Weak<RefCell<Node<T>>></code>	no	<code>upgrade()</code> returns <code>None</code>

A `Weak` pointer observes a value without keeping it alive. When the last strong reference goes away, the node is freed and every weak pointer to it starts returning `None` from `upgrade`. That is the mechanism that makes the structure agree with the ownership rules instead of working around them.

The `RefCell` is the second half of the design. `Rc` alone gives shared ownership but not mutation; `RefCell` adds mutation with the borrow checked at run time rather than at compile time.

after `pop_front` on the list above

(node 10 is freed: nothing strong refers to it)



Implementation

```
// =====
// Type aliases for clarity.
// =====
use std::cell::{Ref, RefCell}; // Ref is needed for peek return types
use std::rc::{Rc, Weak};

type NodeRef<T> = Rc<RefCell<Node<T>>>;
type WeakNodeRef<T> = Weak<RefCell<Node<T>>>;
type OptNodeRef<T> = Option<NodeRef<T>>;
type OptWeakNodeRef<T> = Option<WeakNodeRef<T>>;

// =====
// Node and LinkedList definitions.
// =====
struct Node<T> {
    data: T,
    next: OptNodeRef<T>,
    prev: OptWeakNodeRef<T>,
}

pub struct LinkedList<T> {
    head: OptNodeRef<T>,
    tail: OptNodeRef<T>,
    len: usize,
}

impl<T> LinkedList<T> {
    /// Creates an empty list.
    pub fn new() -> Self {
        LinkedList {
            head: None,

```

```

        tail: None,
        len: 0,
    }
}

pub fn len(&self) -> usize {
    self.len
}

pub fn is_empty(&self) -> bool {
    self.len == 0
}

// -----
// push_front
// -----
pub fn push_front(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        prev: None,
    }));

    // take() moves the old head out of the Option, leaving None.
    match self.head.take() {
        Some(old_head) => {
            // Link old head back to new node (weakly).
            old_head.borrow_mut().prev = Some(Rc::downgrade(&new_node));
            // Link new node forward to old head.
            new_node.borrow_mut().next = Some(old_head);
        }
        None => {
            // List was empty: new node is also the tail.
            self.tail = Some(new_node.clone());
        }
    }
    self.head = Some(new_node);
    self.len += 1;
}

// -----
// push_back
// -----
pub fn push_back(&mut self, data: T) {
    let new_node = Rc::new(RefCell::new(Node {
        data,
        next: None,
        // self.tail.as_ref() gives Option<&NodeRef<T>>,
        // map(Rc::downgrade) turns it into Option<WeakNodeRef<T>>.
        prev: self.tail.as_ref().map(Rc::downgrade),
    }));

    match self.tail.take() {
        Some(old_tail) => {
            old_tail.borrow_mut().next = Some(new_node.clone());
        }
        None => {
            self.head = Some(new_node.clone());
        }
    }
    self.tail = Some(new_node);
    self.len += 1;
}

// -----
// pop_front
// -----
pub fn pop_front(&mut self) -> Option<T> {

```

```

self.head.take().and_then(|old_head| {
    // Take ownership of the next node (if any).
    let next = old_head.borrow_mut().next.take();

    match next {
        Some(next_node) => {
            // Disconnect the new head's prev pointer.
            next_node.borrow_mut().prev = None;
            self.head = Some(next_node);
        }
        None => {
            self.tail = None; // list becomes empty
        }
    }

    self.len -= 1;

    // Safely extract the data. At this point, old_head is the only
    // strong reference (because we removed it from the list and the
    // new head's prev doesn't point to it).
    let node = Rc::try_unwrap(old_head).ok().unwrap().into_inner();
    Some(node.data)
})
}

// -----
// pop_back
// -----
pub fn pop_back(&mut self) -> Option<T> {
    self.tail.take().and_then(|old_tail| {
        // Get the previous node (upgrade the weak reference).
        let prev = old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade());

        match prev {
            Some(prev_node) => {
                prev_node.borrow_mut().next = None;
                self.tail = Some(prev_node);
            }
            None => {
                self.head = None; // list becomes empty
            }
        }

        self.len -= 1;

        let node = Rc::try_unwrap(old_tail).ok().unwrap().into_inner();
        Some(node.data)
    })
}

// -----
// peek_front: returns a guarded reference to the front data.
// The returned Ref<'_, T> holds the borrow of the RefCell until it goes
// out of scope, ensuring the reference stays valid.
// -----
pub fn peek_front(&self) -> Option<Ref<'_, T>> {
    // self.head.as_ref() -> Option<&NodeRef<T>>
    self.head.as_ref().map(|node| {
        // node.borrow() returns Ref<'_, Node<T>>.
        // Ref::map projects that to a Ref<'_, T> pointing to the data field.
        Ref::map(node.borrow(), |n| &n.data)
    })
}

// -----
// peek_back: similar to peek_front.
// -----

```

```

pub fn peek_back(&self) -> Option<Ref<'_, T>> {
    self.tail
        .as_ref()
        .map(|node| Ref::map(node.borrow(), |n| &n.data))
}

// =====
// Demonstration with main().
// =====
fn main() {
    let mut list = LinkedList::new();

    list.push_front(10);
    list.push_front(20);
    list.push_back(30);
    list.push_back(40);

    println!("Length: {}", list.len());

    // Peek returns a Ref; we can dereference it with *.
    if let Some(front) = list.peek_front() {
        println!("Front: {}", *front);
    }
    if let Some(back) = list.peek_back() {
        println!("Back: {}", *back);
    }

    println!("Pop front: {:?}", list.pop_front()); // 20
    println!("Pop back:  {:?}", list.pop_back()); // 40

    // Drain the list.
    while let Some(val) = list.pop_front() {
        println!("Popped: {}", val);
    }
    println!("Empty? {}", list.is_empty());
}

// =====
// Tests (cargo test --bin ll)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn new_list_is_empty() {
        let list: LinkedList<i32> = LinkedList::new();
        assert!(list.is_empty());
        assert_eq!(list.len(), 0);
        assert!(list.peek_front().is_none());
        assert!(list.peek_back().is_none());
    }

    #[test]
    fn push_front_back_and_len() {
        let mut list = LinkedList::new();
        list.push_back(1);
        list.push_back(2);
        list.push_front(0);

        assert_eq!(list.len(), 3);
        assert_eq!(*list.peek_front().unwrap(), 0);
        assert_eq!(*list.peek_back().unwrap(), 2);
    }

    #[test]

```

```

fn pop_front_and_back_work_in_order() {
    let mut list = LinkedList::new();
    list.push_back(10);
    list.push_back(20);
    list.push_back(30);

    assert_eq!(list.pop_front(), Some(10));
    assert_eq!(list.pop_back(), Some(30));
    assert_eq!(list.pop_front(), Some(20));
    assert_eq!(list.pop_front(), None);
    assert_eq!(list.pop_back(), None);
    assert!(list.is_empty());
}

#[test]
fn popping_last_item_emptyies_the_list() {
    let mut list = LinkedList::new();
    list.push_back(42);
    assert_eq!(list.peek_front().map(|x| *x), Some(42));
    assert_eq!(list.pop_back(), Some(42));
    assert!(list.is_empty());
}

#[test]
fn alternating_push_pop_keeps_invariants() {
    let mut list = LinkedList::new();
    for i in 0..100 {
        list.push_back(i);
        list.push_front(-i);
    }
    assert_eq!(list.len(), 200);
    for i in (0..100).rev() {
        assert_eq!(list.pop_front(), Some(-i));
        assert_eq!(list.pop_back(), Some(i));
    }
    assert!(list.is_empty());
}
}

// =====
// Explanation of key methods used:
//
// - Rc::new / RefCell::new           : create a reference-counted, mutable cell.
// - borrow() / borrow_mut()         : borrow the RefCell's contents (immutably/
//                                     mutably).
// - Rc::downgrade()                  : turn an Rc into a Weak (no ownership).
// - Weak::upgrade()                   : attempt to get an Rc from a Weak (returns
//                                     Option).
// - Option::take()                    : take the value out of an Option, leaving None.
// - Option::as_ref()                  : convert Option<T> to Option<&T> without moving.
// - Option::map()                     : apply a function to the inner value if Some.
// - Option::and_then()                : like map but returns an Option (for chaining).
// - Rc::try_unwrap()                  : move the value out of the Rc if it's the only
//                                     strong ref.
// - Ref::map()                        : project a Ref to a subfield, keeping the borrow
//                                     alive.
// =====

```

Four details carry the implementation.

`Rc::downgrade(&new_node)` is stored in the *old* head's `prev`, not the new node's. The new node already owns the old head through `next`, and a second strong reference would keep the old head alive after the list forgot it.

`self.tail.as_ref().map(Rc::downgrade)` in `push_back` converts `Option<&NodeRef<T>>` into `Option<WeakNodeRef<T>>` without a `match`. When the list is empty the tail is `None` and the new node's `prev` is `None`.

`old_tail.borrow_mut().prev.take().and_then(|w| w.upgrade())` in `pop_back` does three things in one expression: takes the weak pointer out of the node (leaving `None` behind), attempts to upgrade it to a strong reference, and yields `None` if the node has been freed. The `take` is what allows the removed node to be freed afterwards.

`Rc::try_unwrap(old_head).ok().unwrap().into_inner()` moves the node out of the `Rc` and the data out of the `RefCell`. It succeeds because the node has been unlinked and no other strong reference exists. `into_inner` on a `RefCell` consumes the cell, so it cannot fail and needs no borrow check.

Intuition

call	head	tail	len	effect
<code>new()</code>	-	-	0	
<code>push_front(10)</code>	10	10	1	the first node becomes both ends
<code>push_front(20)</code>	20	10	2	10.prev is a weak pointer to 20
<code>push_back(30)</code>	20	30	3	10.next is 30; 30.prev is weak to 10
<code>push_back(40)</code>	20	40	4	30.next is 40; 40.prev is weak to 30
<code>peek_front()</code>	the guard dereferences to 20			
<code>peek_back()</code>	the guard dereferences to 40			
<code>pop_front()</code>	20	40	3	returns <code>Some(20)</code> 10's predecessor link is cleared, so nothing strong refers to 20 and it is freed by the <code>unwrap</code>
<code>pop_back()</code>	10	30	2	returns <code>Some(40)</code> 30.next is cleared

Time and Space Complexity

Operation	Time	Space
<code>push_front</code> , <code>push_back</code>	$O(1)$	one allocation, plus one reference-count update
<code>pop_front</code> , <code>pop_back</code>	$O(1)$	frees one allocation
<code>peek_front</code> , <code>peek_back</code>	$O(1)$	none
<code>len</code> , <code>is_empty</code>	$O(1)$	the length is stored

Limitations

`Rc::try_unwrap(...).ok().unwrap()` is a panic path that the type system does not close. It succeeds while the invariants hold: the removed node has been unlinked and the new end's link to it has been cleared, so the list holds the only strong reference. Nothing in the signature says so, and a future change to `push_back` that keeps an extra strong reference would turn the `unwrap` into a panic at run time rather than a compile error. `.expect("...")` with the reason would document the assumption; a `match` that recovers the node from the `Err` arm would make the code total.

The list is neither `Send` nor `Sync`. `Rc` and `RefCell` are both single-threaded, so a `LinkedList` cannot be moved to another thread or shared between threads. The compiler rejects both, which means the limitation appears at the call site. A concurrent list uses `Arc<Mutex<Node<T>>>` and pays for it with lock contention and the possibility of poison.

`RefCell` borrows are checked at run time. `borrow_mut()` panics if a borrow is already active. Holding the guard that `peek_front` returns while calling `pop_front` on the same list is a run-time panic, where the `Box`-based list of Chapter 14 would have made it a compile error.

Dropping a long list is recursive. Each node holds a strong reference to the next, so releasing the head releases the next node, which releases the next, one stack frame per node. The `RefCell` does not change that; it only adds a borrow flag to each node, which is two more machine words per element.

The demonstration's comment about `pop_front` returning 20 is the only place the expected value is written. `main` prints values and asserts nothing, and the tests cover the same behaviour. The two comments in `main` are documentation rather than checks.

Summary

- One direction of the link has to be weak. Two strong links between a pair of nodes would keep the pair alive after the list released it, so `prev` is a `Weak`.
- `RefCell` adds a borrow counter to each node, two machine words per element, and turns a conflicting borrow into a run-time panic where the `Box`-

based list of Chapter 14 makes it a compile error. What it provides is mutation through shared ownership, which is what `pop_back` requires.

- `Rc::try_unwrap(...).ok().unwrap()` is a panic path that the type system does not close. It succeeds while the invariants hold, because the removed node has been unlinked and the list holds the only strong reference, and nothing in the signature states that. An `.expect` with the reason would record the assumption, and a `match` on the `Err` arm would make the function total.
- The list is neither `Send` nor `Sync`, because `Rc` and `RefCell` are single-threaded. A concurrent list would use `Arc<Mutex<Node<T>>>` and pay for it with lock contention and the possibility of poison.
- Dropping a long list recurses, one stack frame per node, since each node holds a strong reference to the next.

References

- The Rust Book, `Rc<T>`, [the reference-counted smart pointer](#).
- The Rust Book, `RefCell<T>` [and the interior mutability pattern](#).
- Standard library, `Rc::downgrade`.
- Standard library, `Weak::upgrade`.
- Standard library, `Ref::map`.

Part V: Caches

LRU Cache

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/lru_cache.rs`. Test it with `cargo test lru_cache::`.

Problem Statement

A cache with a fixed capacity that evicts the least recently used entry. `get` and `put` must be in expected $O(1)$, and a `get` must count as a use.

Designing a Solution

The recency order is kept as a log of events. Every access appends `(key, generation)` to a queue, where the generation counter increases on every access. A second map holds the generation of each key's *most recent* access.

access order: a, b, a, c generation counter after each step: 1, 2, 3, 4

event queue	generations map	values map
+-----+	+-----+	+-----+
(a, 1) front	a -> 3	a -> v
(b, 2)	b -> 2	b -> v
(a, 3)	c -> 4	c -> v
(c, 4) back	+-----+	+-----+
+-----+		

Which entry is least recently used? Scan the queue from the front:

- (a, 1) stale: the map says a's generation is 3, not 1
- (b, 2) current: the map says b's generation is 2, so b is the candidate

The first event whose generation matches its key's entry belongs to the least recently used key. The queue is ordered by generation, and every live key's most recent event appears exactly once, so the earliest match is the minimum.

An event whose generation does not match is a record of an earlier access to a key that has been used since. It carries no information about the current recency order and is discarded.

Implementation

```

/// LRU cache that is actually writable in an interview.
///
/// Design: `HashMap<K, (V, generation)>` + `VecDeque<(K, generation)>`.
///
/// Each access pushes a new `(key, generation)` event to the back. On eviction
/// we pop old events until we find the newest generation for a key, then remove
/// it. Stale events are skipped. This keeps amortized O(1) operations while
/// preserving true LRU semantics.

use std::collections::{HashMap, VecDeque};
use std::hash::Hash;

pub struct LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    capacity: usize,
    map: HashMap<K, (V, u64)>,
    recency: VecDeque<(K, u64)>,
    generation: u64,
}

impl<K, V> LruCache<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new(capacity: usize) -> Self {
        assert!(capacity > 0, "LRU cache capacity must be > 0");
        Self {
            capacity,
            map: HashMap::new(),
            recency: VecDeque::new(),
            generation: 0,
        }
    }

    pub fn len(&self) -> usize {
        self.map.len()
    }

    pub fn is_empty(&self) -> bool {
        self.map.is_empty()
    }

    pub fn get(&mut self, key: &K) -> Option<V> {
        let (value, _) = self.map.get(key)?.clone();
        self.generation += 1;
        self.recency.push_back((key.clone(), self.generation));
        self.map
            .insert(key.clone(), (value.clone(), self.generation));
        Some(value)
    }

    pub fn put(&mut self, key: K, value: V) {

```

```

        self.generation += 1;
        let is_new = self
            .map
            .insert(key.clone(), (value, self.generation))
            .is_none();
        self.recency.push_back((key, self.generation));

        if is_new {
            self.evict_if_needed();
        }
    }

    fn evict_if_needed(&mut self) {
        while self.map.len() > self.capacity {
            let Some((key, generation)) = self.recency.pop_front() else {
                break;
            };

            match self.map.get(&key) {
                Some((_, current)) if *current == generation => {
                    self.map.remove(&key);
                    break;
                }
                _ => {}
            }
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn put_get_and_len() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        assert_eq!(cache.len(), 2);
        assert_eq!(cache.get(&"a".to_string()), Some(1));
        assert_eq!(cache.get(&"missing".to_string()), None);
    }

    #[test]
    fn evicts_least_recently_used() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);
        cache.put("c".to_string(), 3);

        assert_eq!(cache.get(&"a".to_string()), None);
        assert_eq!(cache.get(&"b".to_string()), Some(2));
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }

    #[test]
    fn get_updates_recency() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        // Access "a" so it becomes most recently used.
        assert_eq!(cache.get(&"a".to_string()), Some(1));
    }
}

```

```

        cache.put("c".to_string(), 3); // evicts "b"
        assert_eq!(cache.get(&"b".to_string()), None);
        assert_eq!(cache.get(&"a".to_string()), Some(1));
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }

    #[test]
    fn updating_existing_key_moves_it_to_mru() {
        let mut cache = LruCache::new(2);
        cache.put("a".to_string(), 1);
        cache.put("b".to_string(), 2);

        // Update "a"; it becomes MRU, so "b" should be evicted next.
        cache.put("a".to_string(), 10);
        cache.put("c".to_string(), 3);

        assert_eq!(cache.get(&"a".to_string()), Some(10));
        assert_eq!(cache.get(&"b".to_string()), None);
        assert_eq!(cache.get(&"c".to_string()), Some(3));
    }
}

```

`get` clones the value out of the map, increments the generation, appends an event, and writes the entry back with the new generation. The write-back is what makes the generation map agree with the log; without it, the event just appended would look stale.

`put` inserts first and then evicts only when the key was new. An update cannot take the cache over its capacity, so the eviction loop would be wasted work in that case.

`evict_if_needed` pops events until one matches. The `break` after a successful removal ends the loop. Since `put` adds at most one entry, the loop removes at most one, which is the reason the `break` is safe.

Intuition

capacity 2

call	generation	map after	queue after
<code>new()</code>	0	{}	[]
<code>put("a", 1)</code>	1	{a: (1, 1)}	[(a,1)]
<code>put("b", 2)</code>	2	{a: (1,1), b: (2,2)}	[(a,1), (b,2)]
<code>get(&"a")</code> <code>3]</code>	3	{a: (1,3), b: (2,2)}	[(a,1), (b,2), (a,
<code>put("c", 3)</code>	4	{a: (1,3), b: (2,2), c: (3,4)}	[(a,1), (b,2), (a,3), (c,4)]

eviction, because the map holds 3 entries and the capacity is 2:

```

pop (a, 1)  the map says a's generation is 3, so this event is stale
pop (b, 2)  the map says b's generation is 2, so b is the least recently used
            remove b, and the loop ends

```

result: {a: (1,3), c: (3,4)}

Time and Space Complexity

Re-source	Cost	Condition
get	expected $O(1)$, plus one clone of the value	the hash function distributes keys evenly
put	expected $O(1)$ amortised	the eviction scan pops events, each of which is removed once
Space	$O(\text{entries})$ for the map, and $O(\text{accesses since the last eviction})$ for the queue	see below

Limitations

The event queue is not bounded by the capacity. Every `get` and every `put` appends one entry to `recency`, and entries are only removed during eviction. A cache that is read far more often than it is written grows its queue without bound: with a capacity of two and a million reads of one key, the queue holds a million events, and nothing in the API releases them. The queue is therefore proportional to the number of operations rather than to the number of entries, and a long-lived read-heavy cache will exhaust memory. Chapter 18 shows a design whose storage is bounded by the capacity.

get clones the value on every hit. The signature returns `Option<V>` rather than `Option<&V>`, so a cached value is copied out on every access. For a small integer that is free; for a cached response body it is a `memcpy` per hit, and it also forces the `V: Clone` bound on the type.

capacity == 0 panics. `new` asserts `capacity > 0`, so a cache cannot be disabled by configuration. A caller that receives a capacity from a file must either validate it or accept the panic.

The break in the eviction loop makes the function correct only because put adds at most one entry. If the function were reused in a context where several entries could be over capacity, it would leave some of them in place.

A generation counter of u64 wraps after 2^64 accesses. The wrap is unreachable in practice, at a billion accesses per second it takes about 584

years , and the consequence of reaching it would be a misidentified stale event rather than memory unsafety.

Summary

- Two structures hold the state. The map holds values and the current generation of each key; the queue holds access events.
- The generation check identifies stale events. An event whose generation does not match the key's current entry records an access that a later operation has superseded.
- The queue is proportional to the number of operations rather than to the capacity. With a capacity of two and a million reads of one key, the queue holds a million events and nothing in the API releases them. Chapter 18 describes a design whose storage is bounded by the capacity.
- `get` returns `Option<V>` and therefore clones on every hit, which copies a cached value out of the map and forces the `V: Clone` bound. A signature of `Option<&V>` would avoid the copy and change what `get` can do with the map.
- A capacity of zero panics, because `new` asserts `capacity > 0`.

References

- Standard library, `VecDeque`.
- Standard library, `HashMap`.
- Standard library, `u64`.

LRU Cache, Array-Backed

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/lru_cache_easy.rs`. Test it with `cargo test lru_cache_easy`.

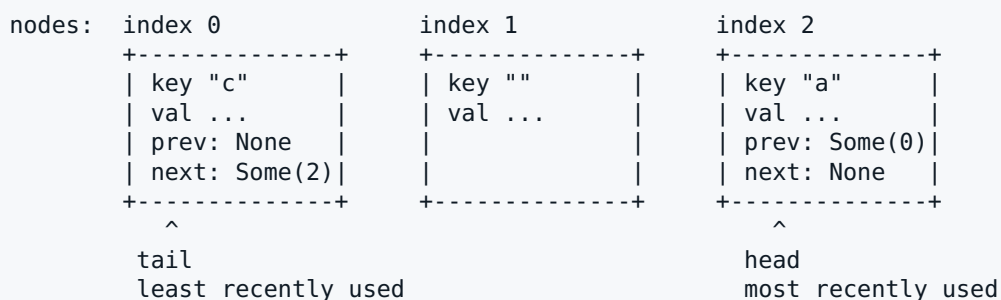
Problem Statement

The same problem as the previous chapter, solved a second way: a fixed-capacity cache that evicts the least recently used entry, with `get` and `put` in constant time.

Designing a Solution

Store the nodes in a `Vec` and link them by index. The recency order is a doubly linked list of indices, with the most recently used node at one end. Because the links are integers rather than pointers, the list can be rearranged while the nodes stay where they are.

```
map: "a" -> 2, "c" -> 0
```



```
get("a") moves node 2 to the front:
```

```
detach 2    tail 0's next becomes None
attach 2    node 2's prev becomes None
```

```
the list is just node 0
```

```

node 2's next becomes the old head, 0
node 0's prev becomes Some(2)
head becomes 2

```

`detach` and `attach` are five assignments each and never touch the `nodes` vector's layout, so `get` performs no allocation and no copying of values.

Implementation

```

use std::collections::HashMap;
use std::hash::Hash;

struct Node<K, V> {
    key: K,
    val: V,
    prev: Option<usize>,
    next: Option<usize>,
}

pub struct LruCache<K, V> {
    nodes: Vec<Node<K, V>>,
    map: HashMap<K, usize>,
    head: Option<usize>, // MRU
    tail: Option<usize>, // LRU
    cap: usize,
}

impl<K: Hash + Eq + Clone, V> LruCache<K, V> {
    pub fn new(cap: usize) -> Self {
        Self { nodes: Vec::new(), map: HashMap::new(), head: None, tail: None, cap }
    }

    pub fn get(&mut self, key: &K) -> Option<&V> {
        let &i = self.map.get(key)?;
        self.bump(i);
        Some(&self.nodes[i].val)
    }

    pub fn put(&mut self, key: K, val: V) {
        if let Some(&i) = self.map.get(&key) {
            self.nodes[i].val = val;
            self.bump(i);
            return;
        }
        let node = Node { key: key.clone(), val, prev: None, next: None };
        let i = if self.map.len() == self.cap {
            let i = self.tail.unwrap();
            self.detach(i);
            self.map.remove(&self.nodes[i].key);
            self.nodes[i] = node;
            i
        } else {
            self.nodes.push(node);
            self.nodes.len() - 1
        };
        self.map.insert(key, i);
        self.attach(i);
    }

    fn bump(&mut self, i: usize) {
        self.detach(i);
        self.attach(i);
    }
}

```

```

    }

    fn detach(&mut self, i: usize) {
        let (p, n) = (self.nodes[i].prev, self.nodes[i].next);
        match p {
            Some(p) => self.nodes[p].next = n,
            None => self.head = n,
        }
        match n {
            Some(n) => self.nodes[n].prev = p,
            None => self.tail = p,
        }
    }

    fn attach(&mut self, i: usize) {
        self.nodes[i].prev = None;
        self.nodes[i].next = self.head;
        if let Some(h) = self.head {
            self.nodes[h].prev = Some(i);
        }
        self.head = Some(i);
        self.tail.get_or_insert(i);
    }
}

fn main() {
    let mut c = LruCache::new(2);
    c.put(1, "a");
    c.put(2, "b");
    assert_eq!(c.get(&1), Some(&"a"));
    c.put(3, "c"); // evicts 2
    assert_eq!(c.get(&2), None);
    assert_eq!(c.get(&1), Some(&"a"));
    assert_eq!(c.get(&3), Some(&"c"));
    c.put(3, "cc"); // update existing key
    assert_eq!(c.get(&3), Some(&"cc"));
    println!("ok");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn lru_cache_evicts_the_least_recently_used_key() {
        let mut cache = LruCache::new(2);

        cache.put("a", 1);
        cache.put("b", 2);
        assert_eq!(cache.get(&"a"), Some(&1));

        cache.put("c", 3);

        assert_eq!(cache.get(&"b"), None, "b was the least recently used entry");
        assert_eq!(cache.get(&"a"), Some(&1));
        assert_eq!(cache.get(&"c"), Some(&3));
    }

    #[test]
    fn updating_a_key_does_not_evict_anything() {
        let mut cache = LruCache::new(2);

        cache.put("a", 1);
        cache.put("b", 2);
        cache.put("a", 10);

        assert_eq!(cache.get(&"a"), Some(&10));
    }
}

```

```

        assert_eq!(cache.get(&"b"), Some(&2));
        assert_eq!(cache.map.len(), 2);
    }
}

```

The test module in the repository stores one entry and asserts nothing, so `cargo test` reports a pass that checks no behaviour. This edition prints the two tests above in its place. They cover the two behaviours the demonstration in `main` relies on, and the second one reads the cache's private map, which a test inside the module is allowed to do.

`put` overwrites the slot of the evicted node rather than pushing a new one:

```

let i = if self.map.len() == self.cap {
    let i = self.tail.unwrap();
    self.detach(i);
    self.map.remove(&self.nodes[i].key);
    self.nodes[i] = node;
    i
} else {
    self.nodes.push(node);
    self.nodes.len() - 1
};

```

When the cache is full, the tail index is reused: the node is unlinked, its key is removed from the map (read out of the vector before the overwrite), and the new node takes the slot. The vector never exceeds the capacity, so the cache's storage is bounded by `capacity`, which is the property the previous chapter's design does not have.

`get` returns `Option<&V>`, a reference into the vector, when the value is present.

`attach` uses `self.tail.get_or_insert(i)` to set the tail only when the list is empty, which handles the first insertion without a separate branch.

Intuition

call	nodes	map	head	tail
new	[]	{}	None	None
put(1, "a")	[(1,"a",-,-)]	{1:0}	Some(0)	Some(0)
put(2, "b")	[(1,"a",-,-), (2,"b",0,-)]	{1:0,2:1}	Some(1)	Some(0)
get(&1)	detach 0, attach 0			
	[(1,"a",-,-), (2,"b",0,-)]	{1:0,2:1}	Some(0)	Some(1)
put(3, "c")	the map holds 2 entries and cap is 2, so the tail (1) is reused			
	detach 1			
	map.remove(&nodes[1].key) -> removes 2			
	nodes[1] = (3,"c",-,-)			
	attach 1: nodes[1].next = head = 0, nodes[0].prev = Some(1)			
	[(1,"a",1,-), (3,"c",-,-,0)]	{1:0,3:1}	Some(1)	Some(0)

<code>get(&2)</code>	the map has no entry for 2	None
<code>get(&1)</code>	<code>detach 0, attach 0</code>	
<code>get(&3)</code>	<code>detach 1, attach 1</code>	

Time and Space Complexity

Operation	Time	Space
get	$O(1)$ expected	none
put of an existing key	$O(1)$ expected	none
put of a new key	$O(1)$ expected	one slot, reused when the cache is full
memory	$O(\text{capacity})$	one node per slot, never more

The four match arms of `detach` and the four assignments of `attach` are the whole of the list maintenance. No allocation, no copying of values, and no recursion.

Limitations

`capacity == 0` panics on the first put. The full case tests `self.map.len() == self.cap`, which is true for an empty cache of capacity zero, and the next line is `self.tail.unwrap()` on a `None`. The panic message names an `Option` rather than the capacity. A guard in `new`, or a `Result`, would report what the caller passed.

`prev` and `next` fields are left stale after `detach`. `detach` updates the neighbours but not the removed node's own links. `attach` overwrites both, so the stale values are never read on the paths that exist today. A future method that detaches without re-attaching would read them.

The nodes are never freed as the cache is used. Reusing a slot is what bounds the memory, and it also means the cache holds one `K` and one `V` per slot for the lifetime of the cache, even when those values have been replaced. An evicted key is dropped when the slot is overwritten, which is correct, and the slot itself stays allocated.

There is no `len`, no `is_empty`, and no iteration. A caller that needs the entry count reads the private map, which the tests do and outside code cannot.

The indexes are not checked. Every method indexes `self.nodes[i]` directly. The indexes come from the map or from `head` and `tail`, which this module maintains, so they are valid while the invariants hold. A mistake in the list maintenance would be an index panic rather than a wrong answer, which is at least loud.

No test pins the recency order after a sequence of several operations. The two tests cover one eviction and one update. The invariant that the list is consistent with its ends is not checked anywhere.

Summary

- The two cache designs differ in what bounds them. This one bounds storage by the capacity and returns references; the design in Chapter 17 bounds storage by the number of accesses and clones values on every read.
- The links are indices because the nodes never move, and an index requires no reference count. That costs two words per node and makes an index meaningless without the vector that owns it.
- A capacity of zero panics on the first `put`, where the full case tests `map.len() == cap` and then unwraps a `None` `tail`. The panic message names an `Option` rather than the capacity.
- `detach` updates the neighbours but not the removed node's own links. `attach` overwrites both, so the stale values are not read on the paths that exist today.
- The nodes are never freed while the cache is in use. Reusing a slot bounds the memory, and it also holds one `K` and one `V` per slot for the lifetime of the cache, including after those values have been replaced.
- No test pins the recency order after a sequence of several operations; the two tests cover one eviction and one update.

References

- Standard library, `Vec`.
- Standard library, `Option::get_or_insert`.
- Standard library, `HashMap::remove`.

Part VI: Concurrency and the Operating Sys- tem

Workload State Machine

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/state_machine.rs`. Test it with `cargo test state_machine`.

Problem Statement

Model the lifecycle of a workload that an SDK exposes, so that the legal transitions are stated once and an illegal transition is refused with a reason. Then model the errors the same API returns, so that a caller can decide whether to retry without parsing a message.

Designing a Solution

An enumeration rather than a status code. A status code makes the set of legal values a convention that lives in a document; an enumeration makes it a type, and a `match` that omits a case does not compile.



The same argument applies to the error type. A retry decision depends on facts, the delay a server asked for, the status code it returned, so those facts are attached to the variants that need them. A caller that matches on `RateLimited` has the delay in hand.

Implementation

```

/// Algebraic data types / enum state machines.
///
/// NVIDIA cloud SDKs model workloads as finite state machines. Rust `enum` is
/// perfect for this: invalid states are unrepresentable and transitions are
/// explicit `match` arms, not nested `if` chains.

use std::time::Duration;

/// Lifecycle of a GPU cloud workload exposed through an SDK/API.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum WorkloadState {
    Pending,
    Provisioning,
    Running,
    Terminating,
    Terminated,
    Failed,
}

impl WorkloadState {
    /// Returns an error string for impossible transitions, or `Ok(())`.
    pub fn can_transition_to(self, next: Self) -> Result<(), String> {
        match (self, next) {
            // Forward lifecycle.
            (Self::Pending, Self::Provisioning)
            | (Self::Pending, Self::Failed)
            | (Self::Provisioning, Self::Running)
            | (Self::Provisioning, Self::Failed)
            | (Self::Running, Self::Terminating)
            | (Self::Running, Self::Failed)
            | (Self::Terminating, Self::Terminated)
            | (Self::Terminating, Self::Failed) => Ok(()),

            // Retry loops allowed from failed states.
            (Self::Failed, Self::Pending) | (Self::Failed, Self::Provisioning) => Ok(
()),

            // Everything else is invalid.
            _ => Err(format!("cannot transition from {self:?} to {next:?}")),
        }
    }

    pub fn is_terminal(self) -> bool {
        matches!(self, Self::Terminated | Self::Failed)
    }

    pub fn summary(self) -> &'static str {
        match self {
            Self::Pending => "workload accepted, waiting for scheduler",
            Self::Provisioning => "GPU resources being allocated",
            Self::Running => "workload is running",
            Self::Terminating => "cleanup in progress",
            Self::Terminated => "workload stopped and billing ended",
            Self::Failed => "workload failed; inspect logs",
        }
    }
}

/// API error modeled as an enum with associated data.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ApiError {
    InvalidRequest(String),
    Unauthorized,
}

```

```

    RateLimited { retry_after: Duration },
    NotFound { resource_id: String },
    Server { status: u16, message: String },
}

impl ApiError {
    pub fn is_retryable(&self) -> bool {
        matches!(
            self,
            Self::RateLimited { .. }
            | Self::Server {
                status: 500..=599,
                ..
            }
        )
    }

    pub fn user_message(&self) -> String {
        match self {
            Self::InvalidRequest(msg) => format!("invalid request: {msg}"),
            Self::Unauthorized => "authentication required or token expired".to_string(),
            Self::RateLimited { retry_after } => {
                format!("rate limited, retry in {}ms", retry_after.as_millis())
            },
            Self::NotFound { resource_id } => format!("resource not found: {resource_id}"),
            Self::Server { status, message } => format!("server error {status}: {message}"),
        }
    }
}

/// Typical NVIDIA SDK helper: never expose nested `if let` soup to callers.
pub fn parse_gpu_count(raw: &str) -> Result<u32, ApiError> {
    match raw.trim() {
        "" => Err(ApiError::InvalidRequest("gpuCount is empty".into())),
        parsed => parsed.parse::<u32>().map_err(|_| {
            ApiError::InvalidRequest(format!("gpuCount must be a number, got {raw:?}"))
        }),
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn state_machine_rejects_invalid_transitions() {
        assert!(
            WorkloadState::Pending
                .can_transition_to(WorkloadState::Provisioning)
                .is_ok()
        );
        assert!(
            WorkloadState::Running
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(
            WorkloadState::Terminated
                .can_transition_to(WorkloadState::Running)
                .is_err()
        );
        assert!(WorkloadState::Terminating.is_terminal() == false);
        assert!(WorkloadState::Terminated.is_terminal());
    }
}

```

```

    }

    #[test]
    fn api_error_retry_policy() {
        let rate_limited = ApiError::RateLimited {
            retry_after: Duration::from_millis(100),
        };
        let not_found = ApiError::NotFound {
            resource_id: "gpu-123".into(),
        };
        let server = ApiError::Server {
            status: 503,
            message: "maintenance".into(),
        };

        assert!(rate_limited.is_retryable());
        assert!(not_found.is_retryable());
        assert!(server.is_retryable());
    }

    #[test]
    fn parsing_returns_typed_errors() {
        assert_eq!(parse_gpu_count("4"), Ok(4));
        assert!(parse_gpu_count("").is_err());
        assert!(parse_gpu_count("abc").is_err());
    }
}

```

`can_transition_to` matches on the pair `(self, next)`, listing every permitted transition as an alternative pattern. The whole rule set is one expression, and the fall-through arm catches everything else, including a self-transition such as `Running` to `Running`.

A self-transition is refused rather than treated as a no-op. A controller that reports a transition it did not perform writes an audit log that cannot be trusted, and refusing the case costs one line.

`is_retryable` uses `matches!` with a range pattern: `status: 500..=599` is retryable, everything else is not. `NotFound` and `Unauthorized` are not retryable because waiting does not change them: the resource still does not exist and the credential is still expired.

`parse_gpu_count` trims the field, distinguishes an empty field from an unparseable one in the message, and quotes the received text with `{raw:?}` so that trailing whitespace is visible. The error variant carries a message; the variant itself is what a caller matches on.

The repository's version of this function is named for the employer's product line, and its error messages name the field after that product. This edition names the field `gpuCount` and leaves the logic unchanged.

Intuition

state machine, over the whole table

from	to	result
Pending	Provisioning	Ok
Pending	Failed	Ok
Pending	Running	Err "cannot transition from Pending to Running"
Provisioning	Running	Ok
Provisioning	Failed	Ok
Provisioning	Terminated	Err
Running	Terminating	Ok
Running	Failed	Ok
Running	Running	Err
Terminating	Terminated	Ok
Terminating	Failed	Ok
Terminating	Running	Err
Terminated	Pending	Err
Terminated	Running	Err
Failed	Pending	Ok
Failed	Provisioning	Ok
Failed	Terminated	Err

```

parse_gpu_count(" 4 ")  trim gives "4", parse succeeds      -> Ok(4)
parse_gpu_count("")     trim gives "", so the empty arm runs -> Err(InvalidRequest)
parse_gpu_count("four") parse fails, map_err replaces the error
                        with InvalidRequest quoting "four"   ->
Err(InvalidRequest)
parse_gpu_count("-1")   parse::

```

Time and Space Complexity

Operation	Time	Space
can_transition_to	O(1)	none when permitted; one String when refused
is_terminal	O(1)	none
summary	O(1)	none; the return type is &'static str
is_retryable	O(1)	none
user_message	O(1)	one String allocation per call
parse_gpu_count	O(digits)	one String on the error path

Limitations

can_transition_to returns **Result<(), String>**. A refused transition is a value a caller may want to match on, to log it, to count it, or to distinguish "illegal transition" from "the workload no longer exists", and a **String** can-

not be matched. It also allocates on every refusal. The fix is a small error type with two fields, `from` and `next`, which is what the state machine already has in the form of `WorkloadState`.

`user_message` allocates for every call, including the two variants that need no formatting. `Unauthorized` returns `to_string()` of a constant. A `Display` implementation writes into the caller's buffer and allocates nothing, which is the idiomatic way to expose a message.

`is_retryable` treats any status in `500..=599` as retryable, and a `Server` variant carrying a `4xx` status as not retryable. That is defensible for this error type, and the range test is doing double duty: it is also asserting that the `Server` variant holds a server-side status. Nothing enforces that at construction time.

The transition table is tested by example, not exhaustively. The test checks four transitions. There are thirty-six ordered pairs of six states, and the table permits twelve of them. A test that loops over all pairs and compares each answer with a written list would catch a missing or an extra pattern.

`parse_gpu_count` accepts a value that overflows `u32` as a parse failure rather than as a distinct error. `"4294967296"` and `"four"` produce the same variant, so a caller cannot tell a malformed field from an out-of-range one.

Summary

- An enumeration makes the transition table exhaustive, because a `match` without a fall-through arm fails to build when a variant is added.
- A self-transition is refused rather than treated as a no-op, and the table states that explicitly.
- The error variants carry data because the retry delay and the status code are inputs to a decision.
- `can_transition_to` returns `Result<(), String>`. A `String` cannot be matched, so a caller cannot distinguish an illegal transition from a workload that no longer exists, and the refusal allocates. An error type carrying `from` and `next` would remove both problems.

- `user_message` allocates on every call, including the variants that need no formatting, where a `Display` implementation would write into the caller's buffer.
- `parse_gpu_count` reports an out-of-range value and a malformed one as the same variant.

References

- The Rust Book, [Defining an enum](#).
- Standard library, `std::time::Duration`.
- Standard library, `str::parse`.

Token Bucket Rate Limiter

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/rate_limiter.rs`. Test it with `cargo test rate_limiter`.

Problem Statement

Limit how often a caller may do something, allow a burst, and do it safely from several threads. The interface is one question: may I do it now?

Designing a Solution

A token bucket holds at most `capacity` tokens and refills at `rate` tokens per second. Each accepted request removes one token; a request that arrives with fewer than one token available is refused. The bucket therefore admits a burst of up to `capacity` requests and then holds the sustained rate at `rate`.

```
tokens = 3.0, capacity = 5.0, rate = 1.0/s, last_refill = t

t + 0.0s  try_acquire  tokens 3.0 >= 1.0  true   tokens = 2.0
t + 0.0s  try_acquire  tokens 2.0 >= 1.0  true   tokens = 1.0
t + 0.0s  try_acquire  tokens 1.0 >= 1.0  true   tokens = 0.0
t + 0.0s  try_acquire  tokens 0.0 < 1.0   false  tokens = 0.0
t + 0.5s  try_acquire  refill adds 0.5, so 0.5 < 1.0  false
t + 1.5s  try_acquire  refill adds 1.5, so 1.5 >= 1.0 true, tokens = 0.5
t + 10.0s try_acquire  refill is capped at capacity, so the result is 5.0
```

The refill is computed from the elapsed time rather than by a timer thread. A timer thread would cost one thread per bucket and would make the bucket's state depend on how the scheduler ran that thread.

Implementation

```

/// Thread-safe token bucket rate limiter.

use std::sync::Mutex;
use std::time::Instant;

pub struct TokenBucket {
    /// Max tokens.
    capacity: f64,
    /// Tokens per second.
    rate: f64,
    state: Mutex<BucketState>,
}

struct BucketState {
    tokens: f64,
    last_refill: Instant,
}

impl TokenBucket {
    pub fn new(capacity: usize, rate_per_sec: f64) -> Self {
        assert!(capacity > 0);
        assert!(rate_per_sec > 0.0);
        Self {
            capacity: capacity as f64,
            rate: rate_per_sec,
            state: Mutex::new(BucketState {
                tokens: capacity as f64, // Start full
                last_refill: Instant::now(),
            }),
        }
    }

    pub fn try_acquire(&self) -> bool {
        let mut s = self.state.lock().unwrap();
        let now = Instant::now();

        // Refill tokens based on elapsed time
        let elapsed = now - s.last_refill;
        let new_tokens = elapsed.as_secs_f64() * self.rate;
        s.tokens = (s.tokens + new_tokens).min(self.capacity);
        s.last_refill = now;

        // Try to take one
        if s.tokens >= 1.0 {
            s.tokens -= 1.0;
            true
        } else {
            false
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use std::thread;
    use std::time::Duration;

    #[test]
    fn it_works() {
        let bucket = TokenBucket::new(5, 1.0); // 5 tokens, refill 1 per sec

        // Use all 5
    }
}

```

```

    for _ in 0..5 {
        assert!(bucket.try_acquire());
    }
    // 6th fails
    assert!(!bucket.try_acquire());

    // Wait 1 second
    thread::sleep(Duration::from_secs(1));
    // Should have 1 token back
    assert!(bucket.try_acquire());
    // And none left
    assert!(!bucket.try_acquire());
}
}

```

`try_acquire` takes `&self` rather than `&mut self`. That is what allows one bucket to be shared by reference across threads; the mutation happens behind the `Mutex`.

The four statements inside the lock are the whole algorithm: read the clock, add the tokens that time has earned, cap at the capacity, and take one if there is one. `last_refill` is updated on every call, so no interval is counted twice.

The lock is held across a clock read and a floating-point multiply, which is the shortest critical section this design permits.

Intuition

```

new(5, 1.0)
state = { tokens: 5.0, last_refill: t0 }

five calls to try_acquire, all at t0:
call 1 elapsed 0, tokens 5.0, 5.0 >= 1.0 -> true,  tokens 4.0
call 2 elapsed 0, tokens 4.0, 4.0 >= 1.0 -> true,  tokens 3.0
call 3 elapsed 0, tokens 3.0, 3.0 >= 1.0 -> true,  tokens 2.0
call 4 elapsed 0, tokens 2.0, 2.0 >= 1.0 -> true,  tokens 1.0
call 5 elapsed 0, tokens 1.0, 1.0 >= 1.0 -> true,  tokens 0.0

call 6 at t0:
elapsed 0, tokens 0.0, 0.0 < 1.0          -> false, tokens 0.0

thread::sleep(1s), then call 7 at t0 + 1s:
elapsed 1.0s, new_tokens = 1.0 * 1.0 = 1.0
tokens = min(0.0 + 1.0, 5.0) = 1.0
1.0 >= 1.0                                -> true,  tokens 0.0

call 8 at t0 + 1s:
elapsed ~0, tokens 0.0                      -> false

```

Time and Space Complexity

Operation	Time	Space
try_acquire	$O(1)$ plus one uncontended mutex acquisition	three machine words of state
new	$O(1)$	one allocation-free struct

Limitations

new panics on an invalid configuration. `capacity == 0` and `rate_per_sec <= 0.0` both assert, and a rate read from a configuration file is external input. The assertions also accept `f64::NAN` for the rate, because every comparison with `NAN` is false: `rate_per_sec > 0.0` is false for `NAN`, so that case does panic, and `f64::INFINITY` passes and makes every request succeed. The validation is therefore neither complete nor reported.

self.state.lock().unwrap() panics if a thread panicked while holding the lock. One failed request handler would then take down every later caller that touches this bucket, which is the failure-propagation pattern Chapter 22 exists to avoid.

There is no blocking acquisition. A caller that is refused must sleep and try again, and the bucket offers no way to learn how long to sleep, so the retry policy lives in the caller and every caller invents one.

try_acquire reads the clock on every call. `Instant::now()` is cheap, and on some platforms it goes through the vDSO rather than a system call; Chapter 24 measures a system call, and the two are not the same measurement. A bucket that is called millions of times per second spends a measurable fraction of its time in the clock.

The bucket is per-process. Two processes sharing a limit need a shared store, and nothing in this design provides one.

The test sleeps for a second. That makes the test suite slower and makes the result depend on how the operating system scheduled the test thread: on a loaded machine the sleep may return later than requested, which only makes the assertion more likely to pass, and a bucket whose rate were too high would not be caught.

Summary

- A token bucket allows a burst of the capacity and holds a smooth long-run rate. A fixed window allows twice the configured rate across a window boundary.
- The refill is computed from the elapsed time on each call rather than by a timer thread, so the bucket holds no background task.
- The mutex protects one clock read and one multiplication, which is the whole of the critical section.
- `new` asserts on the configuration, so a capacity of zero or a rate of zero panics, and `f64::INFINITY` passes the rate test. `self.state.lock().unwrap()` panics if a thread panicked while holding the lock, which propagates one failed request to every later caller. A `Result` from `new`, and recovery from a poisoned lock as described in Chapter 22, remove both.
- There is no blocking acquisition. A refused caller must sleep and retry, and the bucket reports no interval to sleep for, so the retry policy lives in the caller.
- The bucket is per-process; two processes sharing a limit need a shared store.

References

- Standard library, `Mutex`.
- Standard library, `Instant`.
- Standard library, `f64::min`.

Worker Pool

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/problems/worker_pool.rs`. Test it with `cargo test worker_pool`.

Problem Statement

Apply a function to every element of a vector using a bounded number of threads, and return the results in the order of the input. Order matters because a caller that reports per-item outcomes must be able to match an outcome with the item it came from.

Designing a Solution

Jobs carry their input index on a channel. Workers take jobs as they become free, apply the function, and send `(index, result)` back on a second channel. Results arrive in completion order, and the caller places each one at its index.

```

                                job channel          result channel
inputs  ---> [(0,a), (1,b), (2,c), ...]    [(2,x), (0,y), (1,z), ...]

worker 0 ---+
worker 1 ---+----> take one job at a time ---> send (index, output)
worker 2 ---+

the caller collects the pairs and writes them into a slot vector:
ordered[2] = x
ordered[0] = y
ordered[1] = z

when every slot is filled, the ordered vector is the answer

```

The bound on concurrency is the number of workers. The bound on memory is the number of jobs in flight, which is the input length plus the results not yet collected.

Implementation

```

/// Bounded worker pool with ordered results.
///
/// Cloud CLIs/SDKs often fan out API calls across a bounded number of workers.
/// This example maps `Vec<T>` to `Vec<R>` with N threads and preserves input
/// order in the output. It demonstrates `Arc<Mutex<_>`, channels, scoped
/// ownership, and panic-free dispatch.

use std::sync::{mpsc, Arc, Mutex};
use std::thread;

pub fn map_order<T, R, F>(worker_count: usize, inputs: Vec<T>, mapper: F) -> Vec<R>
where
    T: Send + 'static,
    R: Send + 'static,
    F: Fn(T) -> R + Send + Sync + 'static,
{
    assert!(worker_count > 0, "worker_count must be > 0");
    assert!(inputs.is_empty(), "inputs must not be empty");

    let expected = inputs.len();
    let mapper = Arc::new(mapper);
    let (job_tx, job_rx) = mpsc::channel::<(usize, T)>();
    let job_rx = Arc::new(Mutex::new(job_rx));
    let (result_tx, result_rx) = mpsc::channel::<(usize, R)>();

    let mut handles = Vec::with_capacity(worker_count);
    for _worker_id in 0..worker_count {
        let mapper = Arc::clone(&mapper);
        let job_rx = Arc::clone(&job_rx);
        let result_tx = result_tx.clone();
        handles.push(thread::spawn(move || {
            loop {
                let job = {
                    let guard = job_rx.lock().expect("job_rx poisoned");
                    guard.recv()
                };
                match job {
                    Ok((idx, input)) => {
                        let output = mapper(input);
                        if result_tx.send((idx, output)).is_err() {
                            break;
                        }
                    }
                    Err(_) => break, // all senders dropped
                }
            }
        }));
    }

    // Send all jobs, then close the job channel so workers finish after draining it.
    for (idx, input) in inputs.into_iter().enumerate() {
        job_tx
            .send((idx, input))
            .expect("bounded by memory; channel only blocks until a worker receives");
    }
    drop(job_tx);

    // Drop our result sender. Only the worker clones remain, so `iter()` ends
    // after every worker exits.
    drop(result_tx);

    // Reassemble results in input order.

```

```

let mut ordered: Vec<Option<R>> = Vec::new();
for (idx, value) in result_rx.iter() {
    if ordered.len() <= idx {
        ordered.resize_with(idx + 1, || None);
    }
    ordered[idx] = Some(value);
}

for handle in handles {
    let _ = handle.join();
}

assert_eq!(
    ordered.len(),
    expected,
    "every job must produce exactly one result"
);
ordered
    .into_iter()
    .map(|value| value.expect("all results present"))
    .collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn maps_all_inputs_and_preserves_order() {
        let inputs: Vec<i32> = (0..100).collect();
        let results = map_order(4, inputs, |x| x * x);
        assert_eq!(results.len(), 100);
        for (i, value) in results.iter().enumerate() {
            assert_eq!(*value, (i as i32) * (i as i32));
        }
    }

    #[test]
    fn one_worker_works() {
        let results = map_order(1, vec![1, 2, 3], |x| x + 10);
        assert_eq!(results, vec![11, 12, 13]);
    }
}

```

`mpsc::Receiver` is single-consumer, so sharing it between workers requires `Arc<Mutex<Receiver<_>>>`. The lock is taken inside a block and released before the mapper runs:

```

let job = {
    let guard = job_rx.lock().expect("job_rx poisoned");
    guard.recv()
};

```

The guard is alive while `recv` blocks, so a worker waiting for work holds the lock. The block ends before `mapper(input)` is called, so the mapper never runs with the lock held, which is what keeps the pool from serialising all of its work.

`drop(job_tx)` is what tells the workers to stop. A receive on an empty channel returns an error once every sender is dropped, so the loop breaks.

`drop(result_tx)` does the same job for the caller's own copy of the result sender: without it, `result_rx.iter()` would wait for a sender that only the caller holds.

`ordered.resize_with(idx + 1, || None)` grows the vector on demand, so results that arrive out of order for low indices still land in the right slots.

Intuition

```
map_order(2, vec![10, 20, 30], |x| x + 1)

jobs on the channel: (0, 10), (1, 20), (2, 30)

possible interleaving:
  worker 0 takes (0, 10), computes 11, sends (0, 11)
  worker 1 takes (1, 20), computes 21, sends (1, 21)
  worker 0 takes (2, 30), computes 31, sends (2, 31)

result channel delivers, in some order: (0, 11), (1, 21), (2, 31)

caller:
  ordered = []
  ordered.len() is 0 <= 0, so resize to 1; ordered[0] = Some(11)
  receives (1, 21)
  ordered.len() is 1 <= 1, so resize to 2; ordered[1] = Some(21)
  receives (2, 31)
  ordered.len() is 2 <= 2, so resize to 3; ordered[2] = Some(31)

every worker has dropped its result sender, so iter() ends
ordered.len() == 3 == expected
flattened: [11, 21, 31]
```

Time and Space Complexity

Re-source	Cost
Time	$O(n)$ mapper calls, $O(\text{worker_count})$ thread creations, $O(n)$ lock acquisitions and $O(n)$ sends
Space	$O(n)$ for the jobs, the results, and the slot vector

Limitations

The receivers are contended, and the lock is held while a worker waits. A worker that finds the queue empty holds the mutex while blocking in `recv`, so any other worker that finishes a job and comes back for more waits for the mutex to be released. `recv` releases the lock internally once it

starts waiting, the standard library's channel implementation releases the "blocking" case through a condition variable, but the window between the lock being taken and the wait beginning is real, and it is the part that a multiple-consumer channel exists to remove. The `crossbeam-channel` crate provides such a channel, at the cost of one dependency.

A panic in the mapper is invisible. `let _ = handle.join()` discards the join result, so a worker that panics mid-item leaves that item's slot empty. The `assert_eq!(ordered.len(), expected)` in the common case does not fire, because `ordered` was resized by the results that did arrive, so the failure that reaches the caller is the `expect("all results present")` on the missing slot, which names no item and no cause.

The function asserts on empty input. `assert!(!inputs.is_empty())` refuses a call that has an obvious correct answer, an empty vector, and turns it into a panic. That is a design choice rather than a necessity: the loop would send nothing, the workers would exit, and the result would be empty.

The order of results on the wire is unspecified, and only the reassembly makes the output ordered. A caller that wants results as they arrive, in completion order, cannot use this function.

The closure must be 'static. `F: Fn(T) -> R + Send + Sync + 'static` means the mapper cannot borrow anything from the caller's stack. Scoped threads would lift that restriction, and they would also let `T` and `R` borrow.

Summary

- Two channels carry the work. The index recorded with each job and returned with each result is the mechanism that restores the input order.
- `Arc<Mutex<Receiver>>` is present because `mpsc` is single-consumer. The mutex makes the receiver shareable rather than protecting data.
- The lock is released before the mapper runs, so the workers execute in parallel rather than one at a time.
- A panic in the mapper is discarded by `let _ = handle.join()`. The slot for that item stays empty, and the failure surfaces as an `expect` that names neither the item nor the cause.
- `assert!(!inputs.is_empty())` turns a call with an obvious correct answer into a panic; the loop would send nothing and the result would be empty.

- The closure bound is `'static`, so the mapper cannot borrow from the caller's stack. Scoped threads would lift that restriction.

References

- Standard library, `std::sync::mpsc`.
- Standard library, `Mutex`.
- Standard library, `std::thread::scope`, the alternative that removes the `'static` bounds.
- Standard library, `JoinHandle::join`.

Mutex Poisoning

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/mutex_poisoning.rs`. Run it with `cargo run --bin mutex_poisoning`.

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

, Leslie Lamport, *Distribution*, 1987

Problem Statement

Handle the case where a thread panics while holding a lock. The data behind the lock may be half-updated, and the next thread to acquire it has to decide whether to trust what it finds.

Designing a Solution

A panic can happen in the middle of a sequence of writes that were meant to succeed or fail together. If the lock were released and the data left as it was, the next reader would see the partial update. Rust makes that failure visible rather than silent: a `Mutex` whose guard was held during a panic is *poisoned*, and later calls to `lock` return `Err(PoisonError)` instead of a guard.

thread A	thread B
-----	-----
lock() -> guard	
stock_value -= 1000	
cash_on_hand += 1200	
panic! while guard is alive	
	lock() -> Err(PoisonError)

the Mutex records that a panic happened while the lock was held	the guard is still reachable through <code>PoisonError::into_inner()</code> , so B can read the state, correct it, and release the lock
---	---

The recovery path in the file is a reconciliation: an audit prints the discrepancy, corrects the cash figure, marks the transaction closed, and takes a copy of the corrected state.

The important detail is what poisoning does *not* do. Poisoning is not access control, and recovering the data does not clear the mark. Every later `lock()` on that mutex still returns `Err`, so the decision to trust the repaired state is made by each caller rather than by the act of repair.

Implementation

```
// =====
// Mutex poisoning: failed transaction and audit recovery
//
// Ported from the Rust playground repo (`scam_transaction.rs`).
//
// Story:
//   A branch executes a financial transaction that mutates shared state under
//   a `Mutex`. Mid-transaction the thread panics. In Rust, a `Mutex` whose
//   guard was held during a panic becomes **poisoned**. Future lock() calls
//   return Err(PoisonError) instead of silently letting another thread
//   observe an inconsistent partial update.
//
// Why this appears in cloud interviews:
//   - Controllers/reconcilers mutate shared state; a panic while holding a lock
//     is a real failure mode in worker pools, caches, and rate limiters.
//   - Rust chooses fail-fast over data corruption: a poisoned mutex tells
//     callers that the protected invariant may be broken. The caller must
//     decide whether to abort, recover, or reset state.
//   - PoisonError::into_inner() returns the MutexGuard, so a recovery path
//     can inspect/correct the last known state without waiting for the lock to
//     be released by the panicked thread.
//
// Run:
//   cargo run --bin mutex_poisoning
// =====

use std::error::Error;
use std::sync::{Arc, Mutex, MutexGuard, PoisonError};
use std::thread;

/// Shared ledger state. In a real system this would be behind a transactional
/// store; here it is plain in-memory state to illustrate lock poisoning.
#[derive(Debug, Clone)]
struct Ledger {
    cash_on_hand: i64,
    stock_value: i64,
    transaction_id: u32,
    is_settled: bool,
}

impl Ledger {
    fn new() -> Self {
        Ledger {

```

```

        cash_on_hand: 10_000,
        stock_value: 5_000,
        transaction_id: 0,
        is_settled: false,
    }
}

/// Recovers from a poisoned mutex by taking ownership of the guard that the
/// panicked thread left behind.
///
/// `PoisonError::into_inner()` gives us the `MutexGuard`, which means we still
/// hold the lock while repairing state. The guard is dropped when this function
/// returns.
fn recover_ledger(poisoned: PoisonError<MutexGuard<'_, Ledger>>) -> Ledger {
    // into_inner() returns the MutexGuard. We own the lock exclusively even
    // though the previous holder panicked.
    let mut guard = poisoned.into_inner();

    println!("[Reconciliation] Physical audit reveals a cash discrepancy.");
    println!(
        "[Reconciliation] Digital cash: {} | Corrected cash: {}",
        guard.cash_on_hand,
        guard.cash_on_hand - 200
    );

    // Correct the data while we still hold the guard.
    guard.cash_on_hand -= 200;
    guard.is_settled = true;
    guard.transaction_id += 1;

    println!("[Reconciliation] Ledger corrected. Transaction marked failed and
closed.");

    // Clone the inner data to return an owned value. The guard drops here and
    // releases the lock.
    guard.clone()
}

fn main() -> Result<(), Box<dyn Error>> {
    let ledger = Arc::new(Mutex::new(Ledger::new()));

    // Thread A: the faulty transaction. It mutates one side of the ledger and
    // then panics before marking the transaction settled.
    let ledger_a = Arc::clone(&ledger);
    let handle_a = thread::spawn(move || {
        // SAFETY-free normal code: if the mutex is already poisoned, unwrap()
        // panics; in a controlled demo this is the first lock so it is fine.
        let mut guard = ledger_a.lock().unwrap();

        println!(
            "[Branch A] Transaction {} commenced.",
            guard.transaction_id + 1
        );
        guard.stock_value -= 1_000;
        println!(
            "[Branch A] Stock deducted. New value: {}",
            guard.stock_value
        );
        guard.cash_on_hand += 1_200;
        println!(
            "[Branch A] Cash updated. New balance: {}",
            guard.cash_on_hand
        );

        // Panic while `guard` is still alive -> the Mutex becomes poisoned.
        panic!("[Branch A] Settlement failure: arithmetic overflow in

```

```

reconciliation.");
});

// A panic in a spawned thread does not abort the process unless the panic
// strategy is set to abort. `join()` returns Err, which is expected here.
let _ = handle_a.join();

// Thread B: the forensic audit. It sees Err(PoisonError), inspects the
// recovered state, and repairs the ledger.
let ledger_b = Arc::clone(&ledger);
let handle_b = thread::spawn(move || {
    println!("\n[Audit] Attempting to access the ledger...");

    match ledger_b.lock() {
        Ok(guard) => {
            println!("[Audit] Ledger is not poisoned: {:?}", *guard);
        }
        Err(poisoned) => {
            println!("[Audit] Mutex is poisoned. Recovered state before
correction:");
            println!("[Audit] {:?}", *poisoned.get_ref());

            let corrected = recover_ledger(poisoned);
            println!("[Audit] Corrected ledger: {:?}", corrected);
        }
    }
});

handle_b.join().unwrap();

// Important: recovering the data does NOT clear the poison bit on the
// Mutex itself. Subsequent lock() calls still return Err(PoisonError) so
// callers can decide whether the repaired state is trustworthy.
// Uncommenting the next lines would panic:
//
// let _guard = ledger.lock().unwrap();

Ok(())
}

// =====
// Tests (cargo test --bin mutex_poisoning)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn ledger_starts_in_expected_state() {
        let ledger = Ledger::new();
        assert_eq!(ledger.cash_on_hand, 10_000);
        assert_eq!(ledger.stock_value, 5_000);
        assert!(!ledger.is_settled);
    }

    #[test]
    fn recovery_repairs_a_poisoned_ledger() {
        let ledger = Arc::new(Mutex::new(Ledger::new()));

        // Poison the mutex from a helper thread, exactly like the demo.
        let ledger_a = Arc::clone(&ledger);
        let handle = thread::spawn(move || {
            let _guard = ledger_a.lock().unwrap();
            panic!("boom");
        });
        let _ = handle.join();
    }
}

```

```

    let lock_result = ledger.lock();
    assert!(lock_result.is_err(), "mutex should be poisoned");

    let recovered = recover_ledger(lock_result.unwrap_err());
    assert!(recovered.is_settled);
    assert_eq!(recovered.transaction_id, 1);
    assert_eq!(recovered.cash_on_hand, 10_000 - 200);
}

```

`recover_ledger` takes the `PoisonError` by value. That is the signature that makes the type system enforce what the function needs: a `PoisonError<MutexGuard<...>>` can only be produced by a failed `lock`, so the function cannot be called with a healthy lock by mistake.

`poisoned.into_inner()` consumes the error and returns the guard. The lock is now held by this thread, so the repair happens under the lock rather than beside it.

`guard.clone()` returns an owned copy of the state. `MutexGuard` is not `Clone`, so the clone is of the inner value, taken through the guard. The guard is dropped when the function returns, which releases the lock.

`handle_b.join().unwrap()` propagates a panic from the audit thread. `main` returns `Result`, so an `unwrap` here would panic rather than return an error; the join result is discarded for the first thread with `let _ = handle_a.join()`, and the comment explains that the panic is the point of the demonstration.

The test calls `recover_ledger` with the error from a real poisoned lock, so the recovery path is exercised rather than described.

Intuition

```

ledger after new:  { cash 10000, stock 5000, transaction_id 0, is_settled false }

thread A          lock() -> guard, the mutex is now marked as locked
                  stock_value -= 1000    -> 4000
                  cash_on_hand += 1200   -> 11200
                  panic! while the guard is alive

                  the guard is dropped during unwinding, the mutex is unlocked,
                  and the poison bit is set

join A            returns Err, which the code discards

thread B          lock() -> Err(PoisonError)
                  poisoned.get_ref() shows the partial update:
                  { cash 11200, stock 4000, transaction_id 0, is_settled false }
                  recover_ledger:
                  cash 11200 - 200 = 11000
                  is_settled = true
                  transaction_id = 1

```

```

the guard is dropped, the lock is released

after the run    the mutex is still poisoned, and the ledger inside it reads
                  { cash 11000, stock 4000, transaction_id 1, is_settled true }

```

Time and Space Complexity

Re-source	Cost
Time	one lock acquisition per access; the recovery path prints four lines and clones the state once
Space	the ledger is four machine words; a <code>PoisonError</code> is the size of a guard

Limitations

The repair rule is a constant in the code. `guard.cash_on_hand -= 200` is the audit's correction, and it is hardcoded. In a real reconciliation the correct figure comes from an external source, a bank statement, a database, and the function would take that source as a parameter. As written, the demonstration shows the mechanism and not a policy.

Recovering does not clear the poison mark. The comment at the end of `main` says so, and the line that would panic is left commented out. Every subsequent `lock()` therefore returns `Err`, and a program whose recovery is complete has no way to say so to the other threads without dropping the mutex entirely. `Mutex::clear_poison` exists for that, and it is not used here.

The failed thread's own recovery is absent. Thread A panics and its work is lost; nothing rolls back the partial update except thread B's audit. A system where either thread might be the one to recover needs the repair to be idempotent and runnable by any caller.

`ledger_a.lock().unwrap()` panics if the mutex is already poisoned. The comment acknowledges it. In a program where several threads may touch the same lock, `unwrap` on a `lock` result converts one thread's failure into a cascade of panics, which is the opposite of what poisoning is for.

The audit thread's panic would be propagated with `unwrap`. `main` returns `Result<(), Box<dyn Error>>`, so the error type on the happy path is unused here: the only failure that can occur is a panic, and a panic cannot

be converted into `Box<dyn Error>` by `?. join()` returns `Result<T, Box<dyn Any + Send>>`, and `Box<dyn Any + Send>` does not implement `Error`.

Summary

- Poisoning records that a panic occurred while the guard was held. It repairs nothing and it does not deny access to the data.
- `PoisonError::into_inner()` returns the guard, so the thread that recovers also holds the lock while it repairs the state, and the repair does not race with other readers.
- Recovering is appropriate where the invariant can be re-established from data the program trusts, and propagating is appropriate where it cannot. In both cases the defect is a recovery with no stated repair rule.
- The repair rule here is a constant, `guard.cash_on_hand -= 200`. A reconciliation would take the correct figure from an external source and pass it in.
- Recovering does not clear the poison mark, so every later `lock()` returns `Err` and the other threads cannot be told that the repair is complete. `Mutex::clear_poison` exists for that purpose and is not used here.
- `ledger_a.lock().unwrap()` converts one thread's failure into a cascade of panics, which is the opposite of what poisoning is for.

References

- Standard library, `Mutex`, including the section on poisoning.
- Standard library, `PoisonError::into_inner`.
- Standard library, `Mutex::clear_poison`, stabilised in Rust 1.77.0.
- Standard library, `JoinHandle::join`.

Counting Islands

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/count_islands.rs`. Run it with `cargo run --bin count_islands`.

Problem Statement

Given a rectangular grid of cells, count the connected regions of a chosen value. Two cells are connected when they share an edge, so a cell touches at most four neighbours. The function is generic over the cell type, so the caller decides whether land is `'1'`, `true`, `1u8`, or something else.

Designing a Solution

Scan the grid and flood-fill each unvisited land cell. Counting the number of flood-fills is the answer, because a flood-fill visits exactly one connected region.

```
grid          count
 1 1 0 0 0
 1 1 0 0 0      scan finds (0,0), fills the four cells of the top-left block
 0 0 1 0 0      scan finds (2,2), fills that single cell
 0 0 0 1 1      scan finds (3,3), fills (3,3) and (3,4)
                  answer: 3

the fill order for the first region, breadth first
queue: [(0,0)]
pop (0,0), push (0,1) and (1,0)
pop (0,1), push (1,1)
pop (1,0), neighbours already visited
pop (1,1), neighbours already visited
```

A cell is marked when it enters the queue rather than when it leaves. Marking on entry is what keeps a cell from being enqueued twice: two neighbours of the

same cell can both see it as unvisited if the mark happens later, and the queue then holds duplicates and the work grows.

Implementation

```
// =====
// Count Islands
//
// Ported from the Rust playground repo.
//
// Problem: given a grid of cells, count the number of connected "land"
// regions. Two land cells are connected when they are adjacent horizontally
// or vertically (4-directional). Water cells are not traversable.
//
// Why this appears in cloud interviews:
// - "Islands" are connected-component counting, the same shape used to find
//   connected GPU nodes, failure domains, network partitions, or storage
//   groups in distributed systems.
// - The generic implementation accepts any land type (char, u8, bool) and
//   uses BFS with an explicit visited set, which is easy to reason about.
//
// Time complexity: O(rows * cols)
// Space complexity: O(rows * cols) for the visited set and BFS queue.
// =====

use std::collections::{HashSet, VecDeque};

/// Counts 4-directionally connected components of `land` in a rectangular grid.
///
/// The grid is generic over cell values so callers can use `'1'/'0'`,
/// `true/false`, `1u8/0u8`, or any other PartialEq value.
fn count_islands<T: PartialEq>(grid: &[&[T]], land: T) -> usize {
    if grid.is_empty() || grid[0].is_empty() {
        return 0;
    }

    let rows = grid.len();
    let cols = grid[0].len();
    let mut visited = HashSet::new();
    let mut count = 0;
    let dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)];

    // BFS closure: flood-fill one island starting from a known land cell.
    // Passing `visited` explicitly avoids borrow-checker friction when calling
    // the closure from the outer loop.
    let bfs = |start: (usize, usize), visited: &mut HashSet<(usize, usize)>| {
        let mut queue = VecDeque::new();
        queue.push_back(start);
        visited.insert(start);

        while let Some((cr, cc)) = queue.pop_front() {
            for &(dr, dc) in &dirs {
                let nr = cr as i32 + dr;
                let nc = cc as i32 + dc;

                if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {
                    let (ur, uc) = (nr as usize, nc as usize);
                    if grid[ur][uc] == land && !visited.contains(&(ur, uc)) {
                        visited.insert((ur, uc));
                        queue.push_back((ur, uc));
                    }
                }
            }
        }
    };

    for (r, row) in grid.iter().enumerate() {
        for (c, cell) in row.iter().enumerate() {
            if cell == land && !visited.contains(&(r, c)) {
                bfs((r, c), &visited);
                count += 1;
            }
        }
    }

    count
}
```

```

    }
};

for r in 0..rows {
    for c in 0..cols {
        if grid[r][c] == land && !visited.contains(&(r, c)) {
            count += 1;
            bfs((r, c), &mut visited);
        }
    }
}

count
}

// =====
// Runnable demonstration
// =====
fn main() {
    let grid = [
        &['1', '1', '0', '0', '0'][..],
        &['1', '1', '0', '0', '0'][..],
        &['0', '0', '1', '0', '0'][..],
        &['0', '0', '0', '1', '1'][..],
    ];

    println!("Islands: {}", count_islands(&grid, '1'));

    let boolean_grid = [
        &[true, true, false][..],
        &[false, false, false][..],
        &[false, true, true][..],
    ];
    println!("Boolean islands: {}", count_islands(&boolean_grid, true));
}

// =====
// Tests (cargo test --bin count_islands)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn empty_grid_has_zero_islands() {
        let empty: [&[char]; 0] = [];
        assert_eq!(count_islands(&empty, '1'), 0);

        let no_cols: [&[char]; 2] = [&[], &[]];
        assert_eq!(count_islands(&no_cols, '1'), 0);
    }

    #[test]
    fn counts_classic_example() {
        let grid = [
            &['1', '1', '0', '0', '0'][..],
            &['1', '1', '0', '0', '0'][..],
            &['0', '0', '1', '0', '0'][..],
            &['0', '0', '0', '1', '1'][..],
        ];
        assert_eq!(count_islands(&grid, '1'), 3);
    }

    #[test]
    fn all_water_is_zero() {
        let grid = [&['0', '0'][..], &['0', '0'][..]];
        assert_eq!(count_islands(&grid, '1'), 0);
    }
}

```

```

    }

    #[test]
    fn generic_bool_grid_works() {
        let grid = [
            &[true, false, false][..],
            &[true, false, true][..],
            &[false, false, true][..],
        ];
        assert_eq!(count_islands(&grid, true), 2);
    }

    #[test]
    fn single_row_and_column() {
        let row = [&['1', '0', '1', '1'][..]];
        assert_eq!(count_islands(&row, '1'), 2);

        let col = [&['1'][..], &['1'][..], &['0'][..], &['1'][..]];
        assert_eq!(count_islands(&col, '1'), 2);
    }
}

```

The grid type is `&[&[T]]`: a slice of row slices. This accepts a fixed-size array of slices (`&['1', '1', '0'][..]`) and a vector of vectors, and it requires every row to have the same length. The code reads `grid[0].len()` as the column count and assumes the rest of the rows match.

`count_islands<T: PartialEq>(grid: &[&[T]], land: T)` takes `land` by value. The comparison `grid[r][c] == land` therefore needs `PartialEq<T>` for `T`, which the single bound provides. The bound is minimal: nothing else about the cell type is required.

The bounds test casts to `i32` before adding the delta:

```

let nr = cr as i32 + dr;
let nc = cc as i32 + dc;

if (0..rows as i32).contains(&nr) && (0..cols as i32).contains(&nc) {

```

A `usize` cannot represent `-1`, so a neighbour above or to the left of the origin would wrap to a very large index rather than failing a range check. Signed arithmetic makes the range test correct for all four directions in one expression.

The `bfs` closure takes `visited` as a parameter rather than capturing it mutably. A closure that captured `visited` would hold a mutable borrow across the whole outer loop, and the loop also reads `visited`. Passing the reference in makes the borrow begin and end at the call.

The demonstration prints two lines, and the tests exercise the same two grids plus four boundary cases.

Intuition

```

grid
row 0: 1 1 0 0 0
row 1: 1 1 0 0 0
row 2: 0 0 1 0 0
row 3: 0 0 0 1 1

scan reach  cell  is land  visited?  action
(0,0)      yes   no      count = 1, fill from (0,0)
           yes   yes     visit (0,0), (1,0), (0,1), (1,1)
(0,1)      yes   yes     skip
(0,2)      no    -       skip
(0,3)      no    -       skip
(0,4)      no    -       skip
(1,0)      yes   yes     skip
(1,1)      yes   yes     skip
(2,2)      yes   no      count = 2, fill from (2,2), visiting one cell
(3,3)      yes   no      count = 3, fill from (3,3), visiting (3,3) and (3,4)

the run prints:  Islands: 3
                  Boolean islands: 2

```

Time and Space Complexity

Re-source	Cost
Time	$O(\text{rows} \times \text{cols})$, each cell is visited once and each visit examines four neighbours
Space	$O(\text{rows} \times \text{cols})$ for the visited set and the queue

Limitations

The visited set is a hash set of coordinate pairs. Each entry holds two `usize` values plus the hash table's overhead: control bytes, a bucket array that may be twice the entry count, and a random seed per set. A bitmap of one bit per cell answers the same question in $\text{rows} \times \text{cols} / 8$ bytes and one shift per lookup. For a 4096×4096 grid the hash set holds up to sixteen million entries, while the bitmap holds two megabytes. That factor, roughly fifty to a hundred, depending on load factor, is the difference between a large allocation and a small one, and the file's own comment says only "O(rows * cols) for the visited set".

The rows are assumed to be the same length. `cols` is read from the first row and used to bound every row. A ragged input does not panic on

the bound check for the shorter rows; it panics on `grid[ur][uc]` when a shorter row is indexed at a column the first row had. A ragged grid is a malformed input, and the failure mode is an index panic deep inside the fill.

A very large connected region is held in the queue. The BFS queue can hold a frontier proportional to the region's size, so memory is bounded by the input area in the worst case. A depth-first fill with an explicit stack has the same bound.

The function is private to the binary. `count_islands` is in `src/bin/`, so it is not part of the library's API, and a second binary that wants it must copy it. Moving it into `src/problems/` would export it with the rest of the problems.

The cell type must be `PartialEq` and the land value is taken by value. So `count_islands` cannot accept a predicate, which would let a caller count something other than strict equality, for example, a region of cells above a threshold. The predicate version is the same code with `land` replaced by a closure.

Summary

- A flood fill visits exactly one connected component, so the number of fills is the number of components.
- A cell is marked when it is enqueued rather than when it is dequeued. Without that, several neighbours can enqueue the same cell and the queue holds it more than once.
- The neighbour offsets use signed arithmetic, which is what prevents the fill from wrapping around the edge of the grid.
- The visited set is a hash set of coordinate pairs, so each entry holds two `usize` values plus the table's overhead. A bitmap of one bit per cell answers the same question in `rows × cols / 8` bytes: for a `4096 × 4096` grid the hash set holds up to sixteen million entries against two megabytes for the bitmap.
- The rows are assumed to be of equal length. A ragged grid does not fail the bound check on a short row; it panics on `grid[ur][uc]` when a short row is indexed at a column the first row had.

- The BFS queue holds a frontier proportional to the region being filled, so memory is bounded by the area of the input in the worst case. A depth-first fill with an explicit stack has the same bound.
- The function is private to its binary, and the cell type must be `PartialEq`, because the land value is taken by value and no predicate is accepted.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, Section 20.2, "Breadth-first search".
- Standard library, `VecDeque`.
- Standard library, `std::ops::Range::contains`.

Syscall Overhead

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/syscall_overhead.rs`. Run it with `cargo run --release --bin syscall_overhead`.

Latency lags bandwidth.

, David A. Patterson, *Latency lags bandwidth*, Communications of the ACM 47(10), 2004

Problem Statement

The cost of crossing from user space into the kernel is measured on one machine, and the figure is reported in a form that can be reproduced.

Designing a Solution

Every system call is a controlled transfer of control: the processor switches to kernel mode, the kernel validates the arguments, does the work, checks whether the calling thread should be descheduled, and returns. For a call that does almost nothing, that machinery is the entire cost.

`getpid` is such a call. It returns the caller's process identifier from a field the kernel already holds, takes no arguments, and touches no user memory. Its cost is therefore close to the irreducible price of the boundary itself, which is why it is the standard probe.

The measurement has three parts, and each exists because of a way a benchmark can lie.

warm-up	run the call 100 000 times without timing it the first calls fault in pages and ramp the clock
timing	read the clock, run the call 10 000 000 times, read the clock the loop body is one call, so the elapsed time per iteration is the cost of the call plus the cost of the loop
division	elapsed nanoseconds / iterations the result is an average, and the spread is not reported

Implementation

```
// =====
// Syscall overhead measurement
//
// Ported from the Rust playground repo (`mkdir_syscall.rs`).
//
// What this measures:
// - Every syscall crosses the user/kernel boundary: argument validation,
//   kernel work, scheduling checks, and return to user space.
// - This program calls `getpid()` in a tight loop and estimates the average
//   per-call cost. `getpid()` is a tiny syscall, so the measured cost is
//   close to the irreducible syscall overhead on this kernel/hardware.
//
// Why this appears in cloud interviews:
// - High-performance data planes care about per-packet or per-I/O syscall
//   cost. The difference between one syscall per event and batching via
//   io_uring/recvmmsg can be enormous.
// - It also demonstrates unsafe FFI, safety comments, warm-up loops, and
//   benchmark methodology: warm the code path, run enough iterations, and
//   report average cost rather than one lucky sample.
//
// Run:
// cargo run --bin syscall_overhead
// =====

use std::time::Instant;

/// Calls `getpid()` `iterations` times and returns the average cost in
/// nanoseconds per call.
fn measure_getpid_ns_per_call(iterations: u64) -> f64 {
    let start = Instant::now();

    for _ in 0..iterations {
        // SAFETY: `libc::getpid` is a pure read-only system call with no
        // pointer arguments and no memory side effects. The FFI declaration in
        // the `libc` crate matches the Linux kernel ABI.
        //
        // `black_box` prevents the optimizer from assuming the loop has no
        // observable effect, which matters because the syscall result is
        // otherwise unused.
        unsafe {
            libc::getpid();
        }
    }

    let elapsed = start.elapsed();
    elapsed.as_nanos() as f64 / iterations as f64
}

fn main() {
    const ITERATIONS: u64 = 10_000_000;
}
```

```

// Warm up the code path before timing. The first calls may fault in the
// libc/PLT, cache lines, or CPU frequency may still be ramping.
for _ in 0..100_000 {
    // SAFETY: same reasoning as above; read-only syscall.
    unsafe { libc::getpid() };
}

let ns_per_call = measure_getpid_ns_per_call(ITERATIONS);

println!("Iterations:      {}", ITERATIONS);
println!(
    "Total time:          {:.3}s",
    ns_per_call * ITERATIONS as f64 / 1e9
);
println!("Per syscall:       {:.1} ns", ns_per_call);

// Example framing: if a service processed 10M packets/s and used one
// getpid-style syscall per packet, this fraction of each second would be
// spent crossing into the kernel just for the syscall overhead.
let cpu_seconds_per_second = ns_per_call * 10_000_000.0 / 1_000_000_000.0;
println!(
    "At 10M calls/s, syscall overhead would consume: {:.2} CPU-seconds per
second",
    cpu_seconds_per_second
);

// =====
// Tests (cargo test --bin syscall_overhead)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn measurement_returns_a_positive_finite_number() {
        let ns = measure_getpid_ns_per_call(10_000);
        assert!(ns.is_finite(), "nanoseconds per call must be finite");
        assert!(ns > 0.0, "a syscall cannot cost zero time");
        // On any real kernel this is microseconds or tens/hundreds of ns, not
        // seconds per call.
        assert!(
            ns < 1_000_000.0,
            "suspiciously slow syscall measurement: {ns} ns"
        );
    }

    #[test]
    fn getpid_returns_nonzero_pid() {
        // SAFETY: read-only syscall, no pointers, no side effects.
        let pid = unsafe { libc::getpid() };
        assert!(pid > 0);
    }
}

```

Three parts of this file need comment.

The SAFETY comments state the obligation: no pointer arguments, no memory side effects, and an FFI declaration that matches the kernel ABI. That is what an `unsafe` block's comment is for, the reasoning that justifies the claim that the block cannot cause undefined behaviour.

The comment in `measure_getpid_ns_per_call` also mentions `black_box`, and the code does not call `black_box`. The call would be the right thing to do: without it, the loop's result is unused, and the optimiser is entitled to remove work whose result nobody observes. What prevents removal today is that a call through `libc` is opaque to the optimiser, an external function call is a side effect it cannot delete, so the loop survives, and the comment describes an intention rather than the code. `std::hint::black_box(libc::getpid())` would make the code match its comment.

The trailing space after the `unsafe` block in the warm-up loop is invisible in a rendered page and present in the source. It is noted here because the listing is otherwise a faithful copy of the file.

The final projection multiplies the measured cost by ten million calls per second and divides by a billion, which yields a figure in CPU-seconds per second. A result above 1.0 means that one core cannot sustain that rate: the arithmetic is saying that the kernel crossing alone would need more than one core's worth of time.

Intuition

Run on an AArch64 Linux machine, release build, with `rustc` 1.96.0-nightly, while no other work was running on the machine:

```
Iterations:      10000000
Total time:      2.076s
Per syscall:     207.6 ns
At 10M calls/s, syscall overhead would consume: 2.08 CPU-seconds per second
```

The last line is the answer to the question this program exists to raise. A service that makes one such call per request at ten million requests per second needs more than two cores doing nothing but crossing the boundary.

Two qualifications belong with the number. The machine is a development board with a specific kernel, and a different machine will produce a different figure; the program prints its platform for that reason. And `getpid` is cheaper than most system calls: a read or a write copies data and an `open` walks the file system, so the number here is a floor for the boundary and not a typical cost.

Time and Space Complexity

Resource	Cost
Time	one clock read per measurement, one system call per iteration
Space	$O(1)$; nothing is allocated

Limitations

The loop is not empty, so the measurement includes it. A call, a bounds check that the optimiser removes, and an increment are part of every iteration. For a 200-nanosecond system call the loop's own cost is a rounding error, and for the control measurement of an empty loop it is the whole measurement. The program has no control row, so the loop's cost is not reported separately.

The average hides the distribution. One number cannot show that a call takes 150 nanoseconds in the common case and a microsecond occasionally, which is what an interference-heavy machine produces. The minimum of several batches is the value least disturbed by other activity on the machine, and the program does not compute it.

getpid may be served without a trap on some architectures. Some kernels implement particular calls through a vDSO shared page that the process reads directly, which turns a system call into a memory read. The program measures what the call costs on this kernel, and the figure is a boundary cost only if the kernel traps.

The program builds the library it links against, and the library emits a warning. Running `cargo run --bin syscall_overhead` prints a `dead_code` warning for an unused function in the library. The warning comes from `merge_intervals2`, which Chapter 5 identifies, and it appears in this chapter's output because Cargo compiles the whole crate.

Summary

- `getpid` gives a floor for the cost of crossing the process boundary. It is not the cost a service would see per request.

- The first iterations pay for page faults, for lazy symbol resolution, and for a clock that has not reached its steady frequency, so the run includes a warm-up pass.
- The figure converts into a consequence. At 207 nanoseconds per call and ten million calls per second, the boundary alone consumes two cores.
- Two measurements would strengthen the result: a control measurement of the empty loop, and the minimum over several batches. Neither is present.

References

- `libc`, `getpid`.
- Standard library, `std::time::Instant`.
- Standard library, `std::hint::black_box`.
- David A. Patterson, "Latency lags bandwidth", *Communications of the ACM* 47(10), 2004, pages 71–75.

Shadowing and Mutation

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/shadowing.rs`. Run it with `cargo run --bin shadowing`.

Problem Statement

Two constructs look similar in source and behave differently: giving a name a new value with `let`, and changing the value a binding already holds.

Designing a Solution

A `let` introduces a binding. Writing a second `let` with the same name introduces a *second* binding that shadows the first; the first is still there, still owns whatever it owns, and is no longer reachable by that name. Because a new binding can have a new type, shadowing is the mechanism for converting a value.

Mutation is a different operation. `let mut count = 0` creates one binding whose value may be replaced through it, and `count += 1` writes to that place. The type is fixed at the declaration, and no new binding appears.

shadowing

```
let value: u32 = 7;
  binding A: u32

let value = value.to_string();
  binding B: String
  A is unreachable by name
  A is dropped at the end of
  its scope
```

mutation

```
let mut count = 0;
  binding: i32, mutable

count += 1;
  the same binding, new value

count += 1;
  again
```

```

let value = value.len();      count
    binding C: usize          one binding in the whole block
    B is unreachable by name

```

The two mechanisms solve different problems. Shadowing changes the type, and it makes every later reader of the name see the new one. Mutation keeps the type and the binding, which is what a counter or an accumulator needs.

Implementation

```

// =====
// Variable shadowing vs mutation
//
// Ported from the Rust playground repo (`shadowing.rs`).
//
// What this demonstrates:
// - Rust allows a new `let` binding with the same name as an existing
//   binding. This is called shadowing.
// - Shadowing can change the value and the type of a variable without
//   declaring it `mut`.
// - The old binding is no longer reachable after the shadowing `let`.
//
// Why this appears in interviews:
// - Rust SDK code often shadows parsed/deserialized values to convert them
//   from raw input to typed domain values:
//
//     let raw = request.header("X-GPU-Count")?;
//     let raw: u64 = raw.parse()?; // shadow with a typed value
//
// - Shadowing is also useful after unwrapping an `Option`/`Result` when the
//   guard value is no longer needed.
//
// Run:
// cargo run --bin shadowing
// =====

fn demonstrate_shadowing() -> String {
    // First binding is an integer.
    let value: u32 = 7;
    let mut log = format!("integer: {value}");

    // Shadow the integer with a String. The old integer is gone.
    let value = value.to_string();
    log.push_str(&format!("\nstring: {value}"));

    // Shadow the String with a usize computed from it.
    let value = value.len();
    log.push_str(&format!("\nusize: {value}"));

    log
}

fn demonstrate_mutation() -> String {
    // If you need to mutate the same binding's value (same type) without
    // creating a new binding, mark it `mut`.
    let mut count = 0;
    let mut log = String::from("mut count:");
    for _ in 0..3 {
        count += 1;
        log.push_str(&format!(" {count}"));
    }
}

```

```

    log
}

fn main() {
    println!("=== Shadowing ===");
    println!("{}", demonstrate_shadowing());

    println!("\n=== Mutation ===");
    println!("{}", demonstrate_mutation());
}

// =====
// Tests (cargo test --bin shadowing)
// =====
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shadowing_allows_type_changes() {
        let log = demonstrate_shadowing();
        assert!(log.contains("integer: 7"));
        assert!(log.contains("string: 7"));
        assert!(log.contains("usize: 1"));
    }

    #[test]
    fn mutation_requires_mut_binding() {
        let log = demonstrate_mutation();
        assert!(log.contains("mut count: 1 2 3"));
    }
}

```

`let value = value.to_string()` uses the old `value` on the right and introduces the new binding on the left. The order is what makes the idiom legal: the right side is evaluated with the old binding in scope, and the new binding takes effect after the statement.

`let value = value.len()` then shadows the `String`, and the value becomes `1` because `"7"` is one character. The chain of three bindings ends with a `usize`.

The conversion idiom in the header comment is the reason the feature matters in application code:

```

let raw = request.header("X-Device-Count")?;
let raw: u64 = raw.parse()?;

```

The second `let` reuses the name for the converted value. There is no chance of using the unparsed text afterwards, because the name now refers to the parsed number. A `mut` version would need two names or an early `drop`, and neither is as clear about which value is in play.

`demonstrate_mutation` shows the other side: one binding, marked `mut`, whose value changes three times. The loop's counter has one name and one type,

which is exactly what shadowing cannot express, because each `let` in a loop body would create a fresh binding that dies at the end of the iteration.

Intuition

`demonstrate_shadowing`

statement	bindings in scope	log so far
<code>let value: u32 = 7</code>	<code>value: u32 = 7</code>	
<code>let mut log = format!(...)</code>	<code>value: u32 = 7</code> <code>log: String</code>	<code>"integer: 7"</code>
<code>let value = value.to_string()</code> <code>7"</code>	<code>value: String = "7"</code> (the <code>u32</code> binding is unreachable)	<code>"integer: 7\nstring: 7"</code>
<code>let value = value.len()</code> <code>7\nusize: 1"</code>	<code>value: usize = 1</code> (the <code>String</code> binding is unreachable)	<code>"integer: 7\nstring: 7\nusize: 1"</code>
<code>return log</code>		

the function returns `"integer: 7\nstring: 7\nusize: 1"`, and the test asserts on the three lines separately

`demonstrate_mutation`

statement	bindings	log
<code>let mut count=0</code>	<code>count: i32 = 0</code>	
<code>let mut log</code>	<code>count: i32 = 0</code> <code>log: String</code>	<code>"mut count:"</code>
iteration 1	<code>count = 1</code>	<code>"mut count: 1"</code>
iteration 2	<code>count = 2</code>	<code>"mut count: 1 2"</code>
iteration 3	<code>count = 3</code>	<code>"mut count: 1 2 3"</code>

one binding for `count`, changed three times

Time and Space Complexity

Resource	Cost
Shadowing	no run-time cost; the old binding is dropped at the end of its scope, if it owns anything
Mutation	no run-time cost beyond the write itself
<code>format!</code>	one <code>String</code> allocation per call

The only allocation in either function is the `log`, which is built with `format!` and `push_str`. The conversion `value.to_string()` also allocates a `String`, and it is freed at the end of `demonstrate_shadowing` when the last binding to it goes out of scope.

Limitations

The comment says the old binding is "gone"; the value is not necessarily freed. Shadowing hides a name. The original binding's storage lasts until the end of its scope, and for a type with a destructor the resource is released at the end of the scope, not at the shadowing `let`. For `u32` and `String` in this file the distinction has no observable effect; for a file handle or a lock guard it does, because the guard stays alive until the end of the block. The comment's wording is more absolute than the rule.

Shadowing hides mistakes as well as values. A `let` that accidentally reuses a name produces no warning, and the compiler's unused-variable lint may not fire because the first binding was used before it was shadowed. The mechanism is powerful and it is not checked.

The file has no test for the shadowed value being unreachable. The tests assert on the log's contents, which proves that each binding was used. Nothing demonstrates the *lifetime* point, that an owned resource in a shadowed binding is released at the end of the scope, not at the shadowing statement.

Two `demonstrate_*` functions return a `String` built by `format!`. A version that took `&mut String` and appended to the caller's buffer would allocate less, and the tests would be the same. For a demonstration the difference is irrelevant; it is the kind of allocation that shows up in a profile.

Summary

- `let` introduces a binding and `mut` allows a place to be written. The two are independent.
- The conversion idiom is the reason shadowing appears in ordinary code. Parsing input and reusing the name avoids inventing a second name for the same value.
- Shadowing does not destroy the shadowed binding. Its storage lasts until the end of the scope, so a value with a destructor is released at the end of the block rather than at the shadowing statement.
- A `mut` binding fixes the type, which is why a counter uses `mut` and a conversion uses `let`.

- Shadowing hides mistakes as well as values: a `let` that reuses a name by accident produces no warning.

References

- The Rust Book, [Shadowing](#).
- The Rust Book, [Variables and mutability](#).
- The Rust Reference, [Names, scopes, and shadowing](#).

Part VII: Files, Directories, and Arguments

Reading and Writing Files

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/read_write_file.rs`, `src/bin/append_to_file_open_options.rs`, and `src/bin/readfile_line_by_line.rs`. Run them with `cargo run --bin read_write_file`, `cargo run --bin append_to_file_open_options`, and `cargo run --bin readfile_line_by_line`.

Problem Statement

Three programs cover the file operations a command-line tool needs: write a file from a string, add lines to an existing file, and read a file back, either whole or one line at a time. Each reports a failure through `io::Result` rather than terminating on the first error.

Designing a Solution

`std::fs` provides the whole-value operations. `fs::write` creates a file or replaces its contents in one call, and `fs::read_to_string` reads an entire file into a `String`. Both return `io::Result`, so `?` propagates a failure to the caller, and returning that result from `main` prints the error and sets a nonzero exit status.

Appending needs `OpenOptions`, because `fs::write` truncates. `OpenOptions::new()` assembles the flags for a single `open` call: `create(true)` makes the file when it is missing, and `append(true)` places every write at the end. Two `writeln!` calls then add two lines without reading the existing contents.

Reading line by line uses a `BufReader`. The reader owns a buffer and fills it in blocks, so `lines()` does not issue one system call per line. `BufRead` must be in

scope for the method to resolve, even though the value is a `BufReader`; that is the only trait import the program needs. Each item is an `io::Result<String>` without its terminator, so an unreadable line is reported rather than skipped.

Implementation

`read_write_file.rs` writes a string and reads it back through two paths built in different ways:

```
use std::fs;
use std::io;
use std::path::{Path, PathBuf}; // Added Path here to change &PathBuf to &Path for
// idiomatic borrowing

// Fixed to be idiomatic by using &Path instead of &PathBuf, and returning the String
fn read_content_of_file(path: &Path) -> io::Result<String> {
    let content = fs::read_to_string(path)?;
    Ok(content)
}

fn main() -> io::Result<()> {
    // Write a file (creates or overwrites)
    fs::write("hello.txt", "Hello, NVIDIA!\n")?;

    // --- Demo 1: Non-mutating .join() ---
    let base_dir = PathBuf::from(".");
    let joined_path = base_dir.join("hello.txt");

    // --- Demo 2: In-place .push() ---
    let mut mut_base_dir = PathBuf::from(".");
    mut_base_dir.push("hello.txt");

    // Read it back as a String using your function and a joined path
    let text = read_content_of_file(&joined_path)?;
    println!("File contents (via .join()):\n{}", text);

    let text_inplace = read_content_of_file(&mut_base_dir)?;
    println!("File contents (via .push()):\n{}", text_inplace);

    Ok(())
}
```

`append_to_file_open_options.rs` adds two lines to a file that survives the run:

```
use std::fs::OpenOptions;
use std::io::Write;

fn main() -> std::io::Result<()> {
    let mut file = OpenOptions::new()
        .create(true) // create if missing
        .append(true) // always write at the end
        .open("log.txt")?;

    writeln!(file, "New log line")?;
    writeln!(file, "Another line")?;

    Ok(())
}
```

`readfile_line_by_line.rs` numbers the lines as it prints them:

```
use std::fs::File;
use std::io::{BufRead, BufReader}; // Huh! You beed to import this explicitly,
                                     interesting!

fn main() -> std::io::Result<()> {
    let file = File::open("log.txt")?;
    let reader = BufReader::new(file);

    for (i, line) in reader.lines().enumerate() {
        let line = line?;
        println!("{}", i + 1, line);
    }

    Ok(())
}
```

`read_content_of_file` takes `&Path` rather than `&PathBuf`. A `&PathBuf` coerces to `&Path` at the call site, but a `&Path` does not coerce to `&PathBuf`, so the `&Path` signature accepts strictly more callers. The body binds the result of `fs::read_to_string` and returns it, which is the same value with one extra binding; the `?` inside the call already converts the error type to `io::Error`.

`OpenOptions` is consumed by `open`, which is why the builder chain ends in a single statement. The two flags are independent: `create` decides what happens when the file is absent, and `append` decides where each write lands. Without `create`, opening a missing `log.txt` returns `ErrorKind::NotFound`.

Intuition

`read_write_file.rs`, from an empty working directory:

statement	effect
<code>fs::write(...)</code>	hello.txt is created with "Hello, NVIDIA!\n"
<code>PathBuf::from(".").join(...)</code>	<code>joined_path = "./hello.txt"</code>
<code>push("hello.txt")</code>	<code>pushed_path = "./hello.txt"</code>
<code>read_content_of_file(joined)</code>	returns "Hello, NVIDIA!\n"
<code>read_content_of_file(pushes)</code>	returns the same string

`append_to_file_open_options.rs`, run twice with `log.txt` absent at the start:

```
run 1  log.txt -> "New log line\nAnother line\n"
run 2  log.txt -> "New log line\nAnother line\nNew log line\nAnother line\n"

the file is opened for append, so the second run adds to the end rather than
replacing the contents
```

`readfile_line_by_line.rs` over that file:

```
1: New log line
2: Another line
```

The index is one-based because `enumerate` starts at zero and the program prints `index + 1`.

Time and Space Complexity

Operation	Cost	Condition
<code>fs::read_to_string</code>	$O(n)$ time, $O(n)$ space	n bytes in the file; the whole file is held in memory
<code>fs::write</code>	$O(n)$ time	one truncation followed by the writes
<code>writeln!</code> on an append handle	one write per call	the handle is not line-buffered; each call reaches the kernel
<code>BufReader::lines</code>	$O(n)$ time, $O(1)$ space in the buffer	one <code>String</code> allocation per line

`read_to_string` fails when the bytes are not valid UTF-8, and it allocates the whole file before returning. A program that reads a large file should keep the `File` and read through the `BufReader` rather than calling `read_to_string`.

Limitations

Neither writer is atomic. A process that stops between the truncation and the writes leaves a partial file. Replacing a file without that window is done by writing a temporary file in the same directory and renaming it over the target.

No call requests a flush. `fs::write` and `writeln!` return once the data has reached the operating system, not the storage device, so a power loss can lose recent writes. `File::sync_all` is the call that waits for the device.

Relative paths depend on the working directory. All three programs use names such as `hello.txt` and `log.txt`, which resolve against the directory the program was started in. Running `readfile_line_by_line` before `append_to_file_open_options` reports `NotFound`.

lines() splits on one separator. It divides on `\n` and removes a trailing `\r`, so a file separated by `;` or by a bare `\r` is read as one line. The final line is yielded even when it has no terminator.

The two writers ignore the returned length. `writeln!` can write fewer bytes than requested; the callers rely on `io::Write` to report that as an error, and neither program checks how many bytes reached the file.

Summary

- `fs::write` replaces contents, `OpenOptions` with `append(true)` adds to them, and the difference is the flag rather than a separate function.
- `?` converts an `io::Error` into the caller's error type, which is what lets these programs read as a sequence of statements.
- `BufReader` turns many small reads into block reads, and `BufRead` has to be imported for `lines` to resolve.
- Relative file names resolve against the working directory, so these programs are not independent of where they are started.

References

- Standard library, `std::fs`.
- Standard library, `OpenOptions`.
- Standard library, `BufReader`.
- Standard library, `BufRead::lines`.
- Standard library, `PathBuf`.

Paths and Directory Traversal

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/path_buff.rs`, `src/bin/list_directory.rs`, and `src/bin/recursive_directory_walk.rs`. Run them with `cargo run --bin path_buff`, `cargo run --bin list_directory`, and `cargo run --bin recursive_directory_walk`.

Problem Statement

Three programs answer the questions a tool needs about the filesystem before it opens anything: how a path is assembled and taken apart, what a directory contains, and how to visit every file below a starting directory.

Designing a Solution

A path is a sequence of components, not a string. `Path` is the borrowed form and `PathBuf` is the owned form, with the same relationship as `str` and `String`. `join` builds a new `PathBuf` and leaves the receiver alone; `push` extends the receiver in place. Because a directory separator differs between platforms, the two methods insert the platform's separator, which string concatenation does not.

The inspection methods return `Option` rather than a sentinel, because a path need not have the component asked for. `/tmp/data.txt` has a parent, a file name and an extension; `/` has none of the three. Returning `None` for those cases keeps the caller from inventing a value.

`fs::read_dir` yields one `io::Result<DirEntry>` per directory entry. The `Result` matters when an entry cannot be read, which `for entry in fs::read_dir(dir)?` would otherwise hide. `DirEntry::path` produces an owned `PathBuf`, and

`DirEntry::file_type` returns the entry's type from the directory listing, which avoids the metadata lookup that `Path::is_dir` performs per entry.

A recursive walk is the same loop with a call to itself for each subdirectory, and `?` on that call stops the walk at the first unreadable directory.

Implementation

`path_buff.rs` builds one path with `join` and another with `push`, then reads parts of the second:

```
use std::path::{Path, PathBuf};

fn main() {
    let dir = Path::new("/tmp");
    let file = dir.join("data.txt");

    let mut buf = PathBuf::from(dir);
    buf.push("nested");
    buf.push("file.txt");

    println!("file: {}", file.display());
    println!("buf: {}", buf.display());
    println!("parent: {:?}", buf.parent());
    println!("file_name: {:?}", buf.file_name());
    println!("ext: {:?}", buf.extension());
}
```

`list_directory.rs` prints one line per entry of the working directory:

```
use std::fs;
use std::io;

fn main() -> io::Result<()> {
    for entry in fs::read_dir(".")? {
        let entry = entry?;
        let path = entry.path();

        if path.is_dir() {
            println!("[DIR] {}", path.display());
        } else {
            println!("[FILE] {}", path.display());
        }
    }
    Ok(())
}
```

`recursive_directory_walk.rs` repeats that loop for every subdirectory:

```
use std::fs;
use std::io;
use std::path::Path;

fn walk(dir: &Path) -> io::Result<()> {
    for entry in fs::read_dir(dir)? {
```

```

    let entry = entry?;
    let path = entry.path();
    let file_type = entry.file_type()?;

    if file_type.is_dir() {
        println!("[DIR] {}", path.display());
        walk(&path)?;
    } else {
        println!("[FILE] {}", path.display());
    }
}
Ok(())
}

fn main() -> io::Result<()> {
    walk(Path::new("."))?;
    Ok(())
}

```

`Path::display` is the conversion for printing. `Path` is not guaranteed to be valid UTF-8, so it does not implement `Display`; `display` returns a wrapper that replaces unrepresentable bytes, which is what a terminal needs.

`DirEntry::file_type` is used in the walk while the single-directory program uses `Path::is_dir`. The two differ for a symbolic link: `file_type` reports the link itself, and `is_dir` follows the link. The walk therefore prints a link to a directory as `[FILE]` and does not descend into it, which is also what keeps a link cycle from producing an unbounded walk.

Intuition

`path_buff.rs` on a Unix host:

```

dir                "/tmp"
dir.join("data.txt")  "/tmp/data.txt"
PathBuf::from(dir)    "/tmp"
push("nested")        "/tmp/nested"
push("file.txt")      "/tmp/nested/file.txt"

file: /tmp/data.txt
buf:  /tmp/nested/file.txt
parent: Some("/tmp/nested")
file_name: Some("file.txt")
ext: Some("txt")

```

`list_directory.rs` in a directory holding `Cargo.toml` and `src`:

```

[FILE] ./Cargo.toml
[DIR]  ./src

```

`recursive_directory_walk.rs` from the same directory:

```
[FILE] ./Cargo.toml
[DIR]  ./src
[FILE] ./src/lib.rs
[FILE] ./src/main.rs
```

Neither listing sorts its output, so the order above is one possible order rather than the order the filesystem guarantees.

Time and Space Complexity

Operation	Cost	Condition
<code>Path::join</code> , <code>PathBuf::push</code>	$O(\text{length})$ time, one allocation for join	the path is copied into the result
<code>read_dir</code> iteration	one directory read plus one <code>DirEntry</code> per entry	the order is unspecified
<code>DirEntry::file_type</code>	no extra metadata call on most platforms	the type comes with the entry
<code>Path::is_dir</code>	one metadata call per call	it resolves the path again
recursive walk	$O(\text{entries})$ time, $O(\text{depth})$ stack	one frame per open directory

Limitations

The walk has no cycle protection beyond not following links. `entry.file_type` does not follow a symbolic link, so a link to a directory is not entered. A bind mount or a hard-linked directory can still present a cycle, and the walk would not terminate.

Depth is bounded by the stack. `walk` calls itself once per directory level, so a tree several thousand levels deep exhausts the stack. An explicit stack of `PathBuf` values removes that limit.

One unreadable directory stops the walk. `walk(&path)?` propagates the error to `main`, so a permission failure at one level abandons the remaining directories. A tool that should continue past such an entry has to collect the error and resume.

The order is unspecified and unsorted. `read_dir` returns entries in the order the filesystem supplies. A reproducible listing has to collect the entries and sort them, which the programs do not do.

Relative names depend on the working directory. Both directory programs walk `"."`, so their output changes with the directory they are started in, and `list_directory` does not recurse.

Summary

- `Path` and `PathBuf` are the borrowed and owned forms of a path, and `join` is the non-mutating builder against `push`.
- The inspection methods return `Option`, because the component asked for may not exist.
- `DirEntry::file_type` avoids a per-entry metadata call, and its refusal to follow symbolic links is what keeps the walk finite.
- The recursive walk trades an explicit stack for call frames, so its depth is limited by the stack.

References

- Standard library, `std::path`.
- Standard library, `PathBuf::push`.
- Standard library, `std::fs::read_dir`.
- Standard library, `DirEntry::file_type`.

File Operations and Errors

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/copy_rename_delete_file.rs` and `src/bin/file_error_handling.rs`. Run them with `cargo run --bin copy_rename_delete_file` and `cargo run --bin file_error_handling`.

Problem Statement

Two programs cover the mutating file operations and the other half of the interface: copy, rename and delete a file, and decide what to report when a read fails.

Designing a Solution

`fs::copy` reads one file and writes another, returning the number of bytes copied. `fs::rename` changes a name, and on one filesystem it is an atomic replacement: the target either has the old contents or the new ones, never a mixture. `fs::remove_file` removes a name for a file, not a directory.

Errors are values. `io::Error` carries an `ErrorKind`, which is the part a caller can branch on without examining the message text. `ErrorKind::NotFound` is the common case for a path that does not exist, and `Err(e) if e.kind() == io::ErrorKind::NotFound` matches it before the general error arm. The guard is the mechanism: the arm is only selected when its condition holds, so the ordering of the arms is explicit rather than incidental.

Implementation

`copy_rename_delete_file.rs` performs the three operations in sequence:

```

use std::fs;
use std::io;

fn main() -> io::Result<()> {
    fs::write("a.txt", "hello")?;

    fs::copy("a.txt", "b.txt")?;
    fs::rename("b.txt", "c.txt")?;
    fs::remove_file("a.txt")?;

    println!("Done");
    Ok(())
}

```

`file_error_handling.rs` turns a failed read into a message that depends on the kind of failure:

```

use std::fs;
use std::io;

fn read_file(path: &str) -> Result<String, io::Error> {
    fs::read_to_string(path)
}

fn main() {
    match read_file("missing.txt") {
        Ok(text) => println!("{}", text),
        Err(e) if e.kind() == io::ErrorKind::NotFound => {
            eprintln!("File not found");
        }
        Err(e) => eprintln!("Error: {}", e),
    }
}

```

Each call in the first program is a fallible operation, and `?` returns early on the first failure. That is the correct default for a sequence in which each step depends on the previous one: copying before the file exists has no meaning.

In the second program the error is matched rather than propagated, because `main` returns `()`. A `main` that returns `Result` instead would let `?` do the work, at the cost of printing the error through the runtime's `Debug` implementation rather than the message chosen here. The `eprintln!` writes to standard error, which keeps diagnostics out of a pipeline that reads standard output.

Intuition

`copy_rename_delete_file.rs` from an empty directory:

statement	files afterwards
<code>fs::write("a.txt", ...)</code>	<code>a.txt</code>
<code>fs::copy("a.txt", "b.txt")</code>	<code>a.txt</code> <code>b.txt</code>

```
fs::rename("b.txt","c.txt") a.txt  c.txt
fs::remove_file("a.txt")    c.txt
println!("Done")           prints Done
```

`file_error_handling.rs` with no `missing.txt`:

```
read_file("missing.txt") Err(kind = NotFound)
  first arm              does not match Ok
  guarded arm            matches, prints "File not found" to stderr
  main                   returns ()
```

Time and Space Complexity

Operation	Cost	Condition
<code>fs::copy</code>	$O(n)$ time and one extra file's worth of writes	n bytes copied; the kernel may choose the buffer size
<code>fs::rename</code>	$O(1)$ time	source and target are on one filesystem
<code>fs::remove_file</code>	$O(1)$ time	the name is unlinked; the data is freed when the last handle closes
<code>ErrorKind</code> match	$O(1)$ time	the kind is a field of the error

Limitations

rename is not portable across filesystems. When the source and target are on different mounts, the operation fails rather than copying, and the error kind is platform specific. Moving between filesystems needs a copy followed by a remove, which is not atomic.

copy does not preserve everything. It writes the file contents and copies the permission bits, but it does not preserve ownership, timestamps, or extended attributes. A tool that needs those has to set them explicitly.

remove_file refuses a directory. A directory needs `remove_dir`, which itself requires the directory to be empty. A recursive delete has to walk the tree and remove children before parents.

A failure in the middle leaves the earlier steps applied. The first program returns as soon as a call fails, so a failure at `rename` leaves `a.txt` and

`b.txt` in place. Nothing here rolls back, and a tool that must leave the directory unchanged needs a temporary directory and a final rename.

The error message in the last arm is the library's. `eprintln!("{}", e)` prints `Display` for `io::Error`, which includes the operating system message but not the path. Adding the path is the caller's job, because the error does not carry it.

Summary

- `copy`, `rename` and `remove_file` are the mutating operations, and `rename` is the atomic replacement primitive within one filesystem.
- Errors are values; `ErrorKind` is the part a program can branch on, and a match guard is how the branch is written.
- `?` is the right default for a sequence of dependent steps, and an explicit match is the right form when the program chooses the message.
- Diagnostics go to standard error, which keeps standard output usable as data.

References

- Standard library, `fs::copy`.
- Standard library, `fs::rename`.
- Standard library, `fs::remove_file`.
- Standard library, `io::ErrorKind`.

Command-Line Arguments

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/command_line_args.rs`. Run it with `cargo run --bin command_line_args -- --name ada --count 3 --verbose`.

Problem Statement

A command-line tool receives its input as a list of strings from the operating system. The program has to know where that list starts and ends, which entry is the name of the program rather than an argument to it, and how to turn the remaining entries into the options the tool accepts. It also has to report a bad command line in a way the shell can see, which means a non-zero exit status.

Designing a Solution

`env::args()` returns an iterator over the arguments as `String`. The first item is the name the program was invoked with, so the arguments proper begin at index one and `args().skip(1)` is the boundary between the two.

The program parses three options. `--name` takes a value, `--count` takes a number, and `--verbose` is a flag. The parser walks the arguments once with an iterator, so a value option consumes the next item and a flag consumes only itself. It returns `Result<Options, String>`: every failure is a message that names the argument at fault, and `main` prints that message and a usage line to standard error before returning `ExitCode::FAILURE`. A successful run returns `ExitCode::SUCCESS`, so the exit status is a fact the shell can test.

`main` returns `ExitCode` rather than `()`. The return value is the program's report to its caller, and the shell reads the status byte and branches on it. Returning

`ExitCode` keeps that decision in `main` instead of in a call to `std::process::exit`, which would skip the destructors of everything still alive.

The iterator has one failure mode worth knowing. `args()` panics when an argument is not valid UTF-8. `args_os` yields `OsString` instead and never panics, which is what a tool that forwards an arbitrary path should use.

Implementation

```
use std::env::args;
use std::process::ExitCode;

/// The options this program understands.
#[derive(Debug, PartialEq)]
struct Options {
    name: String,
    count: u32,
    verbose: bool,
}

impl Default for Options {
    fn default() -> Self {
        Self { name: String::from("world"), count: 1, verbose: false }
    }
}

/// Turn the arguments after the program name into `Options`.
fn parse(arguments: impl IntoIterator<Item = String>) -> Result<Options, String> {
    let mut options = Options::default();
    let mut arguments = arguments.into_iter();

    while let Some(argument) = arguments.next() {
        match argument.as_str() {
            "--name" => {
                options.name = arguments.next().ok_or("--name needs a value")?;
            }
            "--count" => {
                let value = arguments.next().ok_or("--count needs a value")?;
                options.count = value
                    .parse()
                    .map_err(|_| format!("--count needs a number, got {value:?}"))?;
            }
            "--verbose" => options.verbose = true,
            other => return Err(format!("unknown argument {other:?}")),
        }
    }

    Ok(options)
}

fn main() -> ExitCode {
    let options = match parse(args().skip(1)) {
        Ok(options) => options,
        Err(message) => {
            eprintln!("error: {message}");
            eprintln!("usage: command_line_args [--name NAME] [--count N] [--verbose]");
            return ExitCode::FAILURE;
        }
    };
};
```

```

    for index in 0..options.count {
        if options.verbose {
            println!("[{index}] hello, {}", options.name);
        } else {
            println!("hello, {}", options.name);
        }
    }

    ExitCode::SUCCESS
}

#[cfg(test)]
mod tests {
    use super::*;

    fn parse_args(arguments: &[&str]) -> Result<Options, String> {
        parse(arguments.iter().map(|value| value.to_string()))
    }

    #[test]
    fn defaults_apply_when_there_are_no_arguments() {
        assert_eq!(parse_args(&[]).unwrap(), Options::default());
    }

    #[test]
    fn a_value_option_consumes_the_next_argument() {
        let options = parse_args(&["--name", "ada", "--count", "3"]).unwrap();
        assert_eq!(options.name, "ada");
        assert_eq!(options.count, 3);
    }

    #[test]
    fn verbose_is_a_flag() {
        assert!(parse_args(&["--verbose"]).unwrap().verbose);
        assert!(!parse_args(&[]).unwrap().verbose);
    }

    #[test]
    fn a_value_option_without_a_value_is_an_error() {
        assert!(parse_args(&["--name"]).is_err());
        assert!(parse_args(&["--count"]).is_err());
    }

    #[test]
    fn a_non_numeric_count_is_an_error() {
        assert!(parse_args(&["--count", "many"]).is_err());
    }

    #[test]
    fn an_unknown_argument_is_an_error() {
        assert!(parse_args(&["--colour", "red"]).is_err());
        assert!(parse_args(&["extra"]).is_err());
    }
}

```

The listing is the whole file. Taken in order:

- `parse` accepts any iterator of `String`, so the tests can pass a slice and `main` can pass `args().skip(1)`. The argument is not collected into a `Vec` first, so a value is consumed exactly when the parser reaches it.
- `--name` and `--count` both call `arguments.next()`. When the iterator is exhausted they receive `None`, and `ok_or` turns that into the message `--name`

needs a value. A flag that silently accepted a missing value would hide a typo.

- `--count` parses with `value.parse()`. `map_err` replaces the parser's error with one that quotes the offending text, because the caller needs to know which word was not a number.
- The final arm catches every other argument, including a stray positional one, and returns an error. An unknown option is a failure rather than something ignored, so a misspelled flag does not silently do nothing.
- `main` returns `ExitCode`. On success the loop prints `count` greetings, and `--verbose` prefixes each line with its index.

The `Default` implementation gives the program the values it uses when an option is absent. Defaults live in one place, so the parser only writes a field when the command line supplies one.

Intuition

With no arguments, the defaults apply:

```
$ cargo run --bin command_line_args
hello, world
```

With the options:

```
$ cargo run --bin command_line_args -- --name ada --count 3 --verbose
[0] hello, ada
[1] hello, ada
[2] hello, ada
```

A bad command line goes to standard error and sets the exit status:

```
$ cargo run --bin command_line_args -- --count many
error: --count needs a number, got "many"
usage: command_line_args [--name NAME] [--count N] [--verbose]
$ echo $?
1
```

`cargo run` passes everything after a bare `--` to the program, which is why the examples use it. Without the separator, `cargo` would treat `--name` as one of its own flags.

The process receives its arguments after the shell has removed quoting and performed expansion. `--name "two words"` arrives as one element, and `*` ar-

rives already expanded by the shell, so the program never sees the command line as a single string.

Time and Space Complexity

Operation	Cost	Condition
<code>env::args()</code>	no allocation until an item is taken	each item is a <code>String</code> when produced
parse	$O(n)$ time	one pass, one <code>next()</code> per option and value
<code>skip(1)</code>	$O(1)$	it drops the first item of the iterator
memory	$O(1)$ beyond the strings already produced	the parser keeps no list of its own

Collecting into a `Vec<String>` would cost one allocation per argument plus the vector, and it would hold the whole command line at once. The parser avoids both, because it only ever needs the current argument and, for a value option, the one after it.

Limitations

Only the separated form is accepted. `--name ada` is an option and its value; `--name=ada` is an unknown argument, because the match is on the whole word. Accepting the second form means splitting each argument at its first `=` before the match.

There is no `--` separator of its own. The program does not treat a bare `--` as the end of the options, so a value that begins with `--` cannot be passed as a name except by changing the parser.

There is no help flag. `--help` is reported as an unknown argument. A tool that ships to other people usually answers `--help` with the usage line and the exit status for success, rather than with the failure path.

`args()` panics on a non-UTF-8 argument. On Unix an argument is a byte string, and a file name that is not valid UTF-8 reaches the program as

bytes. A program that must accept such a name has to use `args_os` and handle `OsString`.

The program name is not the same as the executable. The first argument is whatever the caller passed as `argv[0]`, which is the path used to invoke the program and can be changed by the caller. It is not a reliable source of the program's location.

Precedence between sources is not implemented. A flag over an environment variable over a file over a default is the usual order, and a tool that reads more than one source has to state that order somewhere. This parser has one source.

Summary

- `env::args()` yields `String` values whose first item is the invocation name; `skip(1)` is the boundary between the program and its arguments.
- The parser walks the arguments once with an iterator, so a value option consumes the next item and a flag consumes only itself.
- Every failure returns a message that names the argument at fault, and `main` turns it into standard error plus `ExitCode::FAILURE`.
- `args()` panics on non-UTF-8 input, and `args_os` is the version that does not.

References

- Standard library, `std::env::args`.
- Standard library, `std::env::args_os`.
- Standard library, `std::process::ExitCode`.
- The Rust Book, [Accepting command line arguments](#).

Part VIII: Coordination and Parallelism

A Bounded Buffer

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/bounded_buffer.rs`. Run it with `cargo run --bin bounded_buffer`.

Problem Statement

Build a queue with a fixed capacity that several threads share: producers must wait while the queue is full, and consumers must wait while it is empty. No thread may spin on a flag, and no item may be lost or handed out twice.

Designing a Solution

A `Mutex<VecDeque<T>>` makes the deque safe to share, but a lock alone cannot express waiting. A consumer that finds the queue empty has to release the lock and sleep, and it has to be woken when an item arrives. That is what a condition variable is for.

The queue uses two condition variables. `not_empty` is the one consumers wait on and producers signal after a push; `not_full` is the one producers wait on and consumers signal after a pop. With a single variable, every waiter would be woken by every change, and the threads of the wrong kind would find their own condition still false and go back to sleep.

`Condvar::wait` takes the guard by value. It releases the lock, parks the thread, and returns a new guard once the lock has been reacquired. That reacquisition is why the condition has to be checked in a `while` loop rather than an `if`: the thread can wake without a signal, and another thread can take the slot between the wake-up and the reacquisition. The `while` re-checks the condition

with the lock held, so the thread only proceeds when the state is what it observed.

`notify_one` is used rather than `notify_all`. Each signal follows exactly one state change, one item added or one slot freed, so exactly one waiter can make progress. Waking every waiter would produce a crowd that immediately re-checks the condition and sleeps again.

Implementation

```
use std::collections::VecDeque;
use std::sync::{Arc, Condvar, Mutex};
use std::thread;
use std::time::Duration;

// A thread-safe bounded buffer (a "shelf" that holds at most `capacity` items).
//
// Three fields do all the work:
// - `inner` : the actual VecDeque, wrapped in a Mutex so only one thread
//             touches it at a time.
// - `not_empty` : waiting room for CONSUMERS. A consumer sleeps here when the
//               shelf is empty. Producers ring this bell after pushing.
// - `not_full` : waiting room for PRODUCERS. A producer sleeps here when the
//               shelf is full. Consumers ring this bell after popping.
//
// Two separate condvars so we always wake the RIGHT kind of thread.
// One shared condvar would cause livelock: consumers waking consumers,
// producers waking producers, everyone re-checking and going back to sleep.
struct BoundedQueue<T> {
    inner: Mutex<VecDeque<T>>,
    capacity: usize,
    not_empty: Condvar,
    not_full: Condvar,
}

impl<T> BoundedQueue<T> {
    fn new(capacity: usize) -> Self {
        // Capacity 0 makes no sense: no producer could ever push,
        // and every pop would block forever.
        assert!(capacity > 0, "capacity must be > 0");

        Self {
            inner: Mutex::new(VecDeque::with_capacity(capacity)),
            capacity,
            not_empty: Condvar::new(),
            not_full: Condvar::new(),
        }
    }

    fn push(&self, item: T) {
        // Acquire the shelf lock. `guard` is our exclusive handle to the deque.
        let mut guard = self.inner.lock().unwrap();

        // Wait while the shelf is full.
        //
        // Why `while` and not `if`:
        // 1. Condvars are allowed to wake up for no reason (spurious wakeup).
        // 2. Even on a real wakeup, another producer may have raced us and
        //    taken the free slot before we reacquired the lock.
        // So we must re-check the condition every time we wake.
```

```

//
// Why `guard = ...wait(guard)` :
//   `wait` consumes the guard (it releases the lock on our behalf),
//   puts us to sleep, then re-locks and hands us a NEW guard.
//   The reassignment just means "here's my fresh key to the shelf."
//   The VecDeque inside is the same one – we're not replacing the queue.
while guard.len() == self.capacity {
    guard = self.not_full.wait(guard).unwrap();
}

// We have room. Add the item to the back of the deque.
guard.push_back(item);

// Release the lock BEFORE ringing the bell.
// This is an optimization, not a correctness requirement.
// If we notify while still holding the lock, the thread we wake will
// immediately block on `lock()` until we release it – a wasted wakeup.
drop(guard);

// Ring the "not empty" bell: one consumer can now make progress.
// `notify_one` (not `notify_all`) because exactly ONE item was added,
// so only ONE consumer can succeed. Waking all of them = stampede.
self.not_empty.notify_one();
}

fn pop(&self) -> T {
    // Acquire the shelf lock.
    let mut guard = self.inner.lock().unwrap();

    // Wait while the shelf is empty.
    // Same `while` reasoning as push: spurious wakeups + races with other
    // consumers who may have grabbed the item before we reacquired the lock.
    while guard.is_empty() {
        guard = self.not_empty.wait(guard).unwrap();
    }

    // Safe to unwrap: the `while` loop guarantees the deque is non-empty,
    // and we hold the lock so nobody can change that between the check
    // and this line.
    let item = guard.pop_front().unwrap();

    // Release before notifying (same optimization as in push).
    drop(guard);

    // Ring the "not full" bell: one producer can now make progress.
    // Exactly one slot opened, so wake exactly one producer.
    self.not_full.notify_one();

    item
}

fn main() {
    // Small capacity (3) so that with two slow consumers you can actually
    // SEE producers block: the queue fills up and push() sleeps in `not_full`.
    let queue = Arc::new(BoundedQueue::new(3));

    let mut handles = vec![];

    // ---- Producers ----
    // Two producers, each pushes 5 items. `Arc::clone` bumps the refcount so
    // each thread shares the same queue; ownership of the clone moves in.
    for p in 0..2 {
        let q = Arc::clone(&queue);
        handles.push(thread::spawn(move || {
            for i in 0..5 {
                let item = format!("p{:}:item{}", p, i);

```

```

println!("[producer {}] pushing {}", p, item);
q.push(item);
// Slow down so we don't blast through the whole buffer instantly.
thread::sleep(Duration::from_millis(30));
    }
  }));
}

// ---- Consumers ----
// Two consumers, each pops 5 items. Consumers are SLOWER than producers
// here (60ms vs 30ms), so the queue fills up and you'll see producers
// stall on `not_full.wait(...)`.
for c in 0..2 {
  let q = Arc::clone(&queue);
  handles.push(thread::spawn(move || {
    for _ in 0..5 {
      let item = q.pop();
      println!("[consumer {}] got {}", c, item);
      thread::sleep(Duration::from_millis(60));
    }
  }));
}

// Join everything. If our synchronization is wrong, this is where you'd
// hang (a thread sleeping forever) or panic (unwrap on empty / on a
// poisoned mutex after another thread panicked while holding it).
for h in handles {
  h.join().unwrap();
}
println!("done");
}

```

The lock is released with `drop(guard)` before `notify_one`. Holding the lock while signalling does not lose the wake-up, but the thread that is woken immediately blocks on `lock` until the signaller releases it, so the signal costs a context switch that accomplishes nothing.

`Arc` is what lets two threads own the same queue. The clone is passed to the thread and the original stays in `main`; the queue is freed when the last handle is dropped.

Intuition

With `BoundedQueue::new(1)` and one producer and one consumer, the state after each operation is:

step	queue	producer	consumer
push(1)	[1]	running	
pop()	[]	running	takes 1
push(2)	[2]	running	
pop()	[]	running	takes 2
if the consumer starts first:			
pop()	[]	-	waits on <code>not_empty</code>
push(1)	[1]	running	wakes, takes 1

The consumer that finds the queue empty parks on `not_empty` and holds no lock while parked, so the producer can take the lock and push.

Time and Space Complexity

Operation	Cost	Condition
push, pop	$O(1)$	one lock, at most one signal
block and wake	one context switch	the lock is released while the thread is parked
memory	one deque at capacity, plus one word per blocked thread	the blocked threads are in the kernel, not in the queue
memory	$O(n)$ per <code>VecDeque</code>	the buffer is sized at construction and does not grow

The mutex is held for the length of the critical section, which is a deque operation and a length check. No work that can block is performed under the lock, which is what keeps the queue from serialising its callers.

Limitations

There is no way to close the queue. A consumer that calls `pop` after the producers have finished waits forever, and with no `close` method there is no state that would release it. A production queue needs a shutdown flag, checked inside the same loop, and a `notify_all` when it is set.

Poisoning is handled with `unwrap`. A thread that panics while holding the lock marks the mutex poisoned, and every later `lock()` returns `Err`. The `unwrap` turns that into a second panic rather than a recoverable error, which is acceptable for a demonstration and not for a library.

There is no fairness guarantee. `notify_one` may wake any waiter, so a pushed item can be taken by a later arrival while an earlier consumer is still parked. A fair queue needs an ordered wait list, which this one does not provide.

The program's output is interleaved. The print statements run outside the lock, so the two producers can print in either order and the lines from different threads can be adjacent. The set of lines is fixed; the order is not.

The file has no tests. Correctness here is a property of the interleaving, and the tests that would catch a missing `while` are the ones that run a producer and a consumer concurrently rather than the ones that assert on a single-threaded push and pop.

Summary

- A mutex makes the deque shared; condition variables make waiting possible.
- The condition is re-checked in a `while` loop, because a wake-up does not prove the condition is true.
- Two condition variables keep each signal directed at a thread that can act on it; `notify_one` follows each single state change.
- The queue has no shutdown path, and a consumer that outlives the producers waits indefinitely.

References

- Standard library, `std::sync::Condvar`.
- Standard library, `Condvar::wait`.
- Standard library, `std::sync::Mutex`.
- The Rust Book, [Shared-state concurrency](#).

Parallel Sum

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/parallel_sum.rs`. Run it with `cargo run --release --bin parallel_sum`, optionally followed by the thread count and the element count.

Problem Statement

Sum a large array twice, once on one thread and once on several, and check that the two results agree. The program is a measurement as much as a demonstration: the same work is done with three element types so that the effect of the element size on the parallel speedup is visible.

Designing a Solution

`thread::scope` creates a scope that waits for every thread spawned inside it, and it lets those threads borrow values from the surrounding stack. That is why the chunks can be `&[T]` slices of one array rather than `Arc<Vec<T>>` clones: the scope's lifetime covers the borrow, so no `'static` bound is needed.

The work is divided by position, not by value. Each thread receives an index, computes the half-open range `[start, end)`, and sums that slice. The last thread takes everything after its start, which covers the array when its length is not a multiple of the thread count. The partial sums are then reduced in order with `sum()`.

The result is order-independent because integer addition is associative here. The same structure with floating-point addition would not be bit-for-bit reproducible, because the grouping would change the rounding.

Implementation

```

use std::env;
use std::thread;
use std::time::{Duration, Instant};

const SIZE: usize = 1_000_000_000;

fn seq_i32(data: &[i32]) -> (i32, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i32>();
    (sum, t.elapsed())
}

fn par_i32(data: &[i32], n: usize) -> (i32, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i32>()
                })
            })
            .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i32>()
    });
    (sum, t.elapsed())
}

fn seq_i64(data: &[i64]) -> (i64, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i64>();
    (sum, t.elapsed())
}

fn par_i64(data: &[i64], n: usize) -> (i64, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {
        let handles: Vec<_> = (0..n)
            .map(|i| {
                s.spawn(move || {
                    let chunk = data.len() / n;
                    let start = i * chunk;
                    let end = if i == n - 1 { data.len() } else { start + chunk };
                    data[start..end].iter().sum::<i64>()
                })
            })
            .collect();
        handles.into_iter().map(|h| h.join().unwrap()).sum::<i64>()
    });
    (sum, t.elapsed())
}

fn seq_i128(data: &[i128]) -> (i128, Duration) {
    let t = Instant::now();
    let sum = data.iter().sum::<i128>();
    (sum, t.elapsed())
}

fn par_i128(data: &[i128], n: usize) -> (i128, Duration) {
    let t = Instant::now();
    let sum = thread::scope(|s| {

```

```

    let handles: Vec<_> = (0..n)
      .map(|i| {
        s.spawn(move || {
          let chunk = data.len() / n;
          let start = i * chunk;
          let end = if i == n - 1 { data.len() } else { start + chunk };
          data[start..end].iter().sum::<i128>()
        })
      })
      .collect();
    handles.into_iter().map(|h| h.join().unwrap()).sum::<i128>()
  });
  (sum, t.elapsed())
}

fn main() {
  let n: usize = env::args()
    .nth(1)
    .and_then(|s| s.parse().ok())
    .unwrap_or(8);

  {
    let data: Vec<i32> = vec![1; SIZE];
    let (s1, t1) = seq_i32(&data);
    let (s2, t2) = par_i32(&data, n);
    assert_eq!(s1, s2);
    println!("i32  seq sum = {s1}, elapsed = {t1:?}");
    println!("i32  par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }

  {
    let data: Vec<i64> = vec![1; SIZE];
    let (s1, t1) = seq_i64(&data);
    let (s2, t2) = par_i64(&data, n);
    assert_eq!(s1, s2);
    println!("i64  seq sum = {s1}, elapsed = {t1:?}");
    println!("i64  par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }

  {
    let data: Vec<i128> = vec![1; SIZE];
    let (s1, t1) = seq_i128(&data);
    let (s2, t2) = par_i128(&data, n);
    assert_eq!(s1, s2);
    println!("i128 seq sum = {s1}, elapsed = {t1:?}");
    println!("i128 par sum = {s2}, elapsed = {t2:?}  (n = {n})");
  }
}

```

`assert_eq!(s1, s2)` is the correctness check. It is what makes the program a test as well as a demonstration: if the chunk boundaries overlapped or left a gap, the two sums would differ.

The three blocks are separate so that each array is dropped before the next is allocated. `SIZE` is one billion, so the `i128` array alone is sixteen gigabytes; the blocks keep the peak at one array rather than three.

Intuition

With `n = 4` and `data.len() = 10`, the ranges each thread receives are:

thread	chunk	start	end	elements	
0	2	0	2	2	
1	2	2	4	2	
2	2	4	6	2	
3	2	6	10	4	(the last thread takes the remainder)

partial sums	a	b	c	d
reduction	((a + b) + c) + d			

The four sub-slices partition `0..10`, and the last range is longer because it absorbs the remainder.

Time and Space Complexity

Re-source	Cost	Condition
Time	$O(n)$ additions	independent of the thread count
Threads	$O(\text{threads})$ stacks	each thread has its own stack
Memory	$O(n)$ elements in the array, $O(\text{threads})$ for the partial sums	the input is not partitioned
Speedup	at most $\min(\text{threads}, \text{cores})$	the reduction adds a small serial tail

The sequential sum is memory-bound: each element is read once and nothing else is done with it. Adding threads divides the arithmetic, but not the memory traffic, so the speedup saturates once the bus or the cache hierarchy is saturated. The three element types are in the file to make that visible: the `i128` pass does more arithmetic per byte than the `i32` pass, so it scales differently even on the same machine.

Limitations

The default size needs a large machine. One billion elements is four gigabytes for `i32`, eight for `i64`, and sixteen for `i128`. The program was

written for a workstation and it will exhaust memory on a smaller host; the size is not a parameter, only the thread count is.

The timing has no warm-up and one pass. `Instant::now()` is called once per function, so the first touch of each page and any frequency scaling are inside the measurement. A measurement intended to be quoted needs a warm-up and several repetitions, as the benchmark harness in this repository does.

More threads than cores do not help. The thread count defaults to eight and is taken from the first argument. Nothing reads the available parallelism, so on a smaller machine the extra threads contend for cores rather than adding work.

The chunking is unbalanced. `data.len() / n` is the same for every thread except the last, which takes the remainder. When `n` is close to the length, the last thread does most of the work. A balanced split would distribute the remainder one element at a time.

The array is not partitioned or aligned for the access pattern. Each thread reads a contiguous range, which is the friendly case. A false-sharing problem would appear if threads wrote partial sums into adjacent fields of one object, which this program avoids by returning each sum through `join`.

Summary

- `thread::scope` allows the workers to borrow the input, which removes the `Arc<Vec<T>>` the naive version would need.
- Dividing by index gives each thread a disjoint half-open range, and the last range absorbs the remainder so that the whole array is covered.
- The correctness check is the equality of the two sums; a boundary error makes the program fail rather than print a wrong number.
- The measured speedup is bounded by memory bandwidth, which is why the element type is part of the experiment.

References

- Standard library, `thread::scope`.

- Standard library, `Instant`.
- Standard library, `Iterator::sum`.

Part IX: Sequences, Re-visited

Running Median

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/median_finder.rs`. Run it with `cargo run --bin median_finder`.

Problem Statement

Accept a stream of numbers one at a time and report the median of everything seen so far after each insertion. The median is the middle value when the count is odd and the average of the two middle values when it is even. Resorting the values on every query is the obvious answer and the wrong one; the type has to maintain the median as the stream arrives.

Designing a Solution

Two heaps split the values into a lower half and an upper half. `low` is a `BinaryHeap<i32>`, which is a max-heap, so its top is the largest value in the lower half. `high` is a `BinaryHeap<Reverse<i32>>`, and `Reverse` flips the ordering, so its top is the smallest value in the upper half. `BinaryHeap` is already a priority queue; `Reverse` is the wrapper that gives the second one the opposite ordering.

Two invariants keep the halves together:

- every value in `low` is less than or equal to every value in `high`;
- `low.len()` equals `high.len()`, or exceeds it by one.

If both hold, the median is `low.peek()` when `low` is longer, and the average of the two tops otherwise. The insertion maintains them in three steps: push onto

low, move low's maximum to high so the halves stay ordered, and move high's minimum back if high has grown too large.

Implementation

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

#[derive(Default)]
pub struct MedianFinder {
    low: BinaryHeap<i32>,
    high: BinaryHeap<Reverse<i32>>,
}

impl MedianFinder {
    pub fn new() -> Self {
        Self::default()
    }

    pub fn add_num(&mut self, num: i32) {
        self.low.push(num);

        if let Some(val) = self.low.pop() {
            self.high.push(Reverse(val));
        }

        if self.high.len() > self.low.len() {
            if let Some(Reverse(val)) = self.high.pop() {
                self.low.push(val);
            }
        }
    }

    pub fn find_median(&self) -> Option<f64> {
        let max_low = *self.low.peek()?;

        if self.low.len() > self.high.len() {
            Some(max_low as f64)
        } else {
            let Reverse(min_high) = *self.high.peek()?;
            Some((max_low as f64 + min_high as f64) / 2.0)
        }
    }
}

const SAMPLE: [i32; 11] = [6, 10, 2, 6, 5, 0, 6, 3, 1, 0, 0];

fn main() {
    let mut finder = MedianFinder::new();

    for value in SAMPLE {
        finder.add_num(value);
        match finder.find_median() {
            Some(median) => println!("after {value:>2}: median {median}"),
            None => println!("after {value:>2}: no median"),
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;
```

```

/// The median of a sorted slice, computed directly.
fn median_of_sorted(values: &[i32]) -> f64 {
    let middle = values.len() / 2;
    if values.len() % 2 == 1 {
        values[middle] as f64
    } else {
        (values[middle - 1] as f64 + values[middle] as f64) / 2.0
    }
}

#[test]
fn an_empty_finder_has_no_median() {
    assert_eq!(MedianFinder::new().find_median(), None);
}

#[test]
fn one_value_is_the_median() {
    let mut finder = MedianFinder::new();
    finder.add_num(7);
    assert_eq!(finder.find_median(), Some(7.0));
}

#[test]
fn an_even_count_averages_the_middle_two() {
    let mut finder = MedianFinder::new();
    finder.add_num(1);
    finder.add_num(2);
    assert_eq!(finder.find_median(), Some(1.5));
}

#[test]
fn three_values_are_ordered() {
    let mut finder = MedianFinder::new();
    for (value, median) in [1, 2, 3].into_iter().zip([1.0, 1.5, 2.0]) {
        finder.add_num(value);
        assert_eq!(finder.find_median(), Some(median));
    }
}

#[test]
fn duplicates_are_kept() {
    let mut finder = MedianFinder::new();
    for value in [4, 4, 4, 4] {
        finder.add_num(value);
    }
    assert_eq!(finder.find_median(), Some(4.0));
}

#[test]
fn the_stream_matches_a_sort_on_every_step() {
    let mut finder = MedianFinder::new();
    let mut seen = Vec::new();

    for value in SAMPLE {
        finder.add_num(value);
        seen.push(value);
        seen.sort_unstable();

        assert_eq!(finder.find_median(), Some(median_of_sorted(&seen)));
    }
}

#[test]
fn the_two_heaps_never_differ_by_more_than_one() {
    let mut finder = MedianFinder::new();

    for value in SAMPLE {

```

```

        finder.add_num(value);
        assert!(finder.low.len().abs_diff(finder.high.len()) <= 1);
    }
}

```

The listing is the whole file. Taken in order:

- `add_num` pushes onto `low` before rebalancing. That order means the new value is considered for the lower half first, and the move of `low`'s maximum to `high` then restores the ordering if the value belongs above the median. The second `if` runs only when `high` became longer, which moves one value back so `low` is never the shorter heap.
- `find_median` returns `Option<f64>`. `self.low.peek()? returns None for an empty finder, so find_median reports absence rather than dividing by zero. When low is longer, low's maximum is the median; otherwise the two tops are averaged.`
- The median is returned as `f64` even when it is a whole number, because the two middle values can be a half-integer. Casting `i32` to `f64` is exact for every `i32`, so no precision is lost on the way in.
- `main` drives the type with the `SAMPLE` stream and prints the median after each insertion, which is the behaviour the chapter claims in one run.
- The tests check the empty case, one value, an even count, the ordered sequence, a repeated value, and the whole `SAMPLE` stream against a sort of the values seen so far. That last test is a differential check: it computes the median a second, slower way and requires the heaps to agree with it on every step. A further test asserts the balancing invariant itself, that the two lengths never differ by more than one.

Intuition

Insert the values 1, then 2, then 3:

step	low (max-heap)	high (min-heap)	median
start	[]	[]	None
add_num(1)	[1]	[]	1.0
add_num(2)	[1]	[2]	1.5
add_num(3)	[2, 1]	[3]	2.0

The third step in detail. 3 goes onto `low`, so `low` is `[3, 1]`. Its maximum, 3, moves to `high`, which becomes `[2, 3]` read as a min-heap: the top is 2. `high` is now longer than `low`, so its minimum, 2, moves back and `low` becomes `[2, 1]`. The median is `low`'s top, 2.0.

The `SAMPLE` stream in the program produces:

```
after 6: median 6
after 10: median 8
after 2: median 6
after 6: median 6
after 5: median 6
after 0: median 5.5
after 6: median 6
after 3: median 5.5
after 1: median 5
after 0: median 4
after 0: median 3
```

Each insertion moves at most two values between the heaps, so the two lengths never differ by more than one and the median is always one or two heap tops away.

Time and Space Complexity

Operation	Cost	Condition
<code>add_num</code>	$O(\log n)$	two heap pushes and at most two pops
<code>find_median</code>	$O(1)$	the median is one or two heap tops
memory	$O(n)$	every value is stored once

The alternative, keeping a sorted `Vec` and re-sorting on each query, costs $O(n \log n)$ per insertion. Sorting once per query and caching the result trades that for $O(1)$ queries but still pays $O(n \log n)$ when the stream continues.

Limitations

The type is fixed to `i32`. The heaps are `BinaryHeap<i32>`, so the caller cannot use `i64` or `f64`. Making the type generic needs an ordering bound plus a way to average two values, which is more than `Ord` provides.

A median is returned as `f64`, which changes the type of the answer. For `i32` inputs the conversion and the average are exact, but a caller that needs an exact fraction of two integers has to compute it from the two tops rather than from the returned value.

There is no removal. The structure answers the median of everything seen. A sliding-window median needs a structure that can remove the oldest value, which the two heaps cannot do without locating that value in one of them.

The tests cover one stream in depth and a few cases by hand. `SAMPLE` is eleven values. The differential test compares against a sort, so it catches a rebalancing error on those eleven steps, but a random or exhaustive stream would cover more of the state space than a fixed array does.

Summary

- Two heaps hold the lower and upper halves, and `Reverse` turns the second into a min-heap.
- The three-step insertion keeps the halves ordered and balanced, which is what makes the median a constant-time query.
- `Option<f64>` states that an empty finder has no median, instead of returning a sentinel.
- The program prints the median after every value of a sample stream, and the tests check the result against a sort of the values seen so far.

References

- Standard library, `BinaryHeap`.
- Standard library, `std::cmp::Reverse`.
- Standard library, `Option`.

Reversing a String In Place

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source file: `src/bin/reverse_string.rs`. Run it with `cargo run --bin reverse_string`.

Problem Statement

Reverse the characters of a `String` without allocating a second one. The constraint is what makes the problem interesting in Rust: a `String` is UTF-8, so its bytes are not the same as its characters, and the safe API does not expose a mutable byte buffer.

Designing a Solution

The function works on bytes and swaps from the two ends inward. Two indices start at the first and last byte, exchange their values, and move toward each other until they meet. Each swap fixes two positions, so the loop runs half as many times as there are bytes and requires no extra storage.

Reversing bytes only produces a valid `String` when every byte is a complete character. For ASCII that is true, because one byte is one code point. For general UTF-8 it is not: a multi-byte character read backwards is not a character. The function therefore asserts that the input is ASCII before it touches the buffer, and the assertion is the precondition that makes the unsafe access sound.

`String::as_mut_vec` is the only way to reach the bytes. It is `unsafe` because a caller that leaves invalid UTF-8 in the buffer creates a `String` that violates its own invariant, and every later operation on it is undefined. The `SAFETY` comment states the obligation and the assertion that discharges it.

Implementation

```
fn reverse_str(s: &mut String) {
    // SAFETY: s.as_mut_vec() is unsafe because it exposes the inner mutability to
    // its character buffer, and we must ensure
    // to leave valid UTF-8. We enforce ASCII only characters.
    assert!(s.is_ascii());
    let bytes = unsafe { s.as_mut_vec() };

    let (mut start, mut end) = (0, bytes.len() - 1);

    while start < end {
        bytes.swap(start, end);
        start+=1; end-=1;
    }
}

fn main() {
    for value in ["abcd", "abc", "a", "rust"] {
        let mut s = String::from(value);
        reverse_str(&mut s);
        println!("{value:>6} -> {s}");
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn an_even_length_string_is_reversed() {
        let mut s = String::from("abcd");
        reverse_str(&mut s);
        assert_eq!(s, "dcba");
    }

    #[test]
    fn an_odd_length_string_keeps_its_middle_byte() {
        let mut s = String::from("abc");
        reverse_str(&mut s);
        assert_eq!(s, "cba");
    }

    #[test]
    fn a_single_byte_is_unchanged() {
        let mut s = String::from("a");
        reverse_str(&mut s);
        assert_eq!(s, "a");
    }

    #[test]
    fn the_buffer_is_reused() {
        let mut s = String::from("abcd");
        let capacity = s.capacity();
        let address = s.as_ptr();
        reverse_str(&mut s);
        assert_eq!(s.capacity(), capacity);
        assert_eq!(s.as_ptr(), address);
    }

    #[test]
    fn reversing_twice_restores_the_string() {
        let mut s = String::from("interview");
        reverse_str(&mut s);
        reverse_str(&mut s);
    }
}
```

```

    assert_eq!(s, "interview");
}

#[test]
#[should_panic]
fn non_ascii_input_panics() {
    let mut s = String::from("café");
    reverse_str(&mut s);
}
}

```

The listing is the whole file. Taken in order:

- `assert!(s.is_ascii())` walks the string before the reversal. It is $O(n)$, which does not change the overall bound, and it runs before the unsafe block rather than inside it, so the panic leaves the string untouched.
- `bytes.swap(start, end)` is `slice::swap`, which exchanges two elements without moving either out. A manual exchange through a temporary would be three assignments on `u8`; `swap` states the intent and compiles to the same place.
- `main` reverses four strings and prints each result, so one run shows the even case, the odd case, the single byte, and a second four-letter word.
- The tests cover both parities, the single byte, the in-place property, and the round trip. `the_buffer_is_reused` reads the capacity and the address before and after and requires both to be unchanged, which is what separates this function from one that builds a new `String`. `non_ascii_input_panics` is marked `#[should_panic]`, so the assertion is checked rather than assumed.

Intuition

Reverse "abcd":

```

before  a  b  c  d
         ^      ^
         start  end

swap 0,3 b  a  c  d
swap 1,2 b  c  a  d
         start >= end, stop
after   b  c  a  d

```

Reverse "abc":

```

before  a  b  c
swap 0,2 c  b  a
         start = 1, end = 1, stop
after   c  b  a

```

The middle byte of an odd-length string is never moved, which is why the loop condition is `start < end` rather than `start <= end`.

The program prints:

```
abcd -> dcba
  abc -> cba
    a -> a
  rust -> tsur
```

The buffer is the same allocation in every line. Only the bytes inside it move.

Time and Space Complexity

Resource	Cost	Condition
Time	$O(n)$	<code>is_ascii</code> reads every byte; the swaps touch half of them
Space	$O(1)$	the swap is in place; no second buffer
Allocations	zero	the <code>String</code> keeps its buffer

The reversal itself is $n / 2$ swaps, and it is performed in place, so the buffer's address and capacity do not change. The assertion adds a second pass over the bytes, which is the price of the safe wrapper around the unsafe block.

Limitations

Non-ASCII input panics. `assert!(s.is_ascii())` refuses any string containing a byte above `0x7F`, which includes every accented letter and every emoji. A general reversal needs to decode the string into characters, reverse the characters, and rebuild the `String`, which allocates.

Empty input panics. With `bytes.len() == 0`, the expression `bytes.len() - 1` underflows. In a debug build that panics with an arithmetic error; in a release build it wraps and the following `swap` is out of bounds. The function needs a check for an empty string, and it does not have one, so the tests do not cover the empty case either.

The `unsafe` block is sound only because of the assertion above it. Removing the assertion makes the function undefined behaviour on any non-

ASCII input. The `SAFETY` comment is the record of that dependency, and a future edit that drops the check has to drop the unsafe approach as well.

The function reverses bytes, not grapheme clusters. For ASCII the two are the same. For text that passes the assertion they cannot differ, but a reader should not carry the assumption to a general string.

The largest test input is nine bytes. The tests establish the two parities and the round trip. A larger input, or one built to exercise the exact midpoint of an even-length string, would test the loop bound more of the way.

Summary

- Byte reversal is `O(n)` time and `O(1)` space, and only the ASCII precondition makes it a valid `String` transformation.
- `as_mut_vec` is `unsafe` because it can leave the buffer invalid; the assertion ahead of it is what discharges the obligation recorded in the `SAFETY` comment.
- The loop runs `n / 2` times and leaves the middle byte of an odd-length string in place.
- The program reverses four strings and prints them, and the tests check both parities, the single byte, the unchanged allocation, and the round trip.
- Empty and non-ASCII inputs are not handled: the first panics on the subtraction, and the second is refused by the assertion.

References

- Standard library, `String::as_mut_vec`.
- Standard library, `str::is_ascii`.
- Standard library, `slice::swap`.
- The Rustonomicon, [Working with unsafe](#).

Part X: Reliability

Idempotent Operations

In this chapter

- Problem Statement
- Designing a Solution
- Implementation
- Intuition
- Time and Space Complexity
- Limitations

Source files: `src/bin/idempotent_operation.rs` and `src/bin/idempotent_operation_with_error_propagation.rs`. Run them with `cargo run --bin idempotent_operation` and `cargo run --bin idempotent_operation_with_error_propagation`.

Problem Statement

A client sends a request, the server performs the work, and the response is lost before the client receives it. The client retries. If the server performs the work again, a card is charged twice or an order is placed twice. An operation is *idempotent* when performing it more than once has the same effect as performing it once. The task here is to give an arbitrary fallible operation that property: run it at most once for a given key, and return the same result to every later call that presents the same key.

Designing a Solution

Keep a map from key to the result already produced for that key. When a call arrives, look the key up. On a hit, return a copy of the stored result and do not run the operation. On a miss, run the operation, store the result, and return it. The map sits behind a `Mutex`, so two threads cannot both observe a miss for the same key.

Holding the lock across the operation is the decision that carries the design. It is what makes the second of two concurrent callers wait on the lock, then find the stored result instead of running the operation. The cost is that the lock is held for the whole of the operation, so callers with *different* keys also wait for

one another. Only a successful result is stored: when the operation returns an error, the key is left absent and a later retry runs it again.

The operation is passed as `FnOnce`, because the function may run zero times (on a hit) or one time (on a miss), and never more than once. The error type is `Box<dyn std::error::Error>`, which lets the caller supply any error without the store naming it.

Implementation

```
use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, Box<dyn std::error::Error>>
    where
        F: FnOnce() -> Result<V, Box<dyn std::error::Error>>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned")?;
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let idem = Idempotent::new();

    let charge = || -> Result<String, Box<dyn std::error::Error>> {
        println!("charging card...");
        Ok("txn_42".into())
    };

    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    assert_eq!(idem.execute("order-1", charge)?, "txn_42");
    Ok(())
}
```

The lock is taken first and held until `execute` returns. `store.get(&key)` borrows the key, so the borrowed value is cloned out with `v.clone()` and the borrow ends before `insert` moves the key into the map.

`let v = f()?` is where the operation runs. The `?` returns early on an error, so the map is never written for a failed call. That is the choice that makes the store *at-most-once for successes*: an error is treated as "the operation did not happen", and a retry is allowed to run it again.

`store.insert(key, v.clone())` stores one copy and returns the other. The value is cloned twice in all, once for the map and once for the caller. A version that returned `v` and let the map own a clone, or that used the previous value `insert` returns, would avoid one of the two.

The `K: Clone` bound is present but unused: `get` borrows the key and `insert` moves it, and neither needs a copy. `V: Clone` is required, because both the hit path and the store path copy the value.

The variant with a named error type

The repository holds a second version of the same type, `idempotent_operation_with_error_propagation.rs`. It differs in one way: the error type is given a name, `RuntimeError`, and the alias is used in every signature.

```
use std::collections::HashMap;
use std::hash::Hash;
use std::sync::Mutex;

type RuntimeError = Box<dyn std::error::Error>;

pub struct Idempotent<K, V> {
    store: Mutex<HashMap<K, V>>,
}

impl<K, V> Idempotent<K, V>
where
    K: Eq + Hash + Clone,
    V: Clone,
{
    pub fn new() -> Self {
        Self { store: Mutex::new(HashMap::new()) }
    }

    pub fn execute<F>(&self, key: K, f: F) -> Result<V, RuntimeError>
    where
        F: FnOnce() -> Result<V, RuntimeError>,
    {
        let mut store = self.store.lock().map_err(|_| "lock poisoned")?;
        if let Some(v) = store.get(&key) {
            return Ok(v.clone());
        }
        let v = f()?;
        store.insert(key, v.clone());
        Ok(v)
    }
}

fn main() -> Result<(), RuntimeError> {
    let idem = Idempotent::new();
```

```

let charge = || -> Result<String, RuntimeError> {
    println!("charging card...");
    Ok("txn_42".into())
};

assert_eq!(idem.execute("order-1", charge)?, "txn_42");
assert_eq!(idem.execute("order-1", charge)?, "txn_42");
Ok(())
}

```

The alias names the error in one place, so a change to the error type is a change to one line and the signatures stay short. The behaviour is the same in both files: the early return on an error, the lock held across the operation, and the rule that only successes are stored.

Intuition

Two sequential calls with the same key, tracing the map:

execute("order-1", charge)	store = {}
lock	
get("order-1")	miss
charge()	prints "charging card..." -> "txn_42"
insert("order-1", "txn_42")	store = {order-1: "txn_42"}
unlock	
return "txn_42"	
 execute("order-1", charge)	 store = {order-1: "txn_42"}
lock	
get("order-1")	hit
return "txn_42"	charge() is not called
unlock	
 output:	
charging card...	

The program prints the line once. The two assertions both hold, but they would hold even without caching, because the same closure produces the same value; the evidence that the closure ran once is the single line of output.

Two concurrent calls with the same key show what the lock is for:

thread A	thread B
lock	
get: miss	
charge() (runs)	
	lock (blocks)
insert	
unlock	
	get: hit
	unlock
	return stored value
 charge() ran once; thread B returned the value thread A stored.	

Without the lock held across `charge`, both threads would miss, both would run the operation, and the effect would happen twice.

Time and Space Complexity

Operation	Time	Space
execute, hit	$O(1)$ expected	one clone of the stored value
execute, miss	$O(1)$ expected, plus the cost of <code>f</code>	one map entry and one clone
store	$O(k)$ entries	one entry per distinct key, held for the life of the value

$O(1)$ expected for the map rests on the usual assumption that the hash function distributes keys evenly. The store is linear in the number of distinct keys and is never pruned.

Limitations

The lock is held across the operation. Every call, for every key, is serialised for the duration of `f`. A slow `f` blocks unrelated keys as well as duplicates of its own. An operation that calls back into the same store deadlocks, because the lock is not reentrant.

The store has no bound and no expiry. Entries live until the process exits, so the map grows with the number of distinct keys. A production store needs a time-to-live and an eviction policy, because the set of keys is not bounded by anything the client controls.

The store is in memory. A restart forgets every key, so a retry after a crash runs the operation again. Durability requires persisting the key and the result, or relying on a unique constraint in the datastore that performs the effect.

Only successes are stored. A failed call leaves the key absent, so a retry runs the operation again. For an operation that can partially succeed, that is the wrong default; the record of the attempt has to be written before the effect, not after it.

The error type erases the concrete error. Callers receive `Box<dyn std::error::Error>` and cannot match on the failure without downcasting. The lock-poisoning case is a string, which is the least informative variant in the file.

A panic while the lock is held poisons the mutex. Every later call returns the `"lock poisoned"` error, and the store never recovers. The error is reported rather than panicking the caller, which is the better of the two available choices, but the store is unusable from that point on.

The key is not scoped. A production idempotency key is scoped to a client and an endpoint, so that two callers cannot collide by choosing the same string. This store treats the key as global.

K: Clone is an unused bound. It constrains callers without buying anything. The bound can be removed with no other change to the implementation.

Summary

- A `Mutex<HashMap<K, V>>` gives at-most-once execution per key and a cached result for every later call with that key.
- Holding the lock across the operation is what prevents two concurrent callers from running it twice; the price is that different keys are serialised too.
- Only success is recorded, so failure leaves the key absent and a retry is permitted.
- The store is in memory, unbounded, and without expiry or scope, so it is a faithful model of the pattern rather than a production idempotency layer.

References

- Rust standard library, `Mutex` and `HashMap`.
- Rust standard library, `FnOnce`, for a closure that runs at most once.
- RFC 9110, [HTTP Semantics](#), Section 9.2.2, on idempotent methods.

Closing

References

How to read this list

Sources are grouped by kind, and each entry says where the claim it supports appears. Language and library claims cite the document that specifies the behaviour: the *Rust Reference* for the language, the standard library documentation for a function, and the *Rust Book* for a mechanism. Algorithm claims cite a textbook or the paper that introduced the algorithm. Quotations are attributed to the person who wrote them, with the work they appear in, and the list is in alphabetical order by surname.

Language and library documentation

- **The Rust Book.** The Rust Project Developers. <https://doc.rust-lang.org/book/>. Cited for monomorphisation (Chapter 1), enumerations (Chapter 7), boxes (Chapters 7, 14), `Rc` and `RefCell` (Chapter 16), shadowing and mutability (Chapter 25), and command-line arguments (Chapter 29).
- **The Rust Reference.** The Rust Project Developers. <https://doc.rust-lang.org/reference/>. Cited for names and scopes (Chapter 25).
- **The Rustonomicon.** The Rust Project Developers. <https://doc.rust-lang.org/nomicon/>. Cited for the rules an `unsafe` block relies on (Chapters 24, 33).
- **Rust API Guidelines.** The Rust Project Developers. <https://rust-lang.github.io/api-guidelines/>. Cited for documentation requirements and for the naming of conversion methods.
- **Standard library documentation.** The Rust Project Developers. <https://doc.rust-lang.org/std/>. Individual items are cited in each chapter's reference list, including `HashMap`, `VecDeque`, `BinaryHeap`, `Reverse`, `Rc`, `Weak`, `RefCell`, `Mutex`, `Condvar`, `PoisonError`, `Instant`, `Duration`, `Box`, `Option`, `JoinHandle`, `fs`, `OpenOptions`, `BufReader`, `Path`, `PathBuf`, `ErrorKind`, `String::as_mut_vec`, `thread::scope`, `black_box`, `sort_unstable_by_key`, `clear_poison`, and `try_reserve`.
- **Clippy lint documentation.** The Rust Project Developers. <https://rust-lang.github.io/rust-clippy/master/>. Cited for `new_without_default` (Chapter 15).
- **The `libc` crate.** <https://docs.rs/libc/>. Cited for `getpid` (Chapter 24).

Papers and books

- **Bellman, Richard.** *Dynamic Programming*. Princeton University Press, 1957. The origin of the method used in Chapter 12.
- **Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.** *Introduction to Algorithms*, 4th edition. MIT Press, 2022. Cited for binary search trees (Chapter 8), dynamic programming (Chapters 12, 13), breadth- first search (Chapter 23), priority queues (Chapter 32), and topological sort (Chapter 11).
- **Fredkin, Edward.** "Trie memory". *Communications of the ACM* 3(9), 1960, pages 490–499. The paper that introduced the structure in Chapter 10.
- **Hoare, C. A. R.** "Null References: The Billion Dollar Mistake". Presentation at QCon London, 2009. The source of the phrasing used in Chapter 12 for the choice between a sentinel and `Option`.
- **Kahn, Arthur B.** "Topological sorting of large networks". *Communications of the ACM* 5(11), 1962, pages 558–562. The algorithm in Chapter 11.
- **Knuth, Donald E.** "Structured Programming with go to Statements". *ACM Computing Surveys* 6(4), 1974, pages 261–301. The source of the observation about premature optimisation, including the qualification that summaries of it usually leave out.
- **Knuth, Donald E.** *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, 2nd edition. Addison-Wesley, 1998. Cited for digital searching (Chapter 10).
- **Lamport, Leslie.** "Distribution". Letter, 1987; collected in *Distributed Systems*, 2nd edition, 1993. The source of the sentence about partial failure quoted in Chapter 22.
- **Patterson, David A.** "Latency lags bandwidth". *Communications of the ACM* 47(10), 2004, pages 71–75. The source of the phrasing quoted in Chapter 24.
- **Perlis, Alan J.** "Epigrams on Programming". *ACM SIGPLAN Notices* 17(9), 1982, pages 7–13.
- **Stroustrup, Bjarne.** "Foundations of C++". Keynote, *Proceedings of the 2012 European Conference on Object-Oriented Programming (ECOOP)*, 2012. The source of the two-part statement of the zero-overhead principle quoted at the opening of the preface.
- **Wirth, Niklaus.** "A Plea for Lean Software". *IEEE Computer* 28(2), 1995, pages 64–68.

Quotations in this book

Four sentences are quoted from other people, each in a place where the quotation states something the surrounding chapter then uses.

Where	Quotation	Source
Preface	The two-part statement of the zero-overhead principle	Stroustrup, <i>Foundations of C++</i> , 2012
Chapter 12	Null references as a mistake that has cost a great deal	Hoare, QCon London, 2009
Chapter 22	A distributed system as one in which a machine you did not know about can stop yours	Lamport, "Distribution", 1987
Chapter 24	"Latency lags bandwidth"	Patterson, <i>Communications of the ACM</i> , 2004

Each quotation is short, is attributed in the chapter where it appears, and is quoted from the work named here. Nothing attributed to anyone in this book is a paraphrase presented as a quotation.

Index

A

- **ApiError**, error enumeration carrying data, 19 (page 114)
- **Arc**, shared ownership across threads, 21 (page 126), 22 (page 132), 30 (page 180)
- **args**, the command-line boundary, 29 (page 173)
- **assert!**, refusal of an empty input as a design decision, 21 (page 126)
- **assertion in a destructor-free type**, 13 (page 75)
- **attach and detach**, index-linked list maintenance, 18 (page 107)
- **average, limits of**, a benchmark statistic that hides the distribution, 24 (page 146)

B

- **backtracking**, `result.push(path.clone())`, `path.pop()`, 13 (page 75)
- **BinaryHeap**, a max-heap, so pairs sort by the first field, 6 (page 37), 32 (page 193)
- **binary search**, half-open range and the sorted-half test, 9 (page 55)
- **binary search tree**, ordering invariant, three removal shapes, 8 (page 48)
- **binary tree**, algebraic data type with boxed children, 7 (page 42)
- **black_box**, keeping a measurement alive, 24 (page 146)
- **bounded buffer**, one mutex and two condition variables, 30 (page 180)
- **Box**, one ownership pointer, one allocation, 7 (page 42), 14 (page 81), 15 (page 86)
- **BFS**, flood fill with a queue, 23 (page 139)
- **borrow counter**, `RefCell` and the run-time check, 16 (page 91)

C

- **cache**, two designs, bounded and unbounded memory, 17 (page 101), 18 (page 107)
- **capacity hint**, `with_capacity` and its units, 1 (page 11), 2 (page 17), 13 (page 75)

- **channel**, `mpsc` for jobs and results, 21 (page 126)
- **checked_sub**, the fix for overflow at the target, 1 (page 11)
- **clone**, value copied out of a guard or a map, 16 (page 91), 17 (page 101)
- **closure**, taking a mutable reference as a parameter, 23 (page 139)
- **coin change**, dynamic program over amounts, 12 (page 70)
- **command-line arguments**, `env::args` and the first item, 29 (page 173)
- **condition variable**, a directed wake-up for each state change, 30 (page 180)
- **connected components**, counting them by flood fill, 23 (page 139)
- **copy**, `fs::copy` and what it preserves, 28 (page 169)
- **cost model**, instructions, memory, stack, compilation, 24 (page 146)
- **cycle detection**, a short topological order, 11 (page 64)

D

- **dead_code warning**, the unused second merge function, 5 (page 33), 24 (page 146)
- **Default**, `#[derive(Default)]` and `#[default]` on a variant, 3 (page 23), 8 (page 48), 10 (page 59)
- **destructor**, recursion in derived drop glue, 7 (page 42), 14 (page 81), 15 (page 86), 16 (page 91)
- **directory traversal**, `read_dir` and a recursive walk, 27 (page 164)
- **discriminant**, a `size_of` of one byte for a six-variant enum, 19 (page 114)
- **Display**, the trait a message method should implement, 19 (page 114)
- **drop**, releasing a sender to end a receive loop, 21 (page 126)
- **Duration**, a retry delay carried by an error variant, 19 (page 114)
- **dynamic dispatch**, a trait object and the vtable call, 21 (page 126)

E

- **enum**, states a caller cannot construct illegally, 7 (page 42), 8 (page 48), 19 (page 114)
- **epigraph**, attributed quotations in this book, references (page 16)
- **ErrorKind**, branching on a failure without parsing a message, 28 (page 169)
- **eviction**, least recently used, in two designs, 17 (page 101), 18 (page 107)

- **error type**, variants with data, and matching on them, 19 (page 114)

F

- **f64**, a rate that is neither validated nor reported, 20 (page 121); a median that cannot be an integer, 32 (page 193)
- **file I/O**, `fs::write`, `OpenOptions` and `BufReader`, 26 (page 159)
- **file operations**, `copy`, `rename` and `remove_file`, 28 (page 169)
- **flood fill**, marking on entry, 23 (page 139)
- **format!**, one allocation per call, 19 (page 114), 25 (page 152)

H

- **half-open range**, `[low, high)` as the search invariant, 9 (page 55)
- **hash map**, expected constant time and the assumption behind it, 1 (page 11), 6 (page 37), 10 (page 59)
- **heap**, `BinaryHeap` as a priority queue, 6 (page 37), 32 (page 193)
- **HashSet**, the visited set that a bitmap would replace, 23 (page 139)

I

- **idempotency**, a keyed store that runs an operation at most once, 34 (page 205)
- **in-degree**, the counter Kahn's algorithm waits on, 11 (page 64)
- **index-linked list**, nodes in a vector, links as integers, 18 (page 107)
- **Instant**, the clock a limiter trusts, 20 (page 121); the clock a measurement reads, 31 (page 186)
- **interior mutability**, mutation behind a shared reference, 16 (page 91)
- **invariant**, a sentence true before and after every operation, 3 (page 23), 9 (page 55), 13 (page 75), 32 (page 193)

J

- **JoinHandle::join**, a discarded panic, 21 (page 126), 22 (page 132)

K

- **Kahn's algorithm**, topological sort by in-degree, 11 (page 64)
- **key**, the field a full cache reads back before overwriting a slot, 18 (page 107)

L

- **lifetime**, 'a in the range query, and what it ties together, 8 (page 48)
- **linked list**, reversal with three bindings, 14 (page 81)
- **lock**, held across a clock read, released before the work, 20 (page 121), 21 (page 126), 30 (page 180)
- **LRU**, least recently used, 17 (page 101), 18 (page 107)

M

- **max-heap**, why the top-k result pops the largest pair, 6 (page 37)
- **median, running**, two heaps and a rebalancing rule, 32 (page 193)
- **mem::replace**, moving a value out of a place, 15 (page 86), 8 (page 48)
- **mem::swap**, exchanging two subtrees, 7 (page 42); two bytes, 33 (page 199)
- **Mutex**, protection, and the poison mark, 20 (page 121), 21 (page 126), 22 (page 132), 30 (page 180)
- **monomorphisation**, one copy of the code per type argument, 1 (page 11)
- **mutation**, writing to a binding rather than rebinding it, 25 (page 152)

N

- **new**, and the `Default` implementation Clippy expects, 15 (page 86)
- **None**, the end of a list, not a sentinel, 14 (page 81)

O

- **OpenOptions**, `create` and `append` flags, 26 (page 159)
- **Option**, absence in the type, 1 (page 11), 3 (page 23), 12 (page 70), 14 (page 81), 32 (page 193)
- **overhead**, the boundary cost of a system call, 24 (page 146)

- **overflow**, arithmetic that differs between debug and release, 1 (page 11), 9 (page 55), 13 (page 75)

P

- **panic**, unwinding while a guard is alive, 20 (page 121), 22 (page 132); reversal of a non-ASCII string, 33 (page 199)
- **parallel sum**, borrowed workers under a scope, 31 (page 186)
- **PartialEq**, the minimal bound on a grid cell, 23 (page 139)
- **Path** and **PathBuf**, borrowed and owned paths, 27 (page 164)
- **PoisonError**, the error a poisoned lock returns, 22 (page 132)
- **prefix tree**, see **trie**, 10 (page 59)
- **pruning**, a range query that skips whole subtrees, 8 (page 48)

R

- **Rc**, shared ownership on one thread, 16 (page 91)
- **read_dir**, one **Result** per directory entry, 27 (page 164)
- **Ref** and **Ref::map**, a guard projected to a field, 16 (page 91)
- **RefCell**, mutation with a run-time borrow check, 16 (page 91)
- **recursion**, stack depth proportional to input, 7 (page 42), 8 (page 48), 14 (page 81), 27 (page 164)
- **refill**, tokens computed from elapsed time, 20 (page 121)
- **rename**, an atomic replacement within one filesystem, 28 (page 169)
- **Result**, failure in the type, 19 (page 114), 22 (page 132)
- **Reverse**, flipping a heap's ordering, 32 (page 193)

S

- **Send** and **Sync**, what a single-threaded structure does not implement, 16 (page 91)
- **sentinel**, **usize::MAX** kept inside the table, 12 (page 70)
- **shadowing**, a new binding with an old name, 25 (page 152)
- **sliding window**, a start index that only moves forward, 4 (page 29)
- **sort_unstable_by_key**, ordering intervals by start, 5 (page 33)

- **stack**, frames per call, and the depth they reach, 7 (page 42), 15 (page 86), 27 (page 164)
- **state machine**, legal transitions in one `match`, 19 (page 114)
- **String**, an allocated message on an error path, 19 (page 114)
- **string reversal**, an in-place byte swap under an ASCII precondition, 33 (page 199)
- **subsets**, 2^n results and the clone per result, 13 (page 75)
- **syscall**, the user and kernel boundary, 24 (page 146)

T

- **thread::scope**, workers that borrow the input, 31 (page 186)
- **tie-break**, the ordering the top-k result depends on, 6 (page 37)
- **token bucket**, burst then rate, 20 (page 121)
- **topological sort**, an order in which edges point forward, 11 (page 64)
- **trace**, the state after each step, produced by running the code, 2 (page 17), 5 (page 33), 9 (page 55), 11 (page 64)
- **trait object**, `dyn` and what it costs, 21 (page 126)
- **try_unwrap**, moving a value out of an `Rc`, 16 (page 91)

U

- **unsafe**, the obligation a `SAFETY` comment states, 24 (page 146), 33 (page 199)
- **upgrade**, turning a weak pointer into a strong one, 16 (page 91)

V

- **Vec**, amortised growth and reuse of a slot, 18 (page 107)
- **VecDeque**, constant time at both ends, 11 (page 64), 17 (page 101), 30 (page 180)

W

- **warm-up**, why a benchmark runs the code before timing it, 24 (page 146)
- **Weak**, observing without owning, 16 (page 91)
- **worker pool**, bounded threads, ordered results, 21 (page 126)